

First Principles: Data from Zero

Chapters 1–6: what data is, how computers store and move it, how a business runs on it, numbers without fear, thinking like an analyst, and setting up to learn.

Approved chapters · Version 1

Contents

Part 0 — First Principles: Data from Zero

Chapter 1. What Is Data?

Chapter 2. How Computers Store, Move and Protect Data

Chapter 3. How a Business Runs on Data

Chapter 4. Numbers Without Fear

Chapter 5. Thinking Like an Analyst

Chapter 6. Setting Up to Learn

Part 0 — First Principles: Data from Zero

Part 0 starts from nothing. It assumes no technical knowledge, no math beyond school arithmetic, and no tools. By the end of it, you'll understand what data is, where it lives, how a business creates and uses it, how to work with numbers confidently, how to think through a business question, and you'll have every core tool installed and a plan to keep learning.

Every example uses **Riverstone Supplies**, the fictional manufacturer you'll follow through the whole book, and every number was checked against its practice databases.

Chapter	What you'll be able to do	Time needed
1. What Is Data?	tell data from information and insight; name types and levels of measurement; judge data quality	3–4 hours
2. How Computers Store, Move and Protect Data	work with files and formats from CSV to Parquet; explain databases, the cloud, and APIs; keep data safe	4–5 hours
3. How a Business Runs on Data	follow an order from enquiry to cash; tell bookings from billings from collections; define KPIs; find manual work	3–4 hours
4. Numbers Without Fear	calculate percentages, points, growth, and CAGR; choose the right average; read charts; estimate	4–5 hours
5. Thinking Like an Analyst	turn vague requests into precise questions; build issue trees; test hypotheses; spot bias; decide with data	3–4 hours
6. Setting Up to Learn	install and check the book's tools; read documentation; learn with AI assistants; plan your study	3–4 hours

In total, allow 20–26 hours, including the exercises and projects. After Part 0, Part I (Chapters 7–9) shows the map of data careers, and Part II (Chapters 10–27) builds the analyst's toolkit.

Chapter 1. What Is Data?

Chapter at a glance

You will learn to: explain what data is, and how it differs from information, knowledge, and insight · turn everyday records, like a shop receipt, into rows and columns · name the type of any value (number, text, date, true/false) and spot numbers that aren't really numbers · tell qualitative from quantitative data, and discrete from continuous · use the four levels of measurement to decide which calculations make sense · recognize structured, semi-structured, and unstructured data · read and write metadata · say where data comes from · judge whether a small dataset is fit to use.

Before you start: nothing. This chapter assumes no technical knowledge at all.

Time needed: 3–4 hours, including the exercises and the project.

Tools: a pen and a notebook are enough. If you already have Excel or Google Sheets, you can use it for the project, but you don't need to.

Practice data: your own week of spending, plus small examples from Riverstone Supplies, the fictional company you'll follow through this book.

Why this matters

Every job in this book, from a junior analyst's first report to an architect's design for a whole company, is built on one raw material: **data**. Tools change every few years. Spreadsheets gave way to databases, databases to cloud warehouses, and now AI assistants can write the formulas for you. The raw material stays the same.

That's why this chapter comes before any tool. People who skip it can learn the buttons in Excel or the words in SQL and still produce reports that are confidently wrong: an "average" of PIN codes, a customer counted twice because their name was typed in capitals, a sales total that silently left out every blank row. These mistakes aren't about software. They come from not asking basic questions about the data first: *What does each value mean? What kind of value is it? What calculations does it allow? Where did it come from? Can I trust it?*

By the end of this chapter you'll ask those questions automatically. It's a small habit, and it's the one that experienced analysts are quietly relying on every time they say, "Wait, that number doesn't look right."

In plain English

Picture the owner of a small grocery shop in your neighborhood. Next to the till is a thick notebook. Every time a regular customer buys on credit, the owner writes a line: the date, the customer's name, what they bought, and how much they owe. When the customer pays, another line goes in.

- Each **line** in that notebook is a small, recorded fact. That's **data**.
- At the end of the month, the owner adds up what each customer owes and sees that Mrs. Joshi owes ₹2,400. That total is **information**: data organized to answer a question.
- Over the years, the owner notices that customers who owe more than ₹3,000 often stop coming, because they feel embarrassed. That's **knowledge**: an understanding of *why* things happen.
- So the owner decides to send a friendly reminder whenever anyone's balance passes ₹2,500, before it becomes awkward. That's **insight** turned into action.

Now notice two more things about the notebook. First, the owner writes every line **the same way**: date first, then name, then items, then amount. That consistent layout is what makes adding things up possible. Second, some lines are hard to use: a smudged amount, a name written as "Joshi aunty" one day and "Mrs. Joshi" the next. That's a **data quality** problem, and every business, from this shop to the largest bank, has the same problem at a bigger scale.

Everything in this chapter is a version of that notebook.

1.1 What data actually is

Data is a recorded observation about the world. Something happened, someone or something noticed it, and a note was made that can be looked at later.

Three parts of that definition matter:

1. **Observation.** Data describes something: a sale, a temperature, a step, a click, an opinion.
2. **Recorded.** A thought in your head isn't data until it's written, typed, photographed, or saved. The recording is what lets other people, and computers, use it.
3. **Later use.** Data is kept because someone expects to look at it again: to add it up, compare it, check it, or prove something.

You create and use data all day, usually without calling it that:

Everyday thing	The data in it
A shop receipt	what you bought, how many, at what price, when, and how you paid
Your phone's step counter	a count of steps for each hour and each day
A train timetable	train numbers, stations, and arrival and departure times
A class attendance register	each student, each date, present or absent
A food-delivery app	the restaurant, the items, the delivery address, the time taken, your rating
A photo on your phone	the picture itself, plus the date, time, and often the place it was taken

In the singular, one recorded value is sometimes called a **datum**, but almost everyone, including this book, simply says "data" for one value or many.

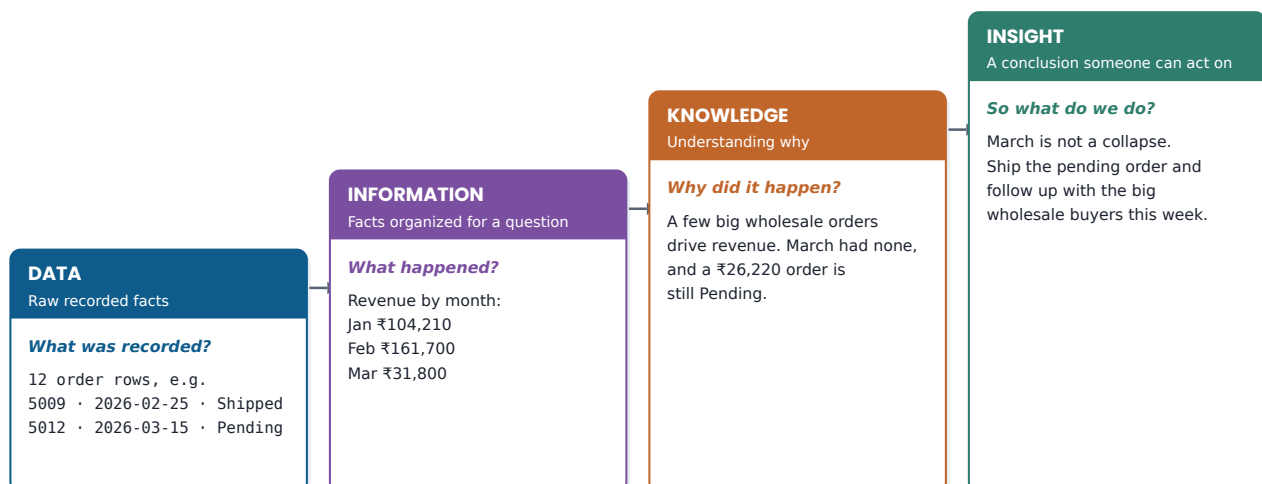
Try it. Look at your phone for thirty seconds. Name five kinds of data it recorded about you today without you typing anything. (Some ideas: steps, screen time, battery use, location, photos, missed calls.)

1.2 From data to information, knowledge, and insight

People often use *data* and *information* as if they mean the same thing. In data work, the difference matters, because your job is to move up from one to the next. A common way to picture it is a four-step ladder, sometimes called the **DIKW** model (data, information, knowledge, wisdom). This book uses *insight* for the top step, because that's the word businesses use.

Step	What it is	The question it answers
Data	raw recorded facts	What was recorded?
Information	data organized, summarized, or put in context	What happened?
Knowledge	an understanding of patterns and causes	Why did it happen?
Insight	a conclusion that points to an action	So what should we do?

Here's the ladder with real numbers from **Riverstone Supplies**, a fictional company that sells storage boxes, kitchenware, and industrial crates to shops, hotels, and wholesalers across India. You'll use its data throughout the book.



Each step up needs more context and more thinking, and is worth more to the business.

Figure 1.1 — The same twelve orders at four levels of usefulness.

- **Data.** Riverstone's system holds twelve orders from January to March 2026. Each has an order number, a customer, a date, and a status such as *Delivered*, *Shipped*, or *Pending*. On their own,

twelve rows tell a manager nothing.

- **Information.** Add up the value of delivered and shipped orders for each month: January ₹104,210, February ₹161,700, March ₹31,800. Now there's a clear message: March fell by 80% compared with February.
- **Knowledge.** Look closer and you learn *why*. Most of Riverstone's revenue comes from a few big wholesale orders, and none were placed in March. One more March order, worth ₹26,220, exists but is still *Pending*, so it isn't counted yet.
- **Insight.** So March is not the collapse the headline number suggests. The useful actions are to ship the pending order before the quarter closes, and to call the big wholesale buyers this week to find out why they haven't reordered.

Notice what made each step possible. Information needed a **question** ("how much did we sell each month?") and a **rule** ("count only delivered and shipped orders"). Knowledge needed a closer look at **detail and context**. Insight needed **judgment about the business**. Tools help most with the first step; your value as an analyst grows as you climb.

Watch out: a number is not an insight. "March revenue: ₹31,800" is information. Reports that stop there leave the reader to guess what it means, and people often guess wrong. Chapter 24 shows how to write the "so what" that turns a number into a decision.

1.3 Records, fields, and datasets

Data becomes easy to use when it's arranged in a **table**: a grid of rows and columns. Almost every tool in this book, from spreadsheets to databases to Python, works with tables.

Here's a receipt from a neighborhood store, and the same receipt as a table.

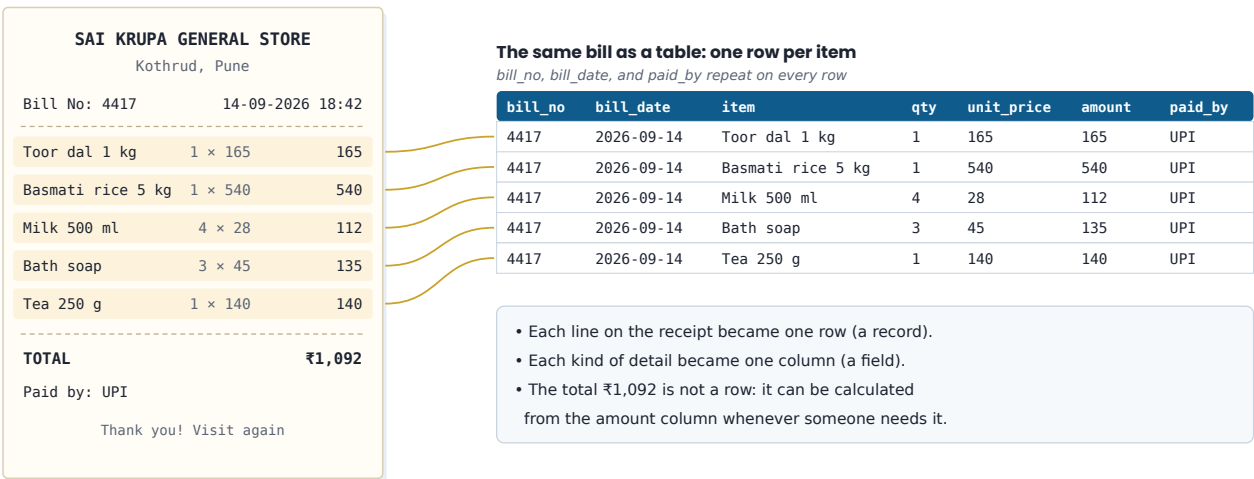


Figure 1.2 — Each line on the receipt becomes a row; each kind of detail becomes a column.

The vocabulary:

- A **row** (also called a **record**) holds the details of one thing. Here, one row is **one item on one bill**.
- A **column** (also called a **field**, **variable**, or **attribute**) holds one kind of detail for every row: `item`, `qty`, `unit_price`.
- A **value** is what sits where a row and a column meet: `4` in the `qty` column of the milk row.
- A **dataset** is a collection of related data. It might be one table (every bill from the shop this year) or several tables that belong together.

Two ideas from this small example will stay with you for the whole book.

1. Decide what one row means. In Figure 1.2, one row is one *item* on a bill, not one *bill*. That's why `bill_no`, `bill_date`, and `paid_by` are repeated on all five rows: each row needs to stand on its own. The level of detail one row represents is called the **grain** of a table. When two people disagree about a number, they're very often counting at different grains: "5 sales" (items) versus "1 sale" (bill). Chapter 12 shows how getting the grain wrong makes totals double-count.

2. Store the ingredients, calculate the results. The receipt prints a total of ₹1,092, but the table has no "total" row. You can always calculate it: $165 + 540 + 112 + 135 + 140 = ₹1,092$. A total typed in as a row would get added in again by anyone who sums the column, and it would be wrong the moment someone corrected a line. Keep the detailed facts; calculate summaries when you need them.

Two more habits make a table easy to work with:

- **One kind of fact per column.** A column called `item` shouldn't also hold "Toor dal 1 kg, 2 packets" with the quantity mixed into the text. Split it: `item` and `qty`.
- **One value per cell.** "UPI + cash" in one cell can't be added up. If a bill was split between two payment methods, that's two payment rows.

Try it. Take any receipt from your wallet or your email. Write its first three lines as rows of a table with the same seven columns as Figure 1.2. What is the grain of your table?

1.4 Kinds of values: data types

Every column holds one **type** of value. The type tells you, and the computer, what you can do with it. Four types cover almost everything you'll meet:

Type	Holds	Examples	You can...
Number	amounts and counts, whole or with decimals	<code>4</code> , <code>28</code> , <code>1450.50</code>	add, average, compare
Text (also called a string)	letters, words, codes	Toor dal 1 kg , UPI , Pune	sort alphabetically, search, group
Date and time	a moment or a calendar day	2026-09-14 , 18:42	sort in time order, find days between, group by month
True/false (also called Boolean)	yes-or-no facts	TRUE , FALSE	count how many are true, filter

Getting the type right matters more than beginners expect.

Numbers that aren't numbers. A PIN code like `411038` is made of digits, but you would never add two PIN codes together or calculate an average one. The same goes for phone numbers, employee codes, bank account numbers, and bill numbers. They're **identifiers** or **codes**, and they should be treated as text. Store a phone number as a number and a spreadsheet may drop its leading zero (`022...` becomes `22...`) or display a long number as `9.82E+09` . **Rule of thumb: if you'd never do arithmetic with it, it's text.**

Dates stored as text. `03/04/2026` means 3 April in India and the UK, but 4 March in the United States. A computer that reads it as text can't tell, and it will sort `10/01/2026` before `9/01/2026` , because the character `1` comes before `9` . The safest way to write a date is **year-month-day**, `2026-04-03` , which sorts correctly and means the same thing everywhere. It's an international standard (ISO 8601), and it's how dates are written in this book's data.

Missing values. Sometimes a value simply wasn't recorded. A blank is not the same as zero: a blank "discount" might mean *no discount* or *nobody wrote it down*. Databases use a special marker, **NULL**, for "unknown", and Chapter 12 shows how it trips up calculations. For now, whenever you see a blank, ask: **does this mean zero, "not applicable", or "unknown"?**

Watch out: the type you see isn't always the type that's stored. A spreadsheet can show `14-09-2026` while storing it as text, and then refuse to sort or group it by month. Chapter 10 shows how to check what a cell really contains.

1.5 Qualitative and quantitative data

A second way to describe data is by what it measures.

Quantitative data is measured or counted in numbers, and the numbers mean an amount: 4 packets of milk, ₹540, 12,000 steps, 38.5 °C.

Qualitative data (also called **categorical data**) describes a quality or puts things into groups: the payment method (UPI, card, cash), a city, a product category, a customer's comment, "satisfied" or "not satisfied".

Quantitative data splits once more:

- **Discrete** data can only take separate, countable values, usually whole numbers: the number of orders, children, or items in a cart. You can have 3 orders or 4, but not 3.7.
- **Continuous** data can take any value in a range, limited only by how precisely you measure: weight, height, time taken, temperature. A parcel can weigh 2.4 kg, 2.43 kg, or 2.4318 kg.

Money is technically discrete (you can't pay a fraction of a paisa), but because it has so many possible values, it's usually treated as continuous in analysis.

Column from the receipt	Qualitative or quantitative	If quantitative
item	qualitative	—
qty	quantitative	discrete
unit_price	quantitative	continuous (in practice)
paid_by	qualitative	—
bill_no	qualitative (it's a label, not an amount)	—

This isn't just labeling. It decides which **chart** fits (a bar chart for categories, a histogram for continuous amounts, Chapter 15) and which **summary** makes sense (a count of each payment method, but an average of prices).

1.6 Levels of measurement: which math is allowed

Not all numbers support the same calculations. A useful way to think about this is the four **levels of measurement**, described by the psychologist S. S. Stevens in the 1940s and still taught in every statistics course. Each level allows everything the level above it allows, plus something new.

Nominal Level 1	Names or labels, no order e.g. segment, city, payment method	count · most common value (mode)
Ordinal Level 2	Order matters, gaps are unknown e.g. T-shirt size, rating 1–5, High/Medium/Low	+ rank · median · "higher than"
Interval Level 3	Equal gaps, but no true zero e.g. temperature in °C, calendar dates	+ differences · mean
Ratio Level 4	Equal gaps and a true zero e.g. revenue, quantity, weight, age	+ "twice as much" · % change

Each level allows everything above it, plus something new. Use only the maths your level allows.

Figure 1.3 — From labels to true amounts: each level unlocks more calculations.

1. Nominal: names and labels. The values are categories with no natural order: city, product category, payment method, customer segment. You can **count** how many are in each group and find the most common one (the **mode**). You can't sort them in any meaningful way. Even when nominal data uses digits, like PIN codes or jersey numbers, the digits are just labels.

2. Ordinal: order matters, but the gaps don't. The values have a clear order, but the distance between them isn't defined: T-shirt sizes (S, M, L, XL), priority (Low, Medium, High), exam rank, or a satisfaction rating from 1 to 5. You can say one is **higher** than another and find the **median** (the middle value). But the gap from "Poor" to "Average" isn't necessarily the same as the gap from "Good" to "Excellent".

3. Interval: equal gaps, but no true zero. The distance between values is meaningful, but zero doesn't mean "none". Temperature in degrees Celsius is the classic example, and calendar dates are another. The **difference** between two values makes sense (it's 20 degrees warmer; the invoice is 30 days late), and so does the mean. But **ratios** don't: 0 °C isn't "no temperature", so 40 °C is not "twice as hot" as 20 °C.

4. Ratio: equal gaps and a true zero. Zero means *none* of it: revenue, quantity, weight, distance, age, time taken. Every calculation works here, including "twice as much" and percentage change. ₹2,000 of sales really is twice ₹1,000, and ₹0 really means no sales.

Most business amounts are ratio data, so most of the time every calculation is allowed. The levels matter at the edges, and those edges are where reports go wrong.

Worked example 1: the average rating that hides the story

Riverstone asks customers to rate two delivery partners from 1 (very poor) to 5 (excellent). Each partner receives five ratings:

Partner	Ratings	Mean	Median
Swift Movers	3, 3, 3, 3, 3	3.0	3
Rapid Wheels	5, 5, 5, 1, 1	3.4	5

Check the means by hand: $15 \div 5 = 3.0$, and $17 \div 5 = 3.4$. ✓

On the mean, Rapid Wheels looks slightly better. But look at the ratings themselves. Swift Movers is consistently average. Rapid Wheels delights most customers and badly fails two out of five, which, for a supplier delivering to hotels, might mean a banquet with no plates. The mean treats the gap between 1 and 5 as four equal steps, which a rating scale doesn't promise. For ordinal data, the **median** and the **count in each category** ("two of five customers gave a 1") are more honest summaries. Chapter 21 goes deeper into choosing averages.

Worked example 2: twice as hot?

Riverstone's warehouse was 20 °C in the morning and 40 °C in the afternoon. A report says, "The warehouse was twice as hot in the afternoon." It wasn't. Celsius has no true zero, so ratios don't work. Temperature *does* have a true zero on the Kelvin scale, which scientists use (0 K is absolute zero). Converting: 20 °C is 293.15 K and 40 °C is 313.15 K, and $313.15 \div 293.15 = 1.068$. In real terms the afternoon was about 7% hotter, not 100%. The accurate sentence is simpler anyway: "The temperature rose by 20 degrees."

Worked example 3: dates

An invoice dated 10 March and due on 9 April: you can subtract the dates to get 30 days, a perfectly meaningful difference. But "20 March is twice 10 March" is nonsense. Dates are interval data.

Watch out: averaging codes. Put a column of PIN codes or employee IDs into a spreadsheet and it will happily calculate their average. The result is meaningless. Before you summarize any column, ask which level of measurement it is.

1.7 Structured, semi-structured, and unstructured data

The last way to describe data is by its **shape**: how organized it is when you receive it. Here's the same Riverstone order, for 20 storage boxes and 50 water bottles, in three shapes.

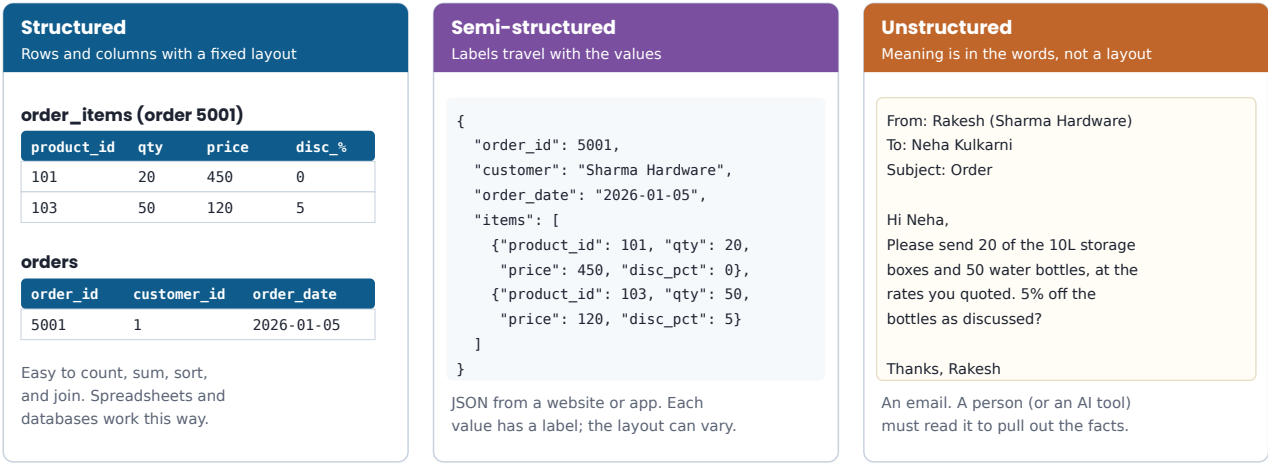


Figure 1.4 — Same facts, three shapes. The less structure, the more work before you can count anything.

Structured data fits a fixed layout of rows and columns, where every row has the same fields and every column has one type. Sales tables, bank statements, attendance registers, and stock lists are structured. It's the easiest to add up, sort, filter, and combine, and it's what spreadsheets and databases are built for. Much of a data analyst's day is spent with structured data.

Semi-structured data has labels that travel with the values, but no fixed table layout. The most common format is **JSON** (say "jay-son"), which websites and apps use to send data to each other. In the middle panel of Figure 1.4, every value has a name ("qty": 20), and the items list can hold one item or fifty. Two orders don't have to have exactly the same fields. Other examples are XML files and the logs that apps write. Semi-structured data is easy for computers to read, and usually needs a step of **flattening** into tables before analysis. Chapter 2 introduces these formats, and Chapter 18 shows how to flatten them in Python.

Unstructured data has no predefined layout at all. The meaning is in the words, pixels, or sounds: emails, WhatsApp messages, PDF contracts, product photos, call recordings, customer reviews. A person can read Rakesh's email in Figure 1.4 and understand the order instantly. A computer has to work out that "the 10L storage boxes" means product 101, and that "the rates you quoted" means ₹450 and ₹120. Most of the information inside organizations is unstructured, and for a long time most of it went unused. Modern AI tools, which you'll meet in Part VI, have made it much easier to pull structured facts out of unstructured text, and that's one of the big changes in data work in recent years.

Shape	Examples	Where you'll work with it
Structured	spreadsheets, database tables, CSV files	Chapters 10–13 and most of Part II
Semi-structured	JSON, XML, app and website logs	Chapters 2, 18, and Part V

Shape	Examples	Where you'll work with it
Unstructured	emails, PDFs, images, audio, free-text reviews	Chapters 41, 55, and 58

Real-life example: the order that arrives by email. In many companies, including plenty of manufacturers and distributors, customers still send orders by email or WhatsApp, and someone re-types them into the billing system. Each re-typing is a chance for a mistake: 50 bottles becomes 500, or the discount is missed. Turning that unstructured message into a structured order automatically is a classic automation project, and you'll build one in Chapter 58.

1.8 Metadata: data about data

Metadata is data that describes other data. Your phone already shows you plenty:

- A photo's metadata includes the date, time, camera model, and often the GPS location where it was taken.
- A file's metadata includes its name, size, type, and when it was last changed.
- A song's metadata includes the title, artist, album, and length.

In data work, the most important kind of metadata is a **data dictionary**: a short document that explains what each column in a dataset means. Here's one for the receipt table from Figure 1.2:

Column	Type	Meaning	Allowed values or format	Can be blank?
bill_no	text	the number printed on the bill	digits only	no
bill_date	date	the date the bill was printed	YYYY-MM-DD	no
item	text	the product name as printed	any text	no
qty	number (whole)	how many units were bought	1 or more	no
unit_price	number (₹, 2 decimals)	price of one unit, before any discount	more than 0	no
amount	number (₹, 2 decimals)	qty × unit_price	more than 0	no
paid_by	text	how the bill was paid	UPI, Card, Cash	no

It looks like a small thing. It's one of the most valuable things a data team produces. Without it, people guess: Is *amount* before or after discount? Does "date" mean the order date or the delivery date? Is the price in rupees or thousands of rupees? Guesses become arguments, and arguments become two departments reporting different numbers for the same month. Other useful metadata includes **where the data came from**, **when it was last updated**, and **who owns it**, meaning who to ask when something looks wrong.

Try it. Pick one column from any table you use at work or college. Could a new colleague understand exactly what it means from its name alone? If not, write the one-line meaning they'd need.

1.9 Where data comes from

Every piece of data was created by someone or something. Knowing the source tells you a lot about how far to trust it.

Source	How it's created	Examples	Typical problems
People	typed, written, selected, or spoken	order forms, surveys, a salesperson updating a CRM, the shop owner's notebook	typos, blanks, inconsistent spelling, fields filled in "just to get past the screen"
Machines and sensors	measured automatically	step counters, temperature sensors on a production line, GPS in delivery vans, electricity meters	faulty or drifting sensors, gaps when a device goes offline, huge volumes
Business systems	recorded as a side effect of doing work	billing software, a website's shopping cart, a bank's payment system, attendance swipes	only as good as the process around them; changes when the system is upgraded
Outside sources	collected by someone else	government statistics, weather data, market prices, data bought from partners	different definitions, delays, unclear collection methods

Two more distinctions you'll hear:

- **Primary data** is collected by you or your organization for your own purpose, like Riverstone's order records or a survey you run. **Secondary data** was collected by someone else for another purpose, like census figures. Secondary data is quicker to get, but you have to check that its definitions match your question.
- **First-party data** is what an organization collects directly from its own customers and operations. **Third-party data** is bought or obtained from outside. The terms come up often in marketing.

Notice where the problems cluster. **Whenever a person types data by hand, and especially when the same data is re-typed from one place into another, errors creep in.** Data captured automatically, at the moment something happens, is usually more complete and more consistent. That single observation explains why so much of a data team's work, and a whole thread of this book from Chapter 3 to the architecture chapters, is about capturing data once, at the source, and letting it flow automatically to reports, dashboards, and other systems.

Watch out: personal data. Some data describes people: names, phone numbers, addresses, health details, salaries, locations. Most countries now have laws about how it may be collected, stored, and shared, including India's Digital Personal Data Protection Act, 2023, and the European Union's GDPR. As a beginner, follow two rules: never copy personal data from work to your own devices or accounts, and use only what you need for the task. Chapter 64 covers privacy and governance properly.

1.10 Data quality in one page

Data is **fit for use** when it's good enough for the decision it supports. A rough count of website visitors can be a little off; the amount on a tax invoice can't. Data quality is usually judged on six dimensions:

Dimension	Question to ask	Example of a problem
Accuracy	Does the value match reality?	a signup date in the year 2062
Completeness	Is anything missing?	a customer with no city
Consistency	Is the same thing recorded the same way everywhere?	"Mumbai", "mumbai", and "Bombay"
Validity	Does the value follow the agreed format and rules?	an email address with no @
Uniqueness	Is each real thing recorded only once?	the same customer entered twice
Timeliness	Is it up to date for the decision?	a stock report from last month used to promise delivery today

Here's a small extract from a customer list that was kept by hand in a spreadsheet for a few months. Look at it for a minute before reading on.

customer_name	city	email	signup_date
Sharma Hardware	Mumbai	sharma.hardware@example.com	2025-11-04
Metro Mart	mumbai	metromart@example.com	2026-01-22
METRO MART	Mumbai	METROMART@EXAMPLE.COM	22/01/2026
Coastal Foods	Chennai	coastalfoods.example.com	2025-10-20
Sunrise Caterers			2026-02-03
Western Logistics	Bombay	westernlogistics@example.com	2062-04-01

There are at least six problems, one or more for every dimension except timeliness:

- Uniqueness.** Metro Mart appears twice, once in capitals. It's the same customer, with the same email address in different letters.
- Consistency.** Mumbai is written three ways: "Mumbai", "mumbai", and "Bombay", the city's former name. (The third row also has an invisible space after "Mumbai", which you can't see here but a computer can.)
- Validity.** The third row's date is written 22/01/2026 while every other row uses year-month-day.

4. **Validity.** Coastal Foods' email address has no @ , so no email will ever reach it.
5. **Completeness.** Sunrise Caterers has no city and no email.
6. **Accuracy.** Western Logistics signed up on 1 April **2062**, forty years in the future. Someone almost certainly meant 2026.

Now try a simple question: **how many customers are in Mumbai?** A computer that matches the exact text "Mumbai" finds **one**: Sharma Hardware. ("mumbai" has a small m, "Bombay" is different text, and the third row's "Mumbai" has a trailing space.) The real answer is **three different customers**: Sharma Hardware, Metro Mart, and Western Logistics. And for Sunrise Caterers, the honest answer is "unknown".

Nothing in that table looks dramatic. Each problem is one small slip by a busy person. Together they turn a simple question into a wrong answer, and nobody sees an error message. **This is why analysts check data before they trust it.** Chapter 14 teaches how to find and fix these problems at scale, and Chapter 47 shows how data teams catch them automatically before a report goes out.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Treating codes as numbers	an "average PIN code", phone numbers showing as 9.82E+09 , lost leading zeros	Store identifiers as text; ask "would I ever do arithmetic with this?"
Not deciding what one row means	two people report different counts for the same thing	State the grain: "one row = one item on one bill"
Storing totals as rows	column sums come out double	Keep the detail; calculate totals when needed
Treating a blank as zero, or zero as blank	totals or averages silently wrong	Ask whether a blank means zero, not applicable, or unknown
Writing dates in local formats	3 April read as 4 March; months sort in the wrong order	Use year-month-day: 2026-04-03
Averaging ordinal ratings without looking at the spread	a polarizing supplier looks "above average"	Report the median and the count in each category
Saying "twice as much" for interval data	"twice as hot", "twice as late in the month"	Describe the difference instead: "20 degrees warmer"
Stopping at information	a report full of numbers that nobody acts on	Add the "why" and the "so what"
Trusting data because it's in a system	wrong counts from duplicates, spelling variations, and typos	Check a small sample against the six quality dimensions first
Using a column without knowing its meaning	"amount" assumed to include tax when it doesn't	Read, or write, the data dictionary

In the real world: Meera's first week

Meera Iyer has just joined Riverstone Supplies as a sales coordinator. On her second day, Anita Rao, the Sales Head, stops by her desk. "We're planning a customer meet in Mumbai next month. How many customers do we have there? I need the number by lunch."

Meera opens the shared customer spreadsheet. It has 240 rows. She filters the city column for "Mumbai" and gets 31. It would be easy to send "31" and move on. Instead, she spends twenty minutes using what she learned in this chapter.

1. What does one row mean? She scrolls and notices some customers appear more than once. Asking around, she learns that the sales team adds a new row whenever a customer opens a new branch. So one row isn't one customer; it's one *customer location*. Anita wants to invite customers, not branches.

2. Is the city column consistent? She sorts the column and finds "Mumbai", "mumbai", "Bombay", "Mumbai " with a trailing space, "Navi Mumbai", "Thane", and "Mum". She groups the obvious variations of Mumbai together, and notes that Navi Mumbai and Thane are separate cities in the same metropolitan region. Whether to invite them is a business question, not a spreadsheet question.

3. What's missing? Eleven rows have no city at all. She checks their addresses, which are in a different column, and finds that four are in Mumbai.

4. Can the data be trusted? She spots two customers marked "closed" in a notes column.

At 12:30 she sends this:

"We have **38 active customers with locations in Mumbai** (44 locations in total). The filter on 'Mumbai' alone shows 31, because the city is spelled several ways and some rows have no city. Another **9 customers are in Navi Mumbai and Thane**; should we invite them too? I've excluded 2 customers marked closed. I've also listed the spelling issues I found, in case we want to clean up the sheet."

Look at what Anita received: a number she can use, the reason it differs from the "obvious" number, a decision only she can make, and a small fix that will help everyone next time. None of it needed an advanced tool. It needed Meera to ask *what does a row mean, what does each value mean, and can I trust it*. That's the whole of this chapter, applied in twenty minutes.

Tools

- **A notebook and pen.** Enough for every exercise in this chapter. Sketching a table by hand is still one of the best ways to think about data.
- **Excel or Google Sheets** (optional). Chapter 10 teaches both from the beginning. If you already have either, use it for the project; Google Sheets is free with a Google account.
- **Your phone.** It's full of data about you: steps, screen time, photos, payments. It's the most convenient practice dataset you own.

Chapter 6 walks you through installing everything else the book uses.

The project: a week of your own spending

Goal: collect a real dataset, describe it the way a data professional would, and turn it into one piece of insight about your own life.

Step 1. Collect. For seven days, record every payment you make: cash, card, and UPI. Write one row per payment with these columns:

Column	What to write
date	year-month-day, e.g. 2026-09-07
time	24-hour clock, e.g. 19:30
item	what you paid for
category	Food, Transport, Groceries, Bills, Entertainment, Shopping, Health, or Other
amount_rs	the amount in rupees (or your currency)
payment_method	UPI, Card, or Cash
shop	where you paid
necessary	Yes or No: would you have been fine without it?

Here's how the first three days looked for Kavya, a college student in Mumbai:

date	time	item	category	amount_rs	payment_method	shop	necessary
2026-09-07	08:40	Tea and poha stall	Food	60	UPI	Station	No
2026-09-07	09:15	Metro card top-up	Transport	500	UPI	Metro station	Yes
2026-09-07	19:30	Vegetables	Groceries	240	Cash	Local market	Yes
2026-09-08	13:05	Lunch thali	Food	180	Card	Canteen	Yes
2026-09-08	21:10	Movie ticket	Entertainment	350	UPI	Online	No
2026-09-09	08:45	Tea and poha stall	Food	60	UPI	Station	No
2026-09-09	18:20	Mobile recharge	Bills	299	UPI	Online	Yes
2026-09-09	20:00	Auto rickshaw	Transport	90	Cash	Street	Yes

Step 2. Describe the dataset. Write down:

- the **grain** (what one row means);
- for every column: its **type**, whether it's **qualitative or quantitative** (and discrete or continuous), and its **level of measurement**;

- a **data dictionary**, using the table in section 1.8 as a model;
- the **source** of the data, and which rows you're least sure about (a cash payment you only remembered the next day, for example).

Step 3. Check the quality. Go through the six dimensions. Did you miss any payments? Did you write a category two different ways ("Food" and "Snacks")? Is every date in the same format?

Step 4. Climb the ladder. Answer these, by hand or in a spreadsheet:

1. *Information:* What did you spend in total, and in each category? Which payment method did you use most, by amount?
2. *Knowledge:* What explains your biggest category? Is it one large payment or many small ones?
3. *Insight:* What is one specific change you'd make next week, and how much would it save?

For Kavya's three days: she spent ₹1,779 in total, about ₹593 a day. Transport was her largest category at ₹590, mostly one metro card top-up, which will last her weeks, so it isn't really a daily cost. UPI covered ₹1,269, or 71.3% of her spending. She marked ₹470 (26.4%) as not necessary, and ₹120 of that was two identical breakfasts at the station stall. Her insight: *"Carrying breakfast from home three days a week would save about ₹180 a week, and the metro top-up shouldn't count as three days of spending."*

Stretch goals:

- Keep the log for a month and compare weeks.
- Add a `mood` column (happy, neutral, stressed) and see whether your "not necessary" spending changes with it. What level of measurement is `mood`?
- Download a month of your UPI or bank statement and compare it with your hand-kept log. Which payments did you forget to write down? That gap is a completeness problem, and a very common one.

You've got it when...

- ☐ I can explain the difference between data, information, knowledge, and insight with an example from my own life or work.
 - ☐ I can turn a receipt, register, or form into a table and state its grain.
 - ☐ I name a column's type before I use it, and I treat codes and phone numbers as text.
 - ☐ I can say whether a column is qualitative or quantitative, and which level of measurement it is.
 - ☐ I don't average ratings or codes without thinking, and I don't say "twice as hot".
 - ☐ I can tell structured, semi-structured, and unstructured data apart.
 - ☐ I can write a data dictionary for a small table.
 - ☐ I check a small dataset against the six quality dimensions before trusting a number from it.
 - ☐ I've logged a week of my own spending and turned it into one insight.
-

Recap

- **Data** is a recorded observation. **Information** is data organized to answer a question. **Knowledge** explains why. **Insight** points to an action. Analysts earn their value by climbing that ladder.
 - Data is easiest to use in **tables: rows** (records), **columns** (fields), and **values**. Always decide what one row means: the **grain**.
 - Store detailed facts and **calculate** totals, rather than typing totals into the data.
 - The four common **data types** are number, text, date/time, and true/false. Codes that look like numbers are text. Write dates as year-month-day. A blank isn't automatically zero.
 - **Quantitative** data measures amounts (discrete or continuous); **qualitative** data describes categories.
 - The **levels of measurement**, nominal, ordinal, interval, and ratio, decide which calculations make sense.
 - Data comes in three shapes: **structured** (tables), **semi-structured** (JSON, logs), and **unstructured** (emails, PDFs, images).
 - **Metadata** describes data. A **data dictionary** says what every column means, and saves hours of argument.
 - Data comes from **people, machines, business systems, and outside sources**. Every manual re-typing adds errors, which is why capturing data once, at the source, matters.
 - **Data quality** is judged on accuracy, completeness, consistency, validity, uniqueness, and timeliness. Check before you trust.
-

Practice exercises

Warm-up

1. Which of these are data? For each "no", say what would make it data. (a) A train timetable on a station wall. (b) A tune you're humming. (c) The number of likes on a post. (d) A thought that you should call your aunt. (e) A doctor's handwritten prescription.
2. Label each statement as data, information, knowledge, or insight: (a) "Order 5012 was placed on 15 March and is Pending." (b) "We should give customers who haven't ordered in 60 days a call before the festive season." (c) "Riverstone sold ₹161,700 in February." (d) "Hotels order less during the monsoon, because fewer events are held."
3. Give the data type (number, text, date/time, or true/false) of each: an Aadhaar-style ID number, a delivery date, the weight of a parcel, whether an invoice is paid, a customer's PIN code, the number of items in a cart.

Core

4. State the level of measurement (nominal, ordinal, interval, or ratio) of each: T-shirt size, monthly revenue, customer segment, temperature in °C, a student's rank in class, the number of years a customer has been with Riverstone, a calendar year such as 2026, and a star rating from 1 to 5.

5. Classify each as structured, semi-structured, or unstructured: a CSV file of bank transactions, a recorded customer-support phone call, the JSON a weather app receives, a scanned PDF invoice, an Excel sheet of attendance with one row per student per day, and a set of Google reviews.
6. Which of these calculations make sense? Explain each in one sentence. (a) The average PIN code of your customers. (b) The median T-shirt size sold. (c) "Revenue in March was 20% lower than in February." (d) "15 March is 50% later than 10 March." (e) The most common payment method.
7. Write a data dictionary entry (type, meaning, allowed values, can it be blank) for these four columns of an attendance register: `student_id`, `class_date`, `status`, `minutes_late`.

Stretch

8. Find every data quality problem in this extract, and name the dimension each one belongs to:

order_id	customer_name	order_date	quantity	unit_price
7001	Green Leaf	2026-04-02	10	450.00
7002	Patel Kitchen	2026-04-02	-5	780.00
7002	Patel Kitchen	2026-04-02	-5	780.00
7003	Blue Bay Cafe	2026-13-01	20	
7004	blue bay cafe	2026-04-05	15	120.00

9. Using the delivery-partner ratings from section 1.6 (Swift Movers: 3, 3, 3, 3, 3; Rapid Wheels: 5, 5, 5, 1, 1), Riverstone's operations manager asks, "Which partner should we use for hotel banquets?" Write a three-sentence answer that uses the right summaries for ordinal data.
10. Kavya's log has a `necessary` column (Yes/No) and an `amount_rs` column. What is the level of measurement of each? Name two summaries you could calculate from `necessary` and one you couldn't.

Think about it (no calculation needed)

1. Your phone's step counter says you walked 12,000 steps yesterday. How could that number be wrong? What metadata would help you decide how much to trust it?
2. The sales head says, "Our customer data is bad." You've been asked to fix it. What three questions would you ask before touching the data?
3. On an order form, the discount box is blank. List three different things the blank could mean, and say who in the company should decide how blanks are treated in reports.

Key terms

data · datum · information · knowledge · insight · DIKW · table · row / record · column / field / variable / attribute · value · dataset · grain · data type · number · text / string · date and time · Boolean · identifier · ISO 8601 date · missing value / NULL · quantitative · qualitative / categorical · discrete · continuous · levels of measurement · nominal · ordinal · interval · ratio · mode · median · mean ·

structured data · semi-structured data · JSON · unstructured data · flattening · metadata · data dictionary · primary data · secondary data · first-party data · third-party data · personal data · data quality · accuracy · completeness · consistency · validity · uniqueness · timeliness · fit for use

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 2, How Computers Store, Move and Protect Data**, shows where data lives: files and formats (CSV, Excel, JSON, Parquet), databases, the cloud, and APIs.
 - **Chapter 3, How a Business Runs on Data**, follows one Riverstone order from enquiry to cash, and shows every system that records data along the way.
 - **Chapter 4, Numbers Without Fear**, builds the everyday math for working with quantitative data: percentages, growth, and averages.
 - **Chapters 10 and 12** put this chapter's ideas into tools: data types and tables in spreadsheets, then in databases.
 - **Chapter 14, Data Cleaning & Preparation**, fixes the quality problems from section 1.10 at scale.
 - **Chapter 21, Descriptive Statistics & Probability**, explains which averages and charts suit each level of measurement.
 - **Interview preparation**: questions on data types, levels of measurement, and data quality appear in the Statistics bank (Chapter 73) and the Business Analyst bank (Chapter 76), with model answers.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Yes: it's recorded information about trains and times. (b) No, not until it's recorded, for example as an audio file or written music. (c) Yes: a recorded count. (d) No, until you write it in a to-do list or set a reminder; then it's a recorded note. (e) Yes: it's recorded, even though it's handwritten and unstructured, which makes it harder to use.
2. (a) Data: a single recorded fact. (b) Insight: an action based on understanding. (c) Information: orders summarized into a monthly total. (d) Knowledge: an explanation of a pattern.
3. ID number: text (a code; you'd never add two). Delivery date: date. Parcel weight: number. Invoice paid: true/false. PIN code: text. Items in a cart: number (a whole number).
4. T-shirt size: ordinal. Monthly revenue: ratio. Customer segment: nominal. Temperature in °C: interval. Rank in class: ordinal (the gap between 1st and 2nd isn't the same as between 2nd and 3rd). Years as a customer: ratio (zero means none). Calendar year: interval (the year 0 isn't "no time", so 2026 isn't twice 1013). Star rating: ordinal.

5. CSV of bank transactions: structured. Recorded phone call: unstructured. Weather app JSON: semi-structured. Scanned PDF invoice: unstructured (it's a picture of a document, even though the invoice itself had a layout). Attendance sheet: structured. Google reviews: unstructured text (though each review sits in a semi-structured record with a date and a star rating).

6. (a) No: PIN codes are nominal labels, so an average has no meaning. (b) Yes: sizes are ordinal, so the middle value is meaningful. (c) Yes: revenue is ratio data, so percentage change works. (d) No: dates are interval data; you can say "5 days later", but not "50% later". (e) Yes: counting and finding the most common value works for nominal data.

7. One good answer:

Column	Type	Meaning	Allowed values	Can be blank?
student_id	text	the school's ID for the student	the ID format used by the school	no
class_date	date	the day of the class	YYYY-MM-DD, a school day	no
status	text	attendance for that day	Present, Absent, Late, Leave	no
minutes_late	number (whole)	minutes after the start time the student arrived	0 or more	yes: blank when status isn't Late

The last row is the kind of detail that prevents arguments later: a blank `minutes_late` means "not applicable", not zero.

8. Problems and dimensions:

- Order 7002 appears twice: **uniqueness**.
- Quantity -5 isn't possible for an order (unless the business records returns this way, which should be stated in the data dictionary): **validity**.
- 2026-13-01 has a month 13: **validity** (and it can't be trusted for **accuracy**).
- Order 7003 has no unit price: **completeness**.
- "Blue Bay Cafe" and "blue bay cafe": **consistency**.
- "Green Leaf" and "Patel Kitchen" may be shortened names of Green Leaf Hotels and Patel Kitchenware, which would be a **consistency** problem if other tables use the full names. Worth checking.

9. A sample answer: "The average ratings are close (3.0 and 3.4), but they hide an important difference: every Swift Movers delivery was rated 3, while two of Rapid Wheels' five deliveries were rated 1, the lowest possible. For hotel banquets, where one failed delivery can ruin an event, consistency matters more than the occasional excellent delivery, so I'd use Swift Movers. With only five ratings each, we should keep collecting ratings and review the choice in a quarter."

10. `necessary` is nominal (two categories, Yes and No; you could argue it's ordinal if "Yes" is seen as higher, but it's usually treated as nominal). `amount_rs` is ratio. From `necessary` you can calculate the **count** (or share) of Yes and No rows, and the **total amount** in each group, such as Kavya's ₹470

marked No. You can't calculate an **average of the Yes/No values** themselves, unless you deliberately code Yes as 1 and No as 0, in which case the average is simply the share of Yes rows.

11. The counter could count arm movements as steps (while cooking or on a bumpy bus ride), miss steps while the phone sat on a desk, or double-count if both a watch and a phone were synced. Useful metadata: which device recorded it, whether the phone was carried all day, the hourly breakdown (12,000 steps between 2 and 3 a.m. would be suspicious), and whether data from several devices was combined.

12. Good questions include: *"Bad for what decision? What were you trying to do when the data let you down?"* (fit for use), *"Which fields matter most: names, contact details, cities, or something else?"* (scope), and *"Who enters and updates customer data today, and how?"* (the source, because fixing the data without fixing how it's captured means it will be bad again in six months).

13. The blank could mean *no discount was given*, *a discount was given but not recorded*, or *the discount wasn't decided yet* (for example, pending approval). The treatment should be decided by the people who own the rule: usually finance, together with sales. The analyst's job is to raise the question, document the answer in the data dictionary, and apply it consistently in every report.

Chapter 2. How Computers Store, Move and Protect Data

Chapter at a glance

You will learn to: explain how a computer stores letters, numbers, and pictures as bits and bytes · read file sizes from bytes to terabytes, and work out how long a download takes · tell memory from storage · work with files, folders, paths, and extensions without surprises · choose between CSV, Excel, JSON, XML, PDF, and Parquet for a job, and avoid each format's traps · explain what a database, a server, the internet, and the cloud are · describe what an API does and read its replies · protect data with good passwords, encryption, access rules, and backups.

Before you start: Chapter 1 (what data is: rows, columns, types, and quality).

Time needed: 4–5 hours, including the exercises and the project.

Tools: any computer. For the project: a spreadsheet program (Excel or Google Sheets) and a plain-text editor (Notepad on Windows, TextEdit on a Mac, or any code editor).

Practice data: four Riverstone orders saved in five formats, and a demonstration API, both in the companion files (Appendix E). Every file size, output, and timing in this chapter comes from running those files.

Why this matters

In Chapter 1 you learned what data is. This chapter is about where it *lives*, how it *travels*, and how it's *kept safe*. That sounds like IT's job, and some of it is. But these everyday situations all land on a data person's desk:

- A colleague emails a CSV export, you open it in Excel, and every product code that started with zero has quietly lost it.
- A report that took two seconds to open last year now takes two minutes, and the file is 80 MB.
- The finance team's figures in the monthly deck don't match yours, because they're working from `Sales_Report_FINAL_v3.xlsx` and you're working from `Sales_Report_FINAL_v4.xlsx`.
- Your manager asks whether the new CRM "has an API" so the weekly report can update itself.
- A laptop with a customer list on it is left in a taxi.

None of these needs you to be an engineer. Each needs you to understand, in plain terms, how computers store, move, and protect data. People who do understand it waste less time, lose less data, and are trusted with more of it, which is exactly the path this book is about.

In plain English

Imagine Riverstone's old paper office before any computers arrived.

- Everything was written in an **alphabet** of letters and digits. A computer has an even smaller alphabet: just 0 and 1.
- Records were kept on **shelves** in a storage room, where they stayed even when the lights went off. That's a computer's **storage**: its disk.
- To work on a record, a clerk carried it to a **desk**. The desk was fast to work at but small, and cleared every evening. That's the computer's **memory**.
- Records sat in **folders with labels**, and the label said what kind of paper was inside. Those are **files, folders, and extensions**.
- The same information could be written as a **ledger** with neat columns, a **letter**, or a **form** with labeled boxes. Those are **file formats**.
- Documents traveled between branches by **courier**. That's the **network** and the **internet**.
- Other companies didn't walk into the storage room. They came to a **counter window**, filled in a request slip, and a clerk brought back exactly what they asked for. That's an **API**.
- The storage room had **locks**, only some staff had **keys**, and important ledgers were **copied** and kept in another building in case of fire. That's **security** and **backups**.

Every section below is one part of that office, in modern form.

2.1 Bits and bytes: how computers store everything

Everything is 0s and 1s

Deep down, a computer can only store and move **two states**: on or off, charged or not charged, magnetized one way or the other. We write those states as **1** and **0**. One of these on/off values is a **bit** (short for *binary digit*).

One bit can only tell two things apart: yes or no. But put bits together and the number of possibilities doubles with each bit you add:

Bits	Possible patterns	Enough for...
1	2	yes/no
2	4	the four seasons
8	256	every English letter, digit, and punctuation mark
16	65,536	a whole number up to 65,535
32	4,294,967,296	a whole number up to about 4.3 billion

A group of **8 bits** is a **byte**, and the byte is the basic unit for measuring data. When someone says a file is 364 bytes, they mean it takes $364 \times 8 = 2,912$ on/off switches to store.

How text is stored

To store text, computers agree on a table that gives every character a number, and store the number. The capital letter `A` is number 65, which in 8 bits is `01000001`. The digit `7` is number 55. A space is number 32. This agreement, first made for English as **ASCII** in the 1960s, fits every English letter, digit, and common symbol into one byte each.

English isn't the world's only language, of course. **Unicode** is the modern table: it gives a number to almost every character in every writing system, including ₹, Hindi, Tamil, Chinese, and emoji. The most common way to store Unicode as bytes is **UTF-8**, which keeps plain English letters at one byte each and uses two, three, or four bytes for everything else. Here's what some characters really take:

Text	Characters	Bytes in UTF-8
A	1	1
Pune	4	4
é	1	2
₹	1	3
न (Hindi <i>na</i>)	1	3
नमस्ते	6	18
😊	1	4
₹14,700	7	9

So "one character is one byte" is only true for plain English text. It's why a report with a lot of Hindi or Tamil text is bigger than the same report in English, and why a column limited to "100 characters" in one system can overflow a "100 bytes" limit in another.

When the table is wrong: garbled text

If a file is saved in one encoding and opened as if it were another, the bytes are read with the wrong table, and you get nonsense. Here's what happens when the UTF-8 bytes for `₹14,700` are opened as an older Windows encoding:

```
â, 14,700
```

And `Café` becomes `CafÃ©`. If you've ever seen `Ã©` or `â€™` in a report, a mismatched encoding is almost always the cause. The fix is to open or import the file with the correct encoding, which is usually UTF-8. Section 2.5 shows where this bites in practice.

How numbers are stored

Whole numbers are stored exactly, as binary. Numbers with decimals are trickier. Most programs store them in a format called **floating point**, which is fast but can only store most decimals *approximately*. Ask Python to add 0.1 and 0.2:

```
0.30000000000000004
```

The tiny error is invisible in most charts, but it's why $0.1 + 0.2 = 0.3$ can come out as *false* in a program, and why financial systems store money in exact decimal types instead (you'll meet `NUMERIC` in Chapter 12). Asked to add the same numbers as exact decimals, Python gives `0.3`.

How pictures and sound are stored

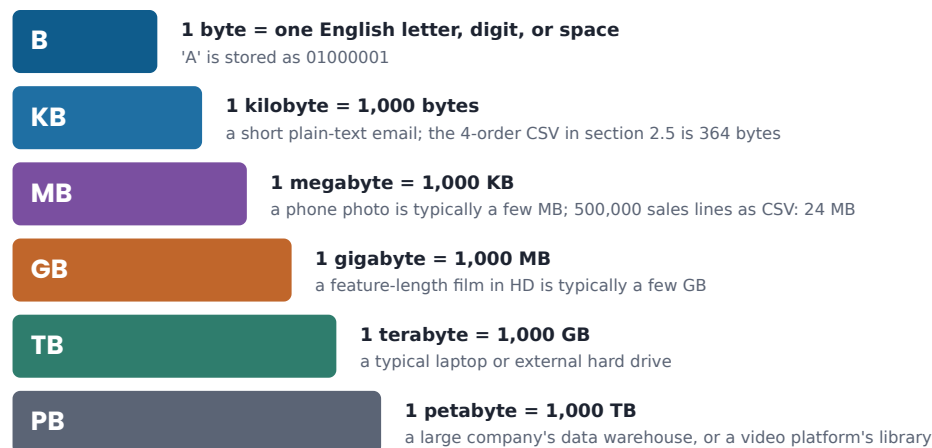
A digital photo is a grid of tiny dots called **pixels**. Each pixel stores three numbers, for how much red, green, and blue light it has, usually one byte each. A 12-megapixel phone photo has 12 million pixels, so before compression it takes $12,000,000 \times 3 = 36$ MB. The file on your phone is usually only a few megabytes, because it has been **compressed** (section 2.5). Sound is stored in a similar way: thousands of measurements of the sound wave every second.

So text, numbers, photos, music, and video are all, in the end, long rows of 0s and 1s. What makes them different is the **agreement** about how to read those bits: the encoding or format.

Try it. Open any plain `.txt` file, count roughly how many characters it has, then check its size in bytes (right-click → *Properties* on Windows, or *Get Info* on a Mac). For English text, the two numbers should be close.

2.2 How big is big? From kilobytes to terabytes

Because data sizes range from a few bytes to many billions, we use prefixes, just as we say kilometers instead of thousands of meters.



Each step is 1,000 times the one before. (Some software counts in steps of 1,024; see section 2.2.)

Figure 2.1 – Each step is a thousand times bigger. The examples are typical sizes, not fixed rules.

A few sizes worth remembering: a page of plain English text is about 2–3 KB; a phone photo is typically a few MB; an hour of HD video is typically a few GB; and Riverstone's test file of 500,000 sales lines, which you'll meet in section 2.5, is 24 MB as a CSV.

Why your 1 TB drive shows 931 GB

There are two ways to count, and both are in use:

- **Decimal (the SI standard):** 1 KB = 1,000 bytes, 1 MB = 1,000,000 bytes, 1 GB = 1,000,000,000 bytes. Drive makers and most network speeds use this.
- **Binary:** 1 "KB" = 1,024 bytes (2^{10}), 1 "MB" = $1,024 \times 1,024$ bytes, and so on. Windows, and some older software, count this way.

The binary units have their own proper names, **KiB**, **MiB**, **GiB** (kibibyte, mebibyte, gibibyte), though few people use them in conversation. The difference grows with size. A drive sold as 1 TB holds 1,000,000,000,000 bytes. Divide by $1,024^3$ (the binary gigabyte) and you get **931.3**, which is the "GB" Windows displays. No space is missing; it's the same number of bytes counted in a different unit.

Bits for speed, bytes for size

Internet speeds are quoted in **megabits per second (Mbps)**, with a small *b*. File sizes are in **megabytes (MB)**, with a capital B. Since a byte is 8 bits, divide by 8:

- A 100 Mbps connection moves at most $100 \div 8 = 12.5$ **MB per second**.
- Downloading a 5 GB file on it takes at least $5,000 \text{ MB} \div 12.5 \text{ MB/s} = 400$ **seconds**, about **6.7 minutes**. In practice it takes longer, because the connection is shared and rarely runs at full speed.

Watch out: b and B. "My file is 80 Mb" and "my file is 80 MB" differ by a factor of eight. In data work, sizes are almost always bytes (MB, GB); speeds are almost always bits (Mbps, Gbps). When a number looks eight times too big or too small, check the letter.

Size limits you'll actually hit

- **Excel** holds at most **1,048,576 rows** and **16,384 columns** per sheet. Long before that, a large workbook becomes slow to open and save. A dataset with millions of rows belongs in a database (Chapter 12) or a format such as Parquet (section 2.5).
- **Email attachments** are usually limited to a few tens of megabytes, which is one reason teams share large files by link instead.
- **Cloud storage plans, phone storage, and laptop drives** are all limited, and duplicate copies of large files (`report_v1` , `report_v2` , `report_final` ...) fill them quickly.

2.3 Memory and storage: the desk and the shelves

Every computer has two very different places to keep data.

	Memory (RAM)	Storage (SSD or hard disk)
Office analogy	the desk you're working at	the shelves in the storage room
What it holds	whatever you have open right now	every file, program, and photo you've saved

	Memory (RAM)	Storage (SSD or hard disk)
Speed	very fast	slower (SSDs are much faster than older spinning hard disks)
Size on a typical laptop	about 8–32 GB	about 256 GB–2 TB
When the power goes off	everything in it is lost	everything stays

Three everyday consequences:

1. **Unsaved work lives only in memory.** If Excel crashes before you save, the changes since your last save were never on the shelves. Save often, or turn on your program's autosave.
2. **Opening a file copies it into memory.** A 200 MB workbook needs well over 200 MB of RAM to work with, often several times more once formulas and formatting are loaded. When a big spreadsheet slows your whole computer, memory is usually what's run out.
3. **Where "saved" means.** Your file might be saved on your own computer's drive, on a **shared network drive** at the office, or in **cloud storage** such as Google Drive or OneDrive, which keeps a copy on the provider's computers and **syncs** it to your devices. Knowing which one matters when you ask, "Can my colleague see this?" or "Is this backed up?"

2.4 Files, folders, paths, and extensions

A **file** is a named chunk of data on storage. A **folder** (or *directory*) is a container for files and other folders. Every file has a **path**, its full address, which lists every folder you pass through to reach it:

```
Windows: C:\Users\Kavya\Documents\Finance\2026-09_spending_log.xlsx
macOS:   /Users/kavya/Documents/Finance/2026-09_spending_log.xlsx
```

Windows separates folders with a backslash `\` and starts with a drive letter such as `C:`. macOS and Linux use a forward slash `/`. You'll type paths when you load files into SQL, Python, or Power BI, and most "file not found" errors are a path with one folder or one letter wrong.

Extensions: what kind of file is this?

The letters after the last dot in a file name are its **extension**. The extension tells the computer which program should open the file:

Extension	What it usually is
<code>.txt</code>	plain text
<code>.csv</code>	comma-separated values: a table as plain text
<code>.xlsx</code>	an Excel workbook (<code>.xls</code> is the older format; <code>.xlsm</code> can contain macros)
<code>.json</code> , <code>.xml</code>	structured text, often exchanged between systems
<code>.pdf</code>	a document laid out for reading and printing

Extension	What it usually is
.jpg , .png	images
.parquet	a compressed, column-based data file (section 2.5)
.sql	a text file of SQL statements
.py	a Python program
.zip	a compressed bundle of other files
.exe	a Windows program that runs when you open it

Renaming `sales.csv` to `sales.xlsx` doesn't convert it into an Excel file. It only changes the label, and Excel will warn that the file doesn't match its name. To convert, open the file and use *Save As*.

Watch out: hidden extensions. Windows hides known extensions by default, so a file named `invoice.pdf.exe` appears as `invoice.pdf`, with a harmless-looking name and a program hidden inside. It's a classic trick in phishing emails. Turn on *File name extensions* in File Explorer's *View* menu, and never open an unexpected attachment that turns out to end in `.exe`, `.js`, `.bat`, `.scr`, or `.xls` from someone you don't know.

Naming files so people can find them

Compare these two folders:

Report final.xlsx	2026-01-31_monthly_sales_report.xlsx
Report final (2).xlsx	2026-02-28_monthly_sales_report.xlsx
Report FINAL v3 use this one.xlsx	2026-03-31_monthly_sales_report.xlsx

The right-hand names follow four habits: **a date first, written year-month-day**, so files sort in time order automatically (Chapter 1); **the same pattern every time**; **lower-case words joined with underscores or hyphens**, which avoids problems in code and web links; and **no words like "final"**, which are always eventually wrong. If you need versions, use the version history in Google Drive, OneDrive, or SharePoint, or, for queries and code, Git (Chapter 26).

2.5 Data file formats: the same data, packed six ways

Riverstone's sales coordinator exports four February orders. Here is exactly the same data saved in five data formats. Look at what each one looks like inside; the differences explain when to use each.

CSV: a table as plain text

```
order_id,customer_name,order_date,status,sales_rep,net_revenue,delivery_note
5006,Metro Mart,2026-02-06,Delivered,Rahul Mehta,14640.00,
5007,Coastal Foods,2026-02-11,Delivered,Rahul Mehta,32625.00,Call before delivery
5008,Sunrise Caterers,2026-02-19,Delivered,,23325.00,
5009,Northgate Distributors,2026-02-25,Shipped,Farah Khan,76560.00,"Gate 2, Okhla Phase II"
```

CSV (comma-separated values) is the simplest data format there is: one line per row, commas between values, and usually a first line of column names. Any spreadsheet, database, and programming language can read it, which is why "export to CSV" is the universal language between systems. This file is **364 bytes**.

Look closely and you'll find three of CSV's limits:

1. **Commas inside values need quotes.** Northgate's delivery note contains a comma, so it's wrapped in double quotes. Without them, a program would see an extra column.
2. **A blank is just nothing.** Order 5008's missing sales rep is two commas in a row. CSV can't tell "unknown", "not applicable", and an empty text value apart (Chapter 1).
3. **There are no types.** Everything in a CSV is text. When a program reads this file back, `2026-02-06` arrives as text, not a date, unless the program guesses or you tell it. Reading it back with Python's pandas library, `order_date` comes back as text while `net_revenue` is guessed to be a number.

Watch out: opening a CSV in Excel changes it. When Excel opens a CSV directly, it guesses a type for every value, and some guesses silently change your data: product codes like `00451` lose their leading zeros and become `451`; long ID numbers turn into `4.52E+13`; codes like `3-4` or `1/2` can become dates. If you then save, the damage is written back into the file. Instead, use *Data → From Text/CSV* (Excel) or *File → Import* (Google Sheets), and set code and ID columns to **Text** before loading. If ₹ or accented names appear garbled, choose **UTF-8** as the file's encoding in that same import window (section 2.1).

Excel (.xlsx): a workbook for people

An `.xlsx` file looks like a grid in Excel, but it isn't plain text. It's actually a **zip file** holding a small collection of XML files. Unzip the four-order workbook and you find ten files inside, including:

```
[Content_Types].xml
xl/workbook.xml
xl/worksheets/sheet1.xml
xl/sharedStrings.xml
xl/styles.xml
```

That structure lets a workbook hold far more than a CSV: several sheets, real dates and numbers, formulas, formatting, charts, and pivot tables. It also means the file isn't meant to be read as text, and needs software that understands the format. This tiny workbook is **5,674 bytes**, more than fifteen times the CSV, because the structure itself takes space. Excel is the right choice when **people** will open, read, and work with the file.

JSON: data with labels, for systems

```
[
  {
    "order_id": 5006,
    "customer_name": "Metro Mart",
    "order_date": "2026-02-06",
    "status": "Delivered",
    "sales_rep": "Rahul Mehta",
    "net_revenue": 14640.0,
    "delivery_note": null
  },
  {
    "order_id": 5008,
    "customer_name": "Sunrise Caterers",
    "order_date": "2026-02-19",
    "status": "Delivered",
    "sales_rep": null,
    "net_revenue": 23325.0,
    "delivery_note": null
  }
]
```

(Two of the four orders shown; the file has all four.)

JSON (JavaScript Object Notation) is the semi-structured format from Chapter 1. Each record is a set of **"name": value** pairs inside curly braces `{ }`, and a list of records goes inside square brackets `[ ]`. Notice what JSON can express that CSV can't: text is in quotes and numbers aren't, so some types survive; and a missing value is written as `null`, clearly different from an empty text value `" "`. A record can also contain a list, such as an order with its order lines inside it. JSON is how most **apps and APIs** exchange data (section 2.8). Because every record repeats every column name, it's larger: **894 bytes** here.

XML: labels as tags

```
<order id="5009">
  <customer_name>Northgate Distributors</customer_name>
  <order_date>2026-02-25</order_date>
  <status>Shipped</status>
  <sales_rep>Farah Khan</sales_rep>
  <net_revenue>76560.00</net_revenue>
  <delivery_note>Gate 2, Okhla Phase II</delivery_note>
</order>
```

XML (eXtensible Markup Language) does the same job as JSON with **tags**: every value sits between an opening `<name>` and a closing `</name>`, and an empty value can be written `<sales_rep/>`. It's older and wordier (the four-order file is **1,118 bytes**), and you'll meet it in banking and government systems, e-invoicing, older business software, and inside Office files like the workbook above.

PDF: made for reading, not for data

A **PDF** fixes exactly how a document looks on screen and on paper, which makes it ideal for invoices, contracts, and reports people read. It's poor for data: a table in a PDF is stored as text placed at positions on a page, not as rows and columns, so copying it into a spreadsheet often scrambles the

columns. When someone offers you data "as a PDF", ask whether the CSV or Excel export behind it exists. (When it doesn't, Chapter 58 shows how AI tools can extract tables from documents, with checks.)

Parquet: built for large-scale analysis

Parquet is a format built for analyzing large datasets, and it's the standard format of cloud data platforms (Part V). You can't read it as text: the file begins with the four letters `PAR1`, and the rest is compressed binary data. What makes it special is how it's laid out.

CSV and Excel: stored row by row

2025-08-30	2	105	77	row 1
2025-02-24	21	107	62	row 2
2025-11-03	7	101	15	row 3
2025-05-19	2	103	140	row 4

To add up qty, the computer still reads every value in every row, then throws most away.

Parquet: stored column by column

date	2025-08-30	2025-02-24	2025-11-03	2025-05-19
customer	2	21	7	2
product	105	107	101	103
qty	77	62	15	140

To add up qty, it reads only the qty column. Similar values sit together, so they compress well.

Figure 2.2 — CSV stores data row by row; Parquet stores it column by column.

A CSV stores **row by row**: order 1's date, customer, product, quantity; then order 2's. Parquet stores **column by column**: all the dates together, then all the customers, then all the quantities. Two big benefits follow. To total the quantity column, a program reads only that column and skips the rest. And values in one column are similar to each other (a column of dates, a column of product IDs), so they **compress** very well. Parquet also stores each column's **type**, so dates come back as dates.

For four orders, Parquet's advantages are invisible: the file is **4,872 bytes**, bigger than the CSV, because of the information it stores about its own structure. Its advantages appear at scale.

The same test at scale

To see the differences properly, the same 500,000 sales lines, with 8 columns, were saved in each format and read back with Python on a small two-processor cloud computer:

Format	File size	Time to read everything	Time to read one column
CSV	24.1 MB	0.23 s	0.10 s
CSV, compressed with gzip	4.7 MB	—	—
JSON	82.6 MB	1.43 s	—
Excel (.xlsx)	18.3 MB	31.4 s	—
Parquet	4.3 MB	0.02 s	0.003 s

Exact times depend on the computer, but the pattern holds everywhere. **Parquet was the smallest file, and about ten times faster than CSV to read in full.** Excel took more than a hundred times longer than CSV to read, because every cell has to be unpacked from zipped XML. JSON was the

largest, because it repeats all eight column names on every one of the 500,000 rows.

Compression

Compression makes files smaller by writing repeated patterns more efficiently. Zipping the CSV cut it from 24.1 MB to 4.7 MB, because a column like `status` repeats the same few words half a million times. There are two kinds:

- **Lossless** compression (ZIP, gzip, and the compression inside Parquet and `.xlsx`) gives back **exactly** the original bytes when you unpack it. Data files must only ever use lossless compression.
- **Lossy** compression (JPEG photos, MP3 music, most video) throws away detail people are unlikely to notice. It's fine for a photo, and unacceptable for a sales ledger.

Choosing a format

Format	People can read it as text?	Keeps types?	Size	Opens in Excel?	Best for
CSV	yes	no	small	yes (import carefully)	moving tables between systems; simple exports
Excel	no (needs Excel or similar)	yes	medium	yes	files people will read, filter, and work in
JSON	yes	partly	large	with Power Query	apps and APIs; nested records
XML	yes	partly	large	with Power Query	older systems, e-invoicing, Office internals
PDF	no	no	varies	no	documents for reading and printing
Parquet	no	yes	smallest	no (Power BI and Python can read it)	large datasets for analysis

Real-life example: why the data team asks for "the raw export". A manager sends a PDF of last quarter's sales, laid out beautifully. To analyze it, the analyst has to copy 40 pages of tables by hand or with a converter, and check every row. The same data exported from the billing system as CSV takes ten seconds to load. The rule most data teams follow: **PDF and formatted Excel for people who read; CSV, JSON, or Parquet for machines that process.**

2.6 Databases: data that many people can use at once

Files are fine for one person and one moment in time. Business data needs more. Picture Riverstone's orders kept as a shared Excel file: three salespeople edit it at once, two copies drift apart, someone sorts one column without the others and scrambles every order, and the file grows until it no longer opens.

A **database** is software built to avoid those problems. It keeps data in **tables** (rows and columns, like a strict spreadsheet), and it:

- lets **thousands of people and programs** read and write at the same time without overwriting each other;
- enforces **rules**: no order for a customer who doesn't exist, no negative quantities, no duplicate invoice numbers;
- answers **questions** in a language called **SQL**, even across millions of rows, in seconds;
- controls **who may see or change** each table;
- keeps a record of changes so it can recover after a crash.

Most business databases run as a **database server**: a program on a computer, usually in a data center or the cloud, that other programs connect to over the network. (A few, like SQLite, which runs inside many phone apps, store the whole database in a single file.) Underneath, a database still saves its data in files on storage; you just never touch those files directly.

This one page is only a preview. **Chapter 12** teaches databases and SQL properly, from creating your first table onward.

2.7 Servers, the internet, and the cloud

Clients and servers

When you open Gmail, two computers are involved. Your laptop or phone is the **client**: it asks for something. A computer in one of Google's data centers is the **server**: it listens for requests and answers them. A server isn't a special kind of machine; it's a computer whose job is to serve requests, usually running all day in a **data center**, a building full of such computers with reliable power, cooling, and network connections.

How a request finds its way

The **internet** is a worldwide network of networks that carries data between clients and servers. Three ideas explain most of it:

- **IP address**. Every device on a network has an address, such as `203.0.113.14`, so data knows where to go, like a postal address.
- **DNS** (Domain Name System). People remember names like `riverstone.example`, not numbers. DNS works like a phone book, turning a name into an IP address before your request is sent.
- **HTTP and HTTPS**. These are the rules for how a browser or program asks a web server for something and gets a reply. **HTTPS** is the secure version: everything sent is **encrypted**, so people along the route can't read or change it. The padlock icon in a browser means the connection uses HTTPS. It does *not* mean the website itself is trustworthy.

Data on the internet travels in small **packets**, each finding its own route, then reassembled at the other end. You never see that, but it's why a large file arrives in pieces and why a slow connection affects everything at once.

The cloud

"The cloud" sounds mysterious. It's simpler than it sounds: **the cloud is computers in someone else's data center that you rent over the internet**, instead of buying and running your own. The largest providers include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud.

Businesses rent at three levels, depending on how much they want to manage themselves:

Level	You rent...	You still manage...	Examples
IaaS (infrastructure as a service)	virtual computers, storage, and networks	the operating system, software, and data	a virtual server to run your own database
PaaS (platform as a service)	a ready-to-use platform, such as a managed database	your data and how you use it	a cloud PostgreSQL service that handles backups and updates for you
SaaS (software as a service)	a finished application, used through a browser	your data and your settings	Gmail, Google Drive, Microsoft 365, CRM and accounting software

Why companies move to the cloud: they **pay for what they use** instead of buying servers upfront, they can **grow or shrink** in minutes, and the provider handles power, hardware failures, and much of the security. The trade-offs: bills that grow quietly if nobody watches them (Chapter 65), dependence on one provider, and questions about **where the data is physically stored**. Many companies, and some laws, require certain data to stay in a particular country, so cloud services let customers choose a **region** for their data.

2.8 APIs: how systems talk to each other

Riverstone's sales system, its accounting software, its website, and its payment gateway are separate programs, often from separate companies. For a website order to appear in the sales system, or for a weekly report to pull the latest invoices, the programs need a way to ask each other for data. That way is an **API** (Application Programming Interface).

Like a restaurant: you (the program) never walk into the kitchen (the database). You give the waiter (the API) a clear order, and it brings back a dish.

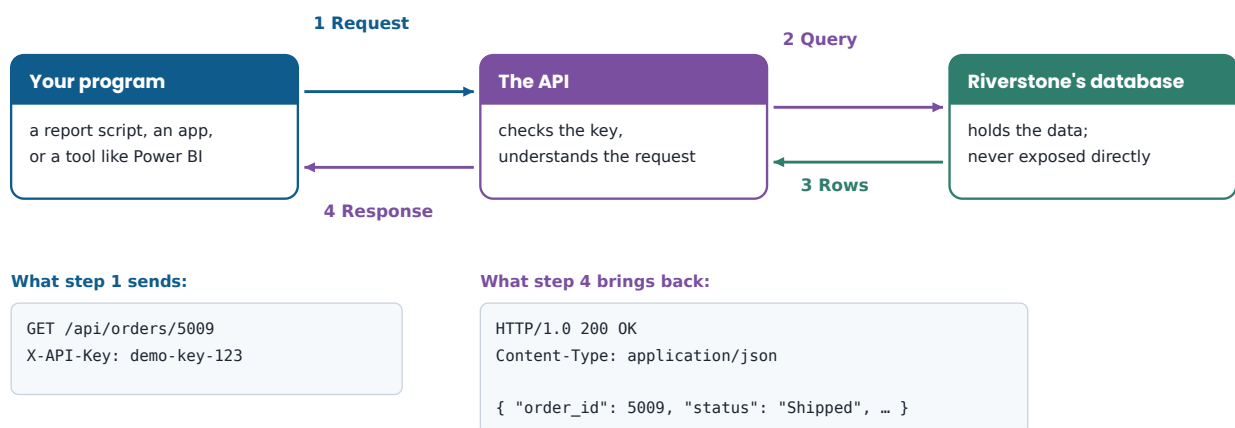


Figure 2.3 — An API is the waiter between your program and someone else's kitchen.

The restaurant analogy is the classic one. You don't walk into the kitchen and take food off the stove; you'd get in the way, and you might take someone else's dish. You give a **waiter** a clear order from the **menu**, and the waiter brings back what you asked for. An API is the waiter: it offers a menu of requests it will accept, checks you're allowed to ask, fetches what's needed from the system behind it, and brings back a reply in an agreed format, usually JSON. The system's database (the kitchen) is never opened up to outsiders.

A real request and response

The companion files include a tiny demonstration API for Riverstone that runs on your own computer (Appendix E). A **request** asks for one order by its address, and includes an **API key**, a secret code that proves the caller is allowed in. Here's the full reply to a request for order 5009, sent with the `curl` command-line tool:

```
HTTP/1.0 200 OK
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 147

{
  "order_id": 5009,
  "customer_name": "Northgate Distributors",
  "order_date": "2026-02-25",
  "status": "Shipped",
  "net_revenue": 76560.0
}
```

Every **response** has two parts. The top is the **header**: a **status code** (200 OK) that says how the request went, plus details such as the type of content coming back. After a blank line comes the **body**: the data itself, here in JSON.

Ask for an order that doesn't exist, and the status code changes:

```
HTTP/1.0 404 Not Found
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 38

{
  "error": "order 5099 not found"
}
```

Leave out the API key, and the API refuses before it even looks for the order:

```
HTTP/1.0 401 Unauthorized
Server: RiverstoneAPI/1.0
Date: Wed, 16 Sep 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 44

{
  "error": "missing or invalid API key"
}
```

Status codes worth knowing

The first digit tells you who's responsible: **2** means it worked, **4** means the request has a problem (your side), **5** means the server has a problem (their side).

Code	Meaning	What to do
200 OK	it worked	read the body
201 Created	a new record was created	note the new record's ID
400 Bad Request	the request is malformed	check what you sent
401 Unauthorized	no valid key or login	check the API key
403 Forbidden	you're recognized, but not allowed to do this	ask for the right permission
404 Not Found	no such record or address	check the ID and the address
429 Too Many Requests	you've hit the API's rate limit	slow down and retry later
500 Internal Server Error	the server failed	retry later; tell the API's owner if it persists

Why APIs matter to data people

Most modern business software, including CRMs, accounting tools, payment gateways, e-commerce platforms, and ad platforms, offers an API. That's what makes automation possible: a script can pull yesterday's invoices every morning without anyone logging in and clicking *Export*, and a dashboard can refresh itself. You'll call real APIs from Python in Chapter 18, automate reports with them in Chapter 20, and design how whole systems exchange data in Chapter 51.

Two more terms you'll hear:

- A **webhook** is an API in reverse. Instead of your program asking "anything new?" every five minutes, the other system calls *your* address the moment something happens, such as "payment received".
- **API keys are passwords**. Anyone who has the key can do whatever the key allows. Never paste one into a spreadsheet, a shared document, a chat, or code you publish online. Chapter 20 shows how to store them safely.

2.9 Keeping data safe

The three goals: confidentiality, integrity, availability

Security professionals describe what they protect with three words, often called the **CIA triad**:

Goal	Means	Example of failure at Riverstone
Confidentiality	only the right people can see the data	the customer price list is emailed to a competitor by mistake

Goal	Means	Example of failure at Riverstone
Integrity	the data is accurate and hasn't been changed without authorization	someone edits invoice amounts in a shared spreadsheet, and nobody can tell
Availability	the data is there when people need it	ransomware locks the sales server during the busiest week of the year

Every habit below protects one or more of the three.

Accounts and passwords

Most data breaches start with a person, not a clever technical attack: a reused password, a shared login, or a click on a fake email. The habits that prevent most of them:

- **Long and unique.** A long passphrase is harder to guess than a short password full of symbols, and **every account needs a different one**. When one website is breached, attackers try the same email and password everywhere else.
- **Use a password manager.** Nobody can remember fifty unique passphrases. A password manager remembers them, fills them in, and warns you about reused ones.
- **Turn on multi-factor authentication (MFA).** A second step, such as a code from an authenticator app or a prompt on your phone, stops most attacks even when a password leaks.
- **Never share logins.** A CRM password shared in a team chat means you can't tell who changed what, and you can't remove one person's access without locking everyone out. Each person gets their own account.
- **Be suspicious of urgency.** Phishing emails pressure you to act now: "Your account will be closed", "Pay this invoice today". Check the sender's real address, hover over links before clicking, and confirm unusual payment requests by phone.

Access: the principle of least privilege

Give each person and program only the access they need, and nothing more. An analyst who builds sales reports needs to *read* the sales tables, not change them, and doesn't need salary data at all. That's why, as Chapter 12 mentions, many companies give analysts **read-only** accounts. Least privilege limits the damage from a mistake or a stolen password.

Encryption

Encryption scrambles data with a key so that only someone with the right key can read it. You rely on it in two places:

- **In transit**, while data moves: HTTPS websites, secure email, and encrypted connections to databases.
- **At rest**, while data sits on storage: an encrypted laptop drive (BitLocker on Windows, FileVault on a Mac) means a stolen laptop's files are unreadable without your login. For any laptop that carries work data, this should be switched on.

Integrity checks: fingerprints for files

How do you know a file hasn't been changed, even by one character? Computers calculate a **hash**: a fixed-length "fingerprint" of the data. The same data always gives the same hash, and the smallest change gives a completely different one. Here are the SHA-256 hashes of two payment instructions that differ only in the order of two digits:

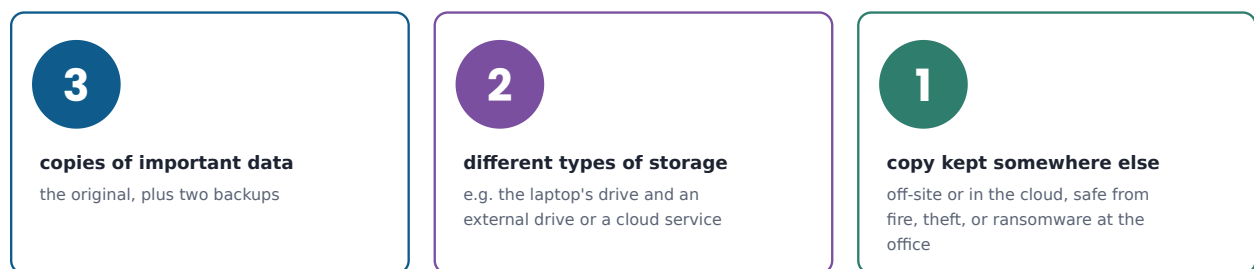
```
Text:    Pay Rs 14,700 to Riverstone Supplies
SHA-256: f4251ff3fb7191f7e79677f3b1871db06c0935c3cc08164496995f1f909f6f71

Text:    Pay Rs 17,400 to Riverstone Supplies
SHA-256: 9d2f842c50110e140abd578e7e8460d1f6f2bbca7eba341ce1e5bee43566d0fa
```

Nothing about the second fingerprint resembles the first. Software uses hashes to check that downloads arrived undamaged, that backups match the original, and that passwords are stored safely (systems store a hash of your password, not the password itself). Data pipelines use them to detect whether a file has changed since yesterday (Chapter 45).

Backups and versions

Hardware fails, laptops are stolen, files are deleted by mistake, and **ransomware** (malicious software that locks files and demands payment) can encrypt everything a computer can reach. The only complete protection is a copy you can restore. The widely used rule of thumb is **3-2-1**:



A backup you have never restored is a hope, not a backup: test a restore regularly.

Figure 2.4 – Three copies, two kinds of storage, one kept elsewhere.

Two points people often miss:

- **Syncing is not backing up.** Cloud storage such as Google Drive or OneDrive **copies every change** to every device, including deleting a file or saving over it with a mistake. Version history and a recycle bin help, but only for a limited time. A true backup is a separate copy that your everyday mistakes don't touch.
- **Test your restore.** Many organizations discover their backups were incomplete only on the day they need them.

Version history is backup's everyday cousin. Google Drive, OneDrive, and SharePoint keep earlier versions of a file, so you can see who changed what and roll back a bad edit. Use it instead of saving `report_v7_FINAL.xlsx`. For SQL queries and code, Git does the same job more precisely (Chapter 26).

Personal data

Some data is about **people**: names, phone numbers, addresses, ID numbers, health information, salaries. Handle it with extra care. India's Digital Personal Data Protection Act, 2023, the European Union's GDPR, and similar laws elsewhere set rules for collecting, storing, and sharing it. The everyday habits: collect and keep only what you need; don't copy it to personal devices, personal email, or personal cloud accounts; share it only with people entitled to see it; and use anonymized or sample data for practice and demos. Chapter 64 covers privacy and governance in depth.

If you make a mistake, say so immediately. Emailed a customer list to the wrong person? Clicked a suspicious link? Lost a laptop? Tell your manager and IT **straight away**. Minutes matter: a password can be reset, a sent email sometimes recalled, a laptop wiped remotely. Organizations forgive a reported mistake far more easily than a hidden one.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Opening a CSV directly in Excel and saving it	leading zeros gone, IDs as 4.52E+13 , codes turned into dates	Import with <i>Data</i> → <i>From Text/CSV</i> ; set ID columns to Text
Wrong text encoding	â,¹, Ã©, â€™ in names and amounts	Import as UTF-8
Confusing Mbps with MB/s	downloads "eight times slower than expected"	Divide Mbps by 8 for MB/s
Renaming a file's extension to "convert" it	the file won't open, or opens with a warning	Open the file and use <i>Save As</i>
Using "final", "new", or "(2)" in file names	nobody knows which file is current	Date-first names plus version history
Sending data as PDF	the receiver re-types or mis-copies tables	Share the CSV or Excel export
Keeping millions of rows in Excel	files take minutes to open; rows cut off at 1,048,576	Use a database or Parquet
Pasting API keys or passwords into files and chats	anyone with the file can use the account	Store secrets in a password manager or secure settings
Sharing one login among a team	no record of who changed what	One account per person, with only the access they need
Treating cloud sync as a backup	a deletion or a bad save spreads to every copy	Keep separate, tested backups (3-2-1)
Unencrypted work laptop	a lost laptop becomes a data breach	Turn on drive encryption
Hiding a security mistake	a small incident becomes a large one	Report it immediately

In the real world: the Friday file

Before Meera (Chapter 1) joined, Riverstone's weekly sales report worked like this. Every Friday afternoon, Imran, who ran sales operations, exported orders from the billing system as CSV, opened the file in Excel, added a few formulas and a pivot table, and saved it as `Weekly Sales FINAL.xlsx` on his laptop. Then he emailed it to eleven people. Over one bad month, four things went wrong:

1. **Product codes broke.** Riverstone's new product range used codes such as `00731`. Excel turned them into `731`, and the lookup to the price list failed for every new product. Two weeks of reports understated new-product sales.
2. **Nobody knew which file was current.** Imran sent a corrected report on a Monday, as `Weekly Sales FINAL (2).xlsx`. Half the recipients kept using the Friday version, and the regional managers argued about numbers that came from two different files.
3. **The file grew until it choked.** Each week's data was pasted under the last. At 40 MB the workbook took four minutes to open, and some recipients' email rejected it.
4. **The laptop was stolen** from a car. It wasn't encrypted, and the workbook held two years of customer names, phone numbers, and prices.

Riverstone's fix used nothing more advanced than this chapter:

- The export was **imported** with codes set to text, and the billing system's **API** later replaced the manual export entirely.
- The report moved to **one shared location** with version history, and the email became a **link** to it, with the date in the file name. Everyone saw the same, current file.
- History moved out of the workbook into a **database**, and the report kept only the latest weeks, so it opened in seconds.
- Every company laptop was **encrypted**, customer phone numbers were removed from reports that didn't need them, and the IT team set up **tested 3-2-1 backups**.

None of those changes required a data engineer. They required someone who understood files, formats, APIs, and basic security well enough to notice what was wrong. Chapter 20 takes this exact report and automates it end to end.

Tools

- **A plain-text editor.** Notepad (Windows), TextEdit in plain-text mode (Mac), or a free code editor such as Visual Studio Code. Opening a CSV or JSON file in a text editor shows you what's really inside, without a spreadsheet's guesses.
- **Excel or Google Sheets**, for the project. Learn the *import* routes (*Data* → *From Text/CSV* in Excel; *File* → *Import* in Google Sheets), not just double-clicking.
- **A password manager and an authenticator app.** Set them up for your own accounts this week.
- **The companion files** (Appendix E): `orders_feb_2026` in `.csv`, `.xlsx`, `.json`, `.xml`, and `.parquet`; and `api_demo.py`, the demonstration API used in section 2.8, which you'll run yourself in Chapter

The project: one dataset, five formats

Goal: see with your own eyes what each format stores, what it loses, and how each program treats it.

Option A: use the companion files `orders_feb_2026.csv`, `.xlsx`, `.json`, and `.xml`. **Option B:** use your spending log from Chapter 1's project, and create the formats yourself: save it from your spreadsheet as `.xlsx` and as CSV (UTF-8), then type a JSON version of the first three rows by hand in a text editor, using section 2.5 as your model.

Steps:

1. **Open each file in a plain-text editor.** Which ones can you read? For the `.xlsx`, what do you see, and why?
2. **Find the missing values.** How does each format show a blank sales rep or delivery note? Can you tell "unknown" from "empty text" in each?
3. **Find the types.** In which formats can you tell that `net_revenue` is a number and `order_date` is a date, just by looking?
4. **Import the CSV into Excel or Google Sheets twice:** once by double-clicking, once through the import screen with every column set to Text. Add a row with a code like `00731` to your copy first. What differs?
5. **Compare sizes.** Record each file's size in bytes. Which is smallest? Why is the Excel file larger than the CSV for so little data?
6. **Break a CSV on purpose.** Remove the quotes around `"Gate 2, Okhla Phase II"`, save, and reimport. What happens to the columns?
7. **Write it up.** A one-page comparison table: format, readable as text, keeps types, shows blanks as, size, and "I would use it for...". Finish with three sentences of advice for a colleague who is about to email a CSV.

Stretch goals:

- Save the same data as a ZIP file and compare its size with the CSV.
- Type ₹ into one value in your spreadsheet, save it once as CSV UTF-8 and once as plain CSV (*Comma delimited*), then open both files in a text editor. Does the symbol survive in each?
- Check which of your own accounts have MFA switched on, and switch it on for the rest.

You've got it when...

- ☐ I can explain bits, bytes, and why ₹ takes more space than R.
- ☐ I can convert between KB, MB, GB, and TB, and between Mbps and MB/s, and I know why a 1 TB drive shows 931 GB.

- ☐ I know the difference between memory and storage, and what "saved" means for my work.
 - ☐ I name files so they sort by date and never need the word "final".
 - ☐ I can say which format I'd use for a person, an app, or a large dataset, and why.
 - ☐ I import CSV files instead of double-clicking them when codes or IDs are involved.
 - ☐ I can explain a server, the cloud, and an API to a non-technical colleague.
 - ☐ I can read an API response and tell from its status code whether the problem is mine or the server's.
 - ☐ My own accounts use unique passwords from a password manager, with MFA.
 - ☐ I can explain the 3-2-1 rule and why syncing isn't a backup.
-

Recap

- Computers store everything as **bits** (0s and 1s), grouped into **bytes**. Text is stored through an **encoding** such as **UTF-8**; the wrong encoding produces garbled characters. Decimals are usually stored approximately, which is why money needs exact types.
 - Sizes go **KB → MB → GB → TB → PB**, each 1,000 times the last (or 1,024 in some software). Speeds are in **bits**, sizes in **bytes**: divide Mbps by 8.
 - **Memory** is fast and temporary; **storage** is slower and permanent. Unsaved work lives only in memory.
 - Files have **paths** and **extensions**. Name them date-first, never "final", and watch for hidden extensions.
 - **CSV** is universal but has no types; **Excel** is for people; **JSON** and **XML** carry labeled data between systems; **PDF** is for reading; **Parquet** is compact, typed, and fast for large-scale analysis. Import CSVs carefully.
 - **Databases** let many people use data at once, with rules and SQL.
 - A **server** answers a **client's** requests over the **internet**; **HTTPS** encrypts the connection; the **cloud** is rented computers and services (IaaS, PaaS, SaaS).
 - An **API** is the waiter between programs: a request goes in, a response with a **status code** and usually JSON comes back. API keys are passwords.
 - Protect **confidentiality, integrity, and availability**: unique passwords, MFA, least privilege, encryption, hashes, tested **3-2-1 backups**, version history, careful handling of personal data, and reporting mistakes immediately.
-

Practice exercises

Warm-up

1. Convert: (a) 3.5 GB to MB; (b) 250,000 KB to MB; (c) 2 TB to GB.

2. How many bytes does each take in UTF-8: `Riverstone` , `₹500` ?
3. A friend's new laptop has "512 GB" of storage, but Windows shows about 476 GB. Is anything wrong? Explain in two sentences.
4. What file type is each of these, and which program would normally open it: `2026-03-31_stock.csv` , `invoice_9007.pdf` , `po_template.xlsm` , `export.json` , `setup.exe` ?

Core

5. How long, at best, does it take to download a 2 GB file on a 50 Mbps connection?
6. Choose the best format for each job, and say why in one line: (a) a monthly summary your manager will open and filter; (b) a website sending a new order to the sales system; (c) two years of sensor readings from Riverstone's machines, to be analyzed in the cloud; (d) a tax invoice sent to a customer; (e) a one-off list of 3,000 customers to import into a new email tool.
7. This line comes from a CSV file with seven columns. What's wrong with it, and how should it be written? `5010,Green Leaf Hotels,2026-03-03,Delivered,Farah Khan,20100.00,Leave at reception,back gate`
8. A script calls Riverstone's API and gets these replies on different days: `401` , `404` , `429` , `503` . For each, say whether the problem is probably on the script's side or the server's, and what you'd do.
9. Section 2.9 shows two SHA-256 hashes. A colleague sends you a backup file and its hash. You calculate the hash of the file you received and get a different value. What does that tell you, and what doesn't it tell you?

Stretch

10. Kavya keeps her only copy of her spending logs in a Google Drive folder that syncs to her laptop. Does her setup meet the 3-2-1 rule? Describe one thing that could lose her data anyway, and a change that would fix it.
11. Riverstone's new product codes look like `00731` . Describe exactly how a code can lose its leading zeros between the billing system's CSV export and the price lookup in a workbook, and give two ways to prevent it.
12. A 12-megapixel photo is 36 MB before compression, but the file on the phone is 3 MB. Could a sales ledger be compressed the same way? Explain using the terms lossless and lossy.

Think about it (no calculation needed)

13. A team shares one login for the company's CRM, and the password is pinned in a group chat. List three risks, and propose a better setup.
14. Your manager asks, "Should we keep our sales database on our own server in the office, or move it to the cloud?" Give two advantages of each option, and one question you'd ask before deciding.
15. You receive an email from "Riverstone Accounts" with an attachment called `Payment_Details.pdf` and the message "Urgent: confirm today or the supplier contract will be cancelled." What do you check before opening it?

Key terms

bit · byte · binary · ASCII · Unicode · UTF-8 · encoding · garbled text (mojibake) · floating point · pixel · kilobyte (KB) · megabyte (MB) · gigabyte (GB) · terabyte (TB) · petabyte (PB) · KiB / MiB / GiB · megabits per second (Mbps) · memory (RAM) · storage (SSD, hard disk) · file · folder / directory · path · extension · CSV · Excel workbook (.xlsx) · JSON · XML · PDF · Parquet · columnar storage · compression · lossless · lossy · database · database server · client · server · data center · internet · IP address · DNS · HTTP / HTTPS · packet · cloud · IaaS · PaaS · SaaS · region · API · request · response · header · body · status code · API key · rate limit · webhook · CIA triad · confidentiality · integrity · availability · password manager · multi-factor authentication (MFA) · phishing · least privilege · encryption in transit · encryption at rest · hash · SHA-256 · ransomware · backup · 3-2-1 rule · sync · version history · personal data

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 3, How a Business Runs on Data**, follows one Riverstone order through every system that stores and passes along its data.
 - **Chapters 10 and 11** teach spreadsheets properly, including importing CSV files without damage.
 - **Chapter 12, Databases & SQL Foundations**, turns the one-page preview in section 2.6 into a full, hands-on skill.
 - **Chapter 18** reads CSV, Excel, JSON, and Parquet files in Python, and calls real APIs.
 - **Chapter 20** automates the Friday report from this chapter, including storing API keys safely.
 - **Part V (Chapters 45–52)** builds on formats, compression, the cloud, and APIs at company scale; **Chapter 64** covers security, privacy, and governance in depth.
 - **Interview preparation:** file formats, APIs, and data security questions appear in the Data Engineering bank (Chapter 77) and the Automation & Integration bank (Chapter 78).
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) $3.5 \times 1,000 = \mathbf{3,500\ MB}$. (b) $250,000 \div 1,000 = \mathbf{250\ MB}$. (c) $2 \times 1,000 = \mathbf{2,000\ GB}$. (Using binary units instead, the answers would be 3,584 MiB, about 244 MiB, and 2,048 GiB. Either is acceptable if you say which you used.)
2. **Riverstone** is 10 plain English letters: **10 bytes**. **₹500** is **6 bytes**: 3 for ₹ and 1 for each digit.
3. Nothing is wrong. The maker counts 512 GB as 512,000,000,000 bytes, while Windows divides by $1,024^3$ and shows about 476.8 "GB", which is really GiB; it's the same storage measured in a different unit.

4. `2026-03-31_stock.csv` : a CSV table as plain text; a spreadsheet or text editor. `invoice_9007.pdf` : a PDF document; a PDF reader or browser. `po_template.xlsx` : an Excel workbook that can contain macros; Excel (be careful enabling macros from unknown senders). `export.json` : JSON data; a text editor, code editor, or a program that reads JSON. `setup.exe` : a Windows program that runs when opened; only open it if you trust exactly where it came from.

5. $50 \text{ Mbps} \div 8 = 6.25 \text{ MB per second}$. $2,000 \text{ MB} \div 6.25 = \mathbf{320 \text{ seconds, about 5.3 minutes}}$, and in practice a little longer.

6. (a) **Excel**: people will read and filter it, and it keeps formatting and types. (b) **JSON through an API**: it's what apps exchange, and it can carry an order with its lines nested inside. (c) **Parquet**: large, typed, compressed, and fast to analyze by column. (d) **PDF**: a fixed document the customer reads and keeps, which looks the same everywhere. (e) **CSV**: nearly every tool imports it; set the ID and phone columns to text.

7. The delivery note contains a comma but isn't in quotes, so the line splits into **eight** values instead of seven: `Leave at reception` and `back gate` become separate columns, and every program reading it will either reject the row or shift the data. Correct version: `5010,Green Leaf Hotels,2026-03-03,Delivered,Farah Khan,20100.00,"Leave at reception, back gate"`

8. `401 Unauthorized` : the script's side. The API key is missing, wrong, or expired, so check how the key is stored and whether it has been changed. `404 Not Found` : usually the script's side: the ID or the address is wrong, or the record really doesn't exist. Check the ID before assuming the API is broken. `429 Too Many Requests` : the script's side. It's calling too often, so add a pause between calls and retry later. `503` (a 5xx code): the server's side. Retry after a delay, and if it continues, check the provider's status page or contact the API's owner.

9. Different hashes mean **the two files are not identical**: the file was changed or damaged somewhere between your colleague and you, or they calculated the hash on a different version. It doesn't tell you **what** changed, **where**, or **whether** the change was accidental or deliberate. Ask for the file again, compare the hashes, and don't use the backup until they match.

10. No. She has one copy that appears in two places, not three independent copies, and nothing is kept separately from her synced accounts. If she deletes a file, saves over it by mistake, or ransomware encrypts her laptop, the change **syncs to Drive** too. Version history might rescue her, but only for a limited time. A fix: a regular backup to an external drive kept at home, plus a second copy in a separate cloud backup service or account that doesn't sync changes automatically, and an occasional test that she can restore a file.

11. The billing system writes `00731` correctly into the CSV as text. When someone **double-clicks** the CSV, Excel guesses that `00731` is a number and stores **731**. If the file is saved, the damage is written back. The price list still says `00731`, so a lookup for `731` finds nothing. Prevention: (1) import the CSV with *Data → From Text/CSV* and set the product code column to **Text** before loading; (2) better still, get the data from the source system through a connection or an API that keeps the column's type, so no one opens the raw CSV at all. (Adding a letter prefix to codes, such as `P00731`, also prevents it, but changing codes is a business decision.)

12. Only with **lossless** compression. A photo can use **lossy** compression (JPEG) because discarding tiny details people won't notice doesn't matter. A sales ledger can't lose a single digit, so it must use lossless compression, such as ZIP, which restores exactly the original bytes. Ledgers compress very well anyway, because columns such as status, product, and customer repeat the same values many times: the 24.1 MB CSV in section 2.5 zipped to 4.7 MB with nothing lost.

13. Risks: (1) nobody can tell who made a change or deleted a record, so mistakes and misuse can't be traced; (2) when someone leaves the team, they still know the password, and removing their access means changing it for everyone; (3) anyone who can read the chat, on any device, including a lost phone, can log in; and a single leaked password exposes the whole CRM. Better: one account per person, with permissions matched to each role (least privilege), MFA switched on, and access removed promptly when people leave.

14. *Own server:* direct physical control of where the data is; can work on the office network even when the internet is down. *Cloud:* no hardware to buy or maintain, with backups, updates, and failover handled by the provider; easy to scale up as data grows, and reachable securely from anywhere. Useful questions include: "What happens if our office server fails at 2 a.m., and who fixes it?", "Are there rules about which country our customer data must stay in?", and "What would the cloud cost each month at our size, compared with buying and running our own server?"

15. Check the **sender's real email address**, not just the display name; check whether the attachment is really a PDF, with extensions visible, because `Payment_Details.pdf.exe` is a program; notice the **pressure** ("urgent", "today", "cancelled"), a classic phishing sign; and **confirm by phone**, using a number you already have rather than one in the email, before opening the attachment or acting on it. If in doubt, report it to IT without opening it.

Chapter 3. How a Business Runs on Data

Chapter at a glance

You will learn to: name a company's departments and the data each creates · follow one order from enquiry to cash, and say which system records each step · explain what ERP, CRM, HRMS, POS, e-commerce, and support systems do · tell a transaction from a report, and bookings from billings from collections · define a KPI so two people calculate the same number · describe how dashboards, meetings, and decision rights turn data into decisions · find where people copy, re-type, and email data by hand, and estimate the cost.

Before you start: Chapter 1 (rows, columns, grain, data quality) and Chapter 2 (files, databases, and APIs).

Time needed: 3–4 hours, including the exercises and the project.

Tools: a pen and a notebook. A spreadsheet or a free diagram tool is useful for the project, but optional.

Practice data: Riverstone Supplies' first quarter of 2026: twelve orders, ten invoices, and ten payments from the mini database you'll query yourself in Chapter 12. Every number in this chapter was calculated from that database and checked by script.

Why this matters

In Chapter 1 you learned what data is. In Chapter 2 you learned where it lives. This chapter answers the question that sits between those and every job in this book: **why does a business have all this data in the first place, and who uses it?**

The short answer is that a company is a chain of people handing work to each other. A salesperson hands an order to the warehouse. The warehouse hands a delivery to finance. Finance hands a number to the managing director. Every handover leaves a record, and those records are the data you'll clean, query, chart, and model for the rest of your career.

Analysts who understand this chain are far more useful than analysts who only know the tools. When a manager asks, "*What were sales last month?*", the analyst who knows the chain asks one question back: "*Orders placed, invoices raised, or cash received?*" Those are three different numbers. At Riverstone Supplies in January 2026 they were ₹116,210, ₹104,210, and ₹0. By the end of this chapter you'll know exactly why, and you'll never again hand over a "sales" number without saying which one it is.

This chapter also starts the book's **automation** thread. Before you can automate anything, you have to see where people work by hand. Section 3.7 shows you how.

In plain English

Think of a relay race. Four runners, one baton. Each runner takes the baton, runs their stretch, and hands it on. Races are often lost at the handovers.

A business works the same way, and **data is the baton**. The salesperson writes down what the customer wants and passes it on. The warehouse packs the goods, records what it shipped, and passes that on. Finance sends a bill, records what was paid, and passes it on to the people who write reports.

Three things follow from the picture:

- **Each runner only sees their own stretch.** The warehouse doesn't care how long the customer took to decide; sales doesn't see when the cash arrives. That's why different departments quote different numbers for the "same" thing.
- **Handovers are where things go wrong.** An order typed twice can be typed wrong once. A file emailed around can be out of date by the time it's opened.
- **Someone has to watch the whole race.** Managers need to know how the team is doing, not just each runner. That's what reports, KPIs, and dashboards are for, and it's where analysts come in.

3.1 The departments of a company and the data they create

Meet the company. **Riverstone Supplies** makes and distributes plastic storage boxes, kitchenware, industrial crates, and a small range of furniture. Its customers are retailers (like Sharma Hardware in Mumbai), hotels and caterers (like Green Leaf Hotels in Pune), and wholesalers (like Coastal Foods in Chennai). It has two plants:

- **Plant 1, Taloja** (Navi Mumbai), which makes storage boxes and kitchenware.
- **Plant 2, Chakan** (Pune), which makes industrial crates and furniture.

Finished goods go to the central warehouse, **Bhiwandi Main**, near Mumbai, and are shipped to customers from there.

Each department exists to do a job, and creates data as a side effect of doing it. Figure 3.1 shows the eight you'll meet most often.

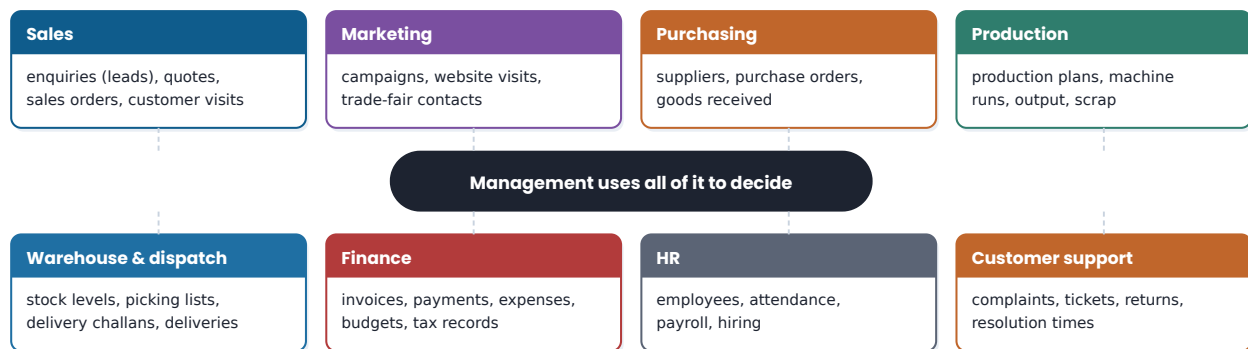


Figure 3.1 – Every department creates data while doing its own job. Management needs all of it to decide.

Department	Its job	Data it creates	A question the data answers
Sales	win and keep customers	enquiries (leads), quotes, orders, visit notes	<i>Which customers are ordering less than last year?</i>
Marketing	make the right people aware of the products	campaigns, website visits, trade-fair contacts	<i>Which campaign brought in enquiries that became orders?</i>
Purchasing	buy raw materials and packaging	suppliers, purchase orders, goods received	<i>Which supplier delivers late most often?</i>
Production	make the goods	production plans, machine runs, output, scrap	<i>How much plastic did we waste last month, and on which machine?</i>
Warehouse and dispatch	store, pick, pack, and ship	stock levels, picking lists, delivery challans , proofs of delivery	<i>How many orders left the warehouse late?</i>
Finance	bill customers, collect cash, pay bills, keep the books	invoices, payments, expenses, budgets, tax records	<i>How much are customers overdue, and who?</i>
HR	hire, pay, and look after employees	employee records, attendance, payroll, hiring	<i>How long does it take to fill a job?</i>
Customer support	fix problems after the sale	complaints, tickets , returns, resolution times	<i>Which product causes the most complaints?</i>

A **lead** is a possible customer who has shown interest but hasn't bought. A **delivery challan** is the document that travels with goods in India, listing what's in the truck (elsewhere, a delivery note).

Finance needs sales' orders to raise invoices. Production needs them to plan what to make. Management needs everything to decide where to spend money. **Data created in one department is almost always used in another**, and that is the root of most of the problems, and most of the jobs, in this book.

Try it. Pick a company you've dealt with recently: a food delivery app, your bank, a hospital. Name three departments it must have and one record each creates about you.

3.2 Following one order from enquiry to cash

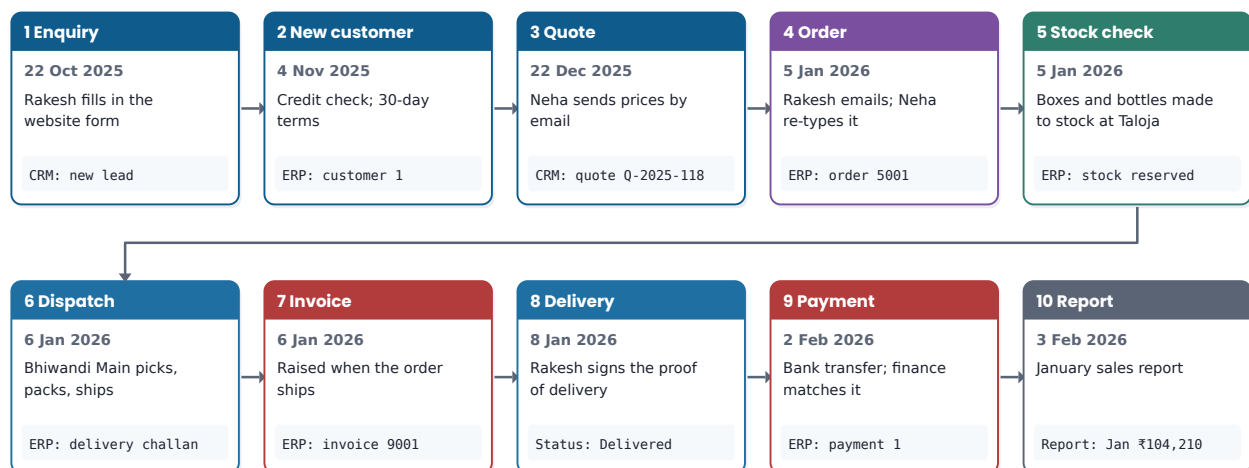
To see how departments depend on each other's data, follow one piece of business all the way through. This journey is called **order to cash**, or **lead to cash** when it includes winning the customer.

Here is a real order from Riverstone's database: **order 5001**, from Sharma Hardware, a retail customer in Mumbai. You'll query these exact rows in Chapter 12. The order has two lines:

Product	Quantity	List price	Discount	Line value
Storage Box 10L	20	₹450	0%	₹9,000
Water Bottle 1L	50	₹120	5%	₹5,700
Total				₹14,700

Check it by hand: $20 \times ₹450 = ₹9,000$. $50 \times ₹120 = ₹6,000$, less 5% is ₹5,700. $₹9,000 + ₹5,700 = ₹14,700$.
✓ (As in the rest of this book, tax is left out to keep numbers simple.)

The order row only says *who ordered what, when*. But it has a history before it and a future after it, and every step was recorded somewhere. Figure 3.2 shows all ten steps.



Ten steps and more than three months from a website enquiry to one line in a monthly report.

Figure 3.2 – One order, ten steps. Each box shows the date, what happened, and the record it left behind.

1. Enquiry (22 October 2025). Rakesh, the buyer at Sharma Hardware, fills in the enquiry form on Riverstone's website: name, shop, city, phone, and "interested in storage boxes for our shop". The form creates a **lead** in the CRM, the sales team's system for tracking deals (section 3.3). Sharma Hardware isn't a customer yet; it's a row with a status of *New*.

2. New customer (4 November 2025). Neha Kulkarni, a sales executive, visits the shop. Rakesh wants to buy on credit, so finance runs a credit check and agrees **payment terms** of 30 days: Sharma Hardware can pay up to 30 days after each invoice. Finance creates the customer in the ERP, Riverstone's main business system, where it gets **customer ID 1** and a signup date of 2025-11-04. Every later order, invoice, and payment carries that ID.

3. Quote (22 December 2025). Rakesh asks for prices. Neha builds a quote, Q-2025-118, in a spreadsheet, emails it as a PDF, and logs it against the lead in the CRM. A **quote** is an offer: *these products, at these prices, until this date.*

4. Order (5 January 2026). Rakesh replies to the quote email: "Please send 20 of the 10-litre boxes and 50 bottles." Neha opens the ERP and **types the order in**: customer 1, two lines, prices, a 5% discount on the bottles that she agreed on the phone. The ERP creates **order 5001** with a status of *Pending*, dated 2026-01-05, with Neha as the sales rep.

5. Stock check (5 January 2026). Both products are **made to stock**: Plant 1 at Taloja makes them in advance and sends them to Bhiwandi Main. But Bhiwandi Main updates the ERP's stock figures once a day from its own spreadsheet, so Neha phones the warehouse to be sure. The stock is **reserved** for order 5001.

6. Dispatch (6 January 2026). The warehouse prints a **picking list**, picks and packs the boxes and bottles, and ships them with a delivery challan. The ERP marks the order *Shipped*.

7. Invoice (6 January 2026). When goods ship, the ERP raises **invoice 9001** for ₹14,700, dated 2026-01-06, **due** on 2026-02-05, 30 days later, because of the terms agreed in step 2. An **invoice** is the formal request for payment: the sale becomes money the customer owes.

8. Delivery (8 January 2026). The truck arrives. Rakesh signs a paper **proof of delivery** (POD). Someone at Bhiwandi Main scans it, emails it to finance, and updates the order to *Delivered*.

9. Payment (2 February 2026). Sharma Hardware pays ₹14,700 by bank transfer. The bank statement shows a short reference, "SHARMA HW JAN". A finance assistant works out that it pays invoice 9001 and records **payment 1** against it.

10. Report (3 February 2026). The team prepares the **January sales report**. Invoice 9001 is one of the three behind the line "*January: ₹104,210*".

It took a long time. From enquiry to cash was 103 days. Once the order existed, things moved faster: order to invoice 1 day, invoice to payment 27 days, order to cash 28 days. Each gap comes from subtracting two dates recorded by two different departments.

The data lives in several places. The lead and quote are in the CRM; the customer, order, invoice, and payment in the ERP; the proof of delivery is a scanned image in someone's email; the report is an Excel file. To answer "*How long does it take to turn an enquiry into cash?*" you need two systems that don't share an ID for Sharma Hardware: the CRM knows it by a lead number, the ERP as customer 1.

People carried the baton by hand at several handovers. Neha re-typed the order and phoned about stock; someone scanned paper, matched a bank line by eye, and built the report. Section 3.7 puts a cost on these steps.

Simplification note. Real processes have more branches: partial shipments, back orders, credit holds, returns, and credit notes. Order 5001 is a clean run. You'll meet messier cases in the data itself: order 5004 was cancelled, invoice 9002 was paid in two instalments, and invoice 9007 hasn't been paid at all.

3.3 Business systems: ERP, CRM, HRMS, POS, and e-commerce

A **business system** is software a department uses for its daily work, storing the records in a database (section 2.6). You'll hear these abbreviations in your first week of any data job.

System	What it's for	Typical records	Main users	Examples of real products
ERP (enterprise resource planning)	running the core of the business: sales orders, stock, production, purchasing, invoicing, and accounts, all in one system	customers, products, orders, invoices, payments, stock movements, purchase orders	sales operations, warehouse, production, purchasing, finance	SAP, Oracle NetSuite, Microsoft Dynamics 365, Odoo; TallyPrime is common in smaller Indian firms
CRM (customer relationship management)	winning and keeping customers	leads, contacts, calls, meetings, quotes, deals and their stages	sales, marketing, customer success	Salesforce, HubSpot, Zoho CRM
HRMS (human resource management system)	managing employees	employee records, attendance, leave, payroll, hiring	HR, managers, every employee for leave and payslips	Workday, Zoho People, Keka
POS (point of sale)	recording sales at a shop counter	bills, items scanned, payment method, cashier, time	shop staff	the billing app at Sai Krupa General Store (Chapter 1); supermarket tills
E-commerce platform	selling online	product listings, carts, online orders, payments, website visits	online sales, marketing	Shopify, WooCommerce, marketplaces
Support desk (ticketing system)	tracking customer problems until they're fixed	tickets, messages, categories, status, time to resolve	customer support	Zendesk, Freshdesk

Riverstone's set-up is typical for a mid-sized manufacturer. This book names its systems by type, because the lessons don't depend on the brand:

- **The ERP** holds customers, products, employees (as sales reps), orders, order lines, invoices, and payments, plus stock, production, and purchasing. People at Riverstone still call its invoicing module "the billing system"; that's where Imran exported his Friday file from in Chapter 2. **The mini database you'll use from Chapter 12 onward is a small copy of the ERP's sales tables.**
- **The CRM** holds leads, contacts, quotes, and the sales pipeline.
- **The website** shows the catalog and feeds enquiries to the CRM (customers don't order online). **The support desk** records complaints and returns. **The HRMS** holds employees and payroll.
- **Spreadsheets and email** fill every gap between them: quotes, the warehouse's stock sheet, the monthly report.

Riverstone has no POS, because it sells to businesses. But Sharma Hardware's POS knows exactly which Riverstone boxes sell on a Saturday afternoon, and Riverstone never sees that unless Sharma shares it: **the most useful data about your products often sits in someone else's system.**

Systems of record and the gaps between systems

Every important fact should have one official source, its **system of record** (or **source of truth**). At Riverstone, that's the ERP for orders and invoices, the CRM for leads, and the HRMS for employees. When the CRM and the ERP disagree about what Sharma Hardware bought, the ERP wins.

The trouble is the gaps. Riverstone's CRM and ERP aren't connected: when a quote becomes an order, someone must mark the deal *Won* in the CRM, and a cancellation in the ERP never reaches the CRM unless someone remembers. Connecting systems so data flows between them automatically is **integration** (Chapters 45 and 51).

Spreadsheet link. A spreadsheet can become a system of record by accident. If the warehouse's stock sheet is more current than the ERP, the sheet is now the source of truth for stock, with no access control or history (section 2.9).

3.4 Transactions and reports: booked, billed, and collected

Everything recorded in Figure 3.2 is a **transaction**: a record of one business event, written when it happens. *Invoice 9001 was raised for ₹14,700.* Transactions are the rows in a business system's database.

A **report** summarizes many transactions to answer a question ("*How much did we invoice in January?*"), at a point in time, using rules someone chose.

	Transaction	Report
What it is	one event	a summary of many events
Grain (Chapter 1)	one order, one invoice, one payment	one month, one customer, one product
Created	by the business system, as work happens	by a person or a scheduled job, after the fact
Changes?	should be corrected, not rewritten; a cancellation is a new status or a new record	changes whenever the data or the rules change
Example	invoice 9001: ₹14,700, due 2026-02-05	"January billings: ₹104,210"

Systems for transactions are tuned to write one record quickly and safely; reporting needs to read millions and add them up. That's why growing companies move reporting into a separate **data warehouse** (Chapter 49).

Three numbers called "sales"

Because reports apply rules, **two reports built from correct transactions can give different answers to the same question.** The classic case is "sales", which can mean three things, each counted at a different step of Figure 3.2:

- **Bookings:** the value of orders placed (step 4). The earliest sign of sales work.

- **Billings:** the value of invoices raised (step 7). Money customers owe.
- **Collections:** cash that arrived (step 9). What pays salaries and suppliers.

Here are all three for Riverstone's first quarter of 2026:

Month	Booked (orders placed)	Billed (invoiced)	Collected (cash in)
January	₹116,210	₹104,210	₹0
February	₹161,700	₹161,700	₹64,700
March	₹58,020	₹31,800	₹132,550
Quarter	₹335,930	₹297,710	₹197,250

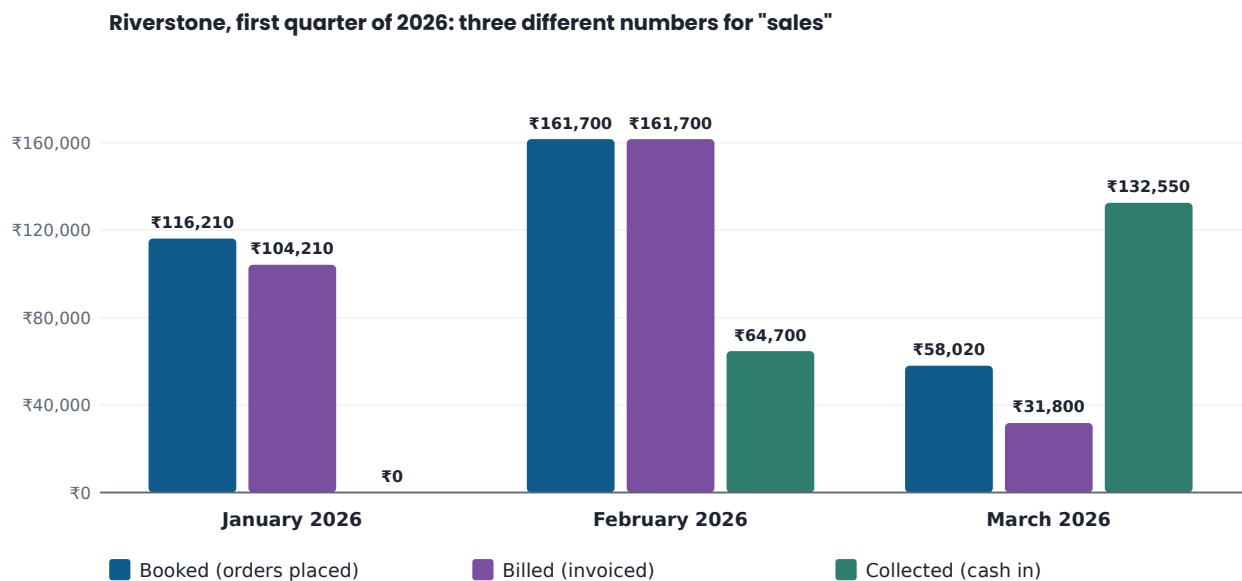


Figure 3.3 – Same company, same quarter, three honest answers to "what were sales?"

Every gap traces to specific transactions:

- **January booked ₹116,210 but billed ₹104,210.** The difference, ₹12,000, is order 5004 from Green Leaf Hotels: 100 water bottles, placed on 20 January and cancelled. Figure 3.3 and the table count bookings the way Riverstone's sales team reports them from the CRM, as every order placed, so the cancelled order is in. It was never shipped or invoiced.
- **January collected ₹0.** Every January invoice had 30-day terms, so none was due until February. Sharma Hardware's ₹14,700 arrived on 2 February.
- **February's three numbers.** All five February orders shipped and were invoiced in February, so bookings and billings match at ₹161,700. The ₹64,700 collected was for *January's* invoices: ₹14,700 from Sharma Hardware, ₹40,000 of Coastal Foods' ₹73,260, and ₹10,000 of Patel Kitchenware's ₹16,250.
- **March booked ₹58,020 but billed ₹31,800.** Order 5012 from Metro Mart, worth ₹26,220, is still *Pending* on 31 March: booked, not yet shipped, so not invoiced. $₹58,020 - ₹26,220 = ₹31,800$. ✓
- **March collected more than it billed,** mostly February's invoices being paid. Cash lags billings by about the payment terms.

And the quarter reconciles:

- Bookings ₹335,930 – cancelled ₹12,000 – pending ₹26,220 = billings ₹297,710. ✓
- Billings ₹297,710 – collected ₹197,250 = **₹100,460 still owed by customers**, which finance calls **receivables** (or accounts receivable). ✓

Watch out: "sales" without a definition. If you don't know which of the three is meant, ask. If you can't, give the name and rule with the number: "Billed sales (invoices raised) in January: ₹104,210." Otherwise two departments argue about who is wrong when both are right.

Simplification note. Accountants recognize **revenue** under formal accounting standards, which decide exactly when a sale counts. This book uses invoices as a stand-in for revenue, as Chapters 12 and 13 do. In a real company, ask finance which rule applies before publishing a revenue number; this is general information, not accounting advice.

When a report runs matters too

Timing also makes reports differ. Run on 31 March, a report shows order 5012 as *Pending*; run on 10 April, after it ships, it shows the order billed in April. Good reports state their run date and **cut-off** ("orders placed up to 31 March 2026"), which is why this book fixes "today" for each dataset.

3.5 What a KPI is

Managers can't read thousands of transactions or dozens of reports. They need a few numbers that show at a glance whether the business is on track.

A **metric** is any number you measure. A **KPI** (key performance indicator) is a metric chosen as one of the few that matter most, with a target, an owner, and a regular review.

Here are nine KPIs Riverstone's management could track, calculated for the first quarter of 2026 as of 31 March:

KPI	Definition	Q1 2026	Owner
Bookings	value of orders placed, excluding cancelled orders	₹323,930 (includes ₹26,220 pending)	Sales Head
Billings	value of invoices raised	₹297,710	Finance Manager
Collections	cash received from customers	₹197,250 (66.3% of billings)	Finance Manager
Gross margin	(billings – cost of the products sold) ÷ billings	22.6%	Finance Manager
Average order value (AOV)	billings ÷ number of invoiced orders	₹29,771	Sales Head
Cancellation rate	cancelled orders ÷ all orders placed	8.3% (1 of 12)	Sales Head

KPI	Definition	Q1 2026	Owner
Overdue receivables	unpaid amounts on invoices past their due date	₹88,760 of ₹100,460 owed	Finance Manager
Average days to collect	days from invoice to final payment, for fully paid invoices	31.6 days	Finance Manager
Active customers	customers with at least one non-cancelled order in the period	7 of 8	Sales Head

Check two of them by hand. **Gross margin:** the products on invoiced orders cost Riverstone ₹230,450 to make, so the margin is ₹297,710 – ₹230,450 = ₹67,260, and $₹67,260 \div ₹297,710 = 22.6\%$. ✓ **Average days to collect:** five invoices are fully paid, taking 27, 51, 26, 33, and 21 days; they add up to 158, and $158 \div 5 = 31.6$ days. ✓ (Invoice 9001, the one from Figure 3.2, is the 27.)

A KPI needs a definition, not just a name

"Average order value" sounds precise. It isn't. Divide bookings by orders and you get $₹323,930 \div 11$ non-cancelled orders = ₹29,448. Divide billings by invoiced orders and you get ₹29,771. Both are reasonable; they're different KPIs with the same name. You've already met the same problem with bookings: the table in section 3.4 counts the cancelled order (₹335,930 for the quarter), and the KPI table above leaves it out (₹323,930). A usable KPI definition answers six questions:

1. **Formula:** exactly what's divided by what?
2. **Inclusions and exclusions:** are cancelled orders in? Pending? Returns? Tax?
3. **Time:** which date decides the month: order date, invoice date, or payment date?
4. **Source:** which system and table? (The ERP's `invoices` table, not the CRM.)
5. **Owner:** who decides when the definition changes?
6. **Target and review:** what's good, and who looks at it how often?

Together, those answers are a **KPI definition**: Chapter 1's data dictionary one level up, saying what a number on a dashboard means.

Watch out: a KPI that can be improved without improving the business. If reps earn a bonus on bookings, a large order booked at quarter-end and later cancelled raises bookings and gains nothing. Pair KPIs that keep each other honest: bookings *with* cancellation rate, billings *with* collections.

Leading and lagging

Lagging indicators, like collections and gross margin, report what already happened: accurate, but too late to change. **Leading indicators**, like new leads, quotes, and bookings, move first: less certain, but early enough to act on. In Figure 3.2, steps on the left lead and steps on the right lag. A sales head who watches only collections learns about a bad quarter three months late.

3.6 Dashboards, meetings, and who decides what

Data changes nothing until someone uses it to decide, usually on **dashboards** and in **meetings**.

A **dashboard** is a screen of a few KPIs and charts, usually refreshed automatically from the systems of record, that answers the questions its viewer asks every week (Chapters 15 and 16 build them).

Companies look at their numbers on a rhythm. Riverstone's is typical:

When	Meeting	Who attends	What they look at	Decisions made
Every morning	dispatch stand-up at Bhiwandi Main	warehouse supervisor, dispatch team	orders to ship today, stock shortfalls	which orders ship first; what to chase from the plants
Every Monday	weekly sales review	Anita Rao (Sales Head), Vikram Singh (Sales Manager), sales executives	bookings, quotes sent, top open deals	which deals to push; which customers to visit
First week of each month	monthly business review	managing director, department heads	booked, billed, and collected; margin; overdue receivables	where to spend; which problems to fix; whether to change prices
Every quarter	board review	managing director, board of directors	results against the annual plan	strategy, investment, targets for next year

The closer a meeting is to daily work, the more detailed its data; the more senior, the more summarized and lagging. A common first analyst job is building the monthly business review pack.

Decision rights: who decides what

Written rules for who owns which decision are called **decision rights**. Riverstone's discount rules are an example: the bigger the discount, the more senior the approver.

Discount on an order line	Who approves	Data they need to decide
up to 5%	the sales executive	the customer's order history
over 5% and up to 10%	the Sales Manager (Vikram Singh)	order history, the margin on the product at that price
over 10%	the Sales Head (Anita Rao), after checking with the Finance Manager (Suresh Menon)	margin at that price, the customer's payment record, overdue amounts

The data follows the rules: Neha's 5% on order 5001 needed no approval. Rahul's 10% on Coastal Foods' orders 5002 and 5007 went to Vikram. Farah's 12% on Northgate Distributors' 60 industrial crates (order 5009) needed Anita.

Anita needed margin and payment data from finance and order history from the ERP. **The person who decides is rarely the person who holds the data.** Getting the right data to them, in time, in a form they can read in a minute, is the job this book trains you for.

3.7 Where manual work hides

Go back to Figure 3.2 and ask one question at every step: *did a person copy, re-type, check, or carry data by hand here?* Figure 3.4 marks the answers.

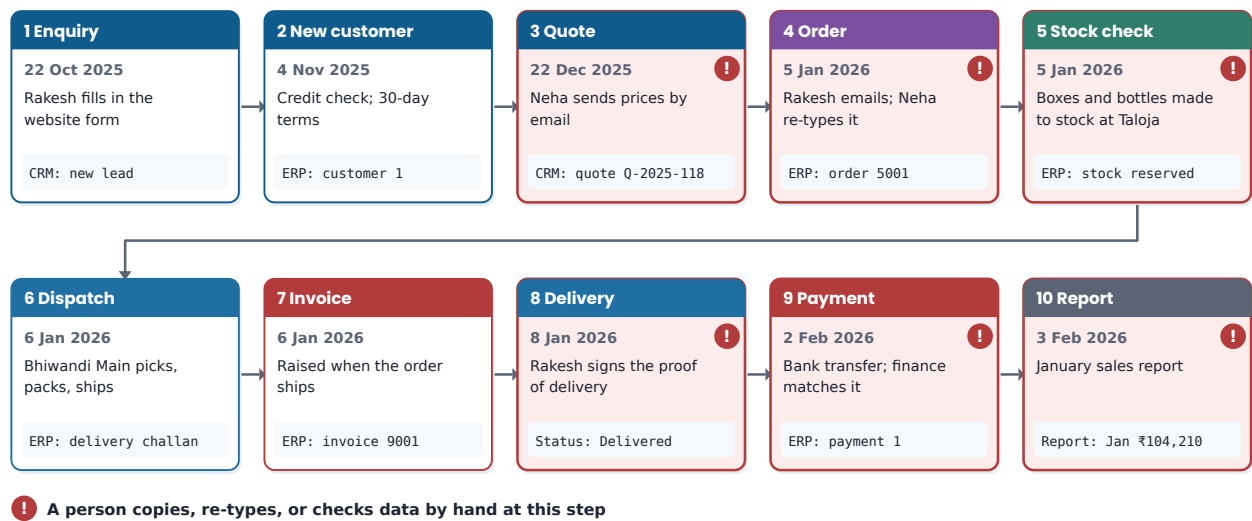


Figure 3.4 – Six of the ten steps depend on someone moving data by hand.

Step	What a person does by hand	What can go wrong
3. Quote	builds the quote in a spreadsheet, saves a PDF, emails it, and logs it in the CRM	old price list used; quote never logged, so the CRM undercounts
4. Order	reads the customer's email and re-types the order into the ERP	wrong quantity, wrong product code, discount forgotten
5. Stock check	phones the warehouse, because the ERP's stock figures are updated once a day from a spreadsheet	promises stock that's already gone
8. Delivery	scans a signed paper POD, emails it to finance, updates the order status	lost paper; order left as <i>Shipped</i> for weeks
9. Payment	reads the bank statement and matches each payment to an invoice	payment matched to the wrong invoice; customer chased for money they've paid
10. Report	exports data, pastes it into Excel, fixes formulas, emails the file	the Friday file problems from Chapter 2: wrong version, broken formulas, a report on one laptop

These steps have names:

- **Re-keying:** typing data that already exists in one place into another (step 4), inviting the typing errors from Chapter 1.
- **Copy-paste integration:** moving data between spreadsheets or systems by copying it (step 10).
- **Emailing files around** (steps 3, 8, and 10): the data stops updating the moment it's attached.
- **Manual matching, or reconciliation:** pairing up two sets of records by eye (step 9).
- **Shadow systems:** personal spreadsheets that hold data the official system should hold, like the warehouse's stock sheet (step 5).

Why it matters

Time. Manual work is small per item and large in total. Suppose a sales team re-types 40 emailed orders a week at about 6 minutes each: 240 minutes, or 4 hours a week, which over a 50-week year is 200 hours, about five working weeks of one person's time. (Round numbers for illustration; exercise 9 works a similar estimate.)

Errors, delay, and people. Mistyped orders are found when the customer receives the wrong goods. A report built by hand on the third of the month can't be seen on the first. And when only one person knows how the report is put together, like Imran and his Friday file in Chapter 2, it stops when they're on leave.

How to find manual work in any process

Ask five questions at every step:

1. **Where does this data first enter a system?** Ideally once, at the source (Chapter 1).
2. **Is it typed again anywhere later?** If yes, that's re-keying.
3. **Does it travel by email, chat, phone, or paper?** Each of those is a handover with no record in a system.
4. **Does anyone keep their own spreadsheet of it?** That's a shadow system.
5. **Does someone check or match things by eye?** That's reconciliation, and usually the slowest step.

Then estimate *how many times a week, how many minutes each*. A list of manual steps with hours attached is where every automation project starts.

Not every manual step should be automated

Anita approving a 12% discount is a judgment, and it should stay with a person; what can be automated is sending her the margin and payment data. And automating a broken process gives you a fast broken process: fix the process first. The book returns to each of Riverstone's manual steps: reports in Chapters 19 and 20, emailed orders in Chapter 58, disconnected systems in Chapters 45 and 51, and the full process map in Chapter 25.

Interview extra point. When an interviewer asks, "What were sales last month?", or gives you a case with a "revenue" figure, say which definition you're using (booked, billed, or collected) before you calculate. It shows in one sentence that you understand the business, not only the tools. Chapters 75 and 76 have practice questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Quoting "sales" without saying which	two managers argue about a ₹12,000 difference	Name the number: booked, billed, or collected, with its rule

Mistake	Symptom	Fix
Treating a difference between reports as an error	hours spent hunting for a bug that isn't there	Reconcile first: cancellations, pending orders, timing, and payment terms explain most gaps
Assuming systems agree	CRM "won deals" total doesn't match ERP orders	Find the system of record for each fact and use it
Reporting a KPI without a written definition	the same KPI name gives different values in different files	Write the formula, inclusions, date rule, source, and owner
Rewarding a single KPI	bookings rise while cancellations rise too	Pair KPIs that keep each other honest
Ignoring the report's run date	the same report shows a different March on 10 April	State the cut-off and the date the report was run
Automating before fixing	wrong numbers delivered faster	Fix the process and the data, then automate
Automating away a judgment	a discount approved by a rule nobody questions	Keep decisions with people; automate getting them the data

In the real world: three numbers for January

It's Tuesday, 3 February 2026. The managing director's monthly business review is on Thursday, and two numbers for January have just reached the MD's inbox. Anita's weekly sales summary, pulled from the CRM's pipeline, says **January sales: ₹116,210**. Suresh Menon, the Finance Manager, sent his month-end report from the ERP: **January sales: ₹104,210**, and underneath it, **cash received in January: ₹0**.

The MD replies to both: *"Which one is right? And why did we collect nothing?"*

Anita asks Meera Iyer to find out. Meera doesn't start by deciding who's wrong. She asks: *which step of the order does each report count?*

1. What does each report count? Anita's summary adds up deals marked **Won** in the CRM, which happens when an order is placed: that's **bookings**. Suresh's report adds up invoices in the ERP: **billings**. His second line is **collections**.

2. What's in one and not the other? Meera lists January's orders from the ERP and January's won deals from the CRM side by side:

Order	Customer	ERP status	Value	In CRM "won"?	Invoiced?
5001	Sharma Hardware	Delivered	₹14,700	yes	yes, 9001
5002	Coastal Foods	Delivered	₹73,260	yes	yes, 9002
5003	Patel Kitchenware	Delivered	₹16,250	yes	yes, 9003
5004	Green Leaf Hotels	Cancelled	₹12,000	yes	no

The CRM total is ₹116,210. The invoiced total is ₹104,210. The difference is exactly order 5004: Green Leaf Hotels cancelled its 100 water bottles in the ERP, and nobody updated the deal in the CRM, because the systems aren't connected.

3. Why no cash? All three invoices had 30-day terms. The earliest, 9001, was due on 5 February. Sharma Hardware paid ₹14,700 yesterday, 2 February, three days early. Coastal Foods' due date is 9 February and Patel Kitchenware's is 14 February. Every invoice raised in January falls due in February, and no older invoices were waiting to be paid, so zero cash in January is exactly what 30-day terms predict.

4. Does it reconcile? ₹116,210 – ₹12,000 = ₹104,210. ✓

On Wednesday afternoon, Meera sends Anita and Suresh a half-page note:

*"Both reports are correct; they count different steps. **Booked** in January (orders placed, per the CRM): ₹116,210. **Billed** (invoices raised, per the ERP): ₹104,210. The ₹12,000 difference is Green Leaf Hotels' order 5004, cancelled in the ERP but still marked Won in the CRM; I've asked Neha's team to update it. **Collected**: ₹0, because January's invoices aren't due until 5–14 February; ₹14,700 has already arrived. Suggestion: the monthly pack shows all three lines with a one-line definition under each, and the ERP is the source for billed and collected."*

On Thursday the pack has three lines instead of one, and the meeting discusses what the numbers mean instead of which is true.

Meera used no tool or formula, only the order's journey, systems of record, three definitions of "sales", and a reconciliation to the rupee. She also found a failed manual handover and fixed it. That's this chapter, applied in an afternoon.

Tools

- **A notebook and pen.** Enough for every exercise, and the best way to draw your first process map.
- **A spreadsheet** (Excel or Google Sheets, optional). Useful for the project's step table and time estimates. Chapter 10 teaches both from the beginning.
- **A diagram tool** (optional). diagrams.net (also called draw.io) is free and runs in a browser; PowerPoint, Google Slides, and Google Drawings work too. Boxes and arrows are all you need.
- **The Riverstone mini database.** Not needed yet; Chapter 6 installs it and Chapter 12 queries the orders, invoices, and payments you followed here.

The project: map the data flow of one process

Goal: map one real process the way Figure 3.2 maps order 5001, find its manual work, and propose one improvement.

Step 1. Choose a process you can observe or ask about: an expense claim, a customer return, a monthly report, or outside work, how a local shop restocks or a clinic books appointments.

Step 2. List 6 to 12 steps in order, recording six things for each:

Column	What to write
step	a number and a two-word name
who	the role (not a person's name)
what_happens	one sentence
data_created_or_used	the record: a form, a bill, a row in a system
where_it_lives	system, spreadsheet, paper, email, someone's memory
how_it_moves_on	automatic, typed again, emailed, phoned, carried

Step 3. Draw it. Boxes for steps, arrows for handovers, the record written under each box, as in Figure 3.2.

Step 4. Mark the manual work. Ask the five questions from section 3.7 at every step. Mark each manual handover in red, as in Figure 3.4.

Step 5. Put a number on it: hours per month for each manual step, with your assumptions.

Step 6. Name one KPI that shows whether the process works (for example, "days from return request to refund"), defined with section 3.5's six questions.

Step 7. Propose one improvement in three sentences: what changes, what it saves, and what could go wrong.

Here is a short worked start, for the restocking process at Sai Krupa General Store from Chapter 1:

step	who	what_happens	data_created_or_used	where_it_lives	how_it_moves_on
1 Spot low stock	shop assistant	notices a shelf is nearly empty	item name	memory	told to the owner
2 Write list	owner	writes items and quantities to reorder	reorder list	paper notebook	photographed and sent by chat
3 Place order	distributor's salesperson	types the list into the distributor's app	purchase order	distributor's system	automatic
4 Receive goods	shop assistant	checks delivered items against the paper bill	delivery bill	paper	carried to the owner
5 Update stock	owner	adds received quantities in the billing app	stock levels	billing app (POS)	typed again

Steps 1, 2, and 5 are manual, and step 5 re-keys what the bill already says. If the billing app flagged low stock, step 1 wouldn't depend on someone noticing.

Deliverable: the table, drawing, hours estimate, KPI definition, and proposal. Keep it for Chapter 25.

You've got it when...

- ☐ I can name the main departments of a company and one kind of data each creates.
 - ☐ I can walk through lead to cash for one order and say what record each step leaves, and in which system.
 - ☐ I can explain what an ERP, CRM, HRMS, POS, e-commerce platform, and support desk are for.
 - ☐ I can tell a transaction from a report, and I know what a system of record is.
 - ☐ I never say "sales" without saying booked, billed, or collected, and I can reconcile the three.
 - ☐ I can write a KPI definition that two people would calculate the same way.
 - ☐ I can find re-keying, copy-paste, emailed files, manual matching, and shadow systems in a process, and estimate their cost in hours.
 - ☐ I've mapped the data flow of one real process and proposed one improvement.
-

Recap

- A company is a chain of departments handing work to each other. **Every handover leaves data**, and data created in one department is almost always used in another.
 - **Lead to cash** follows one piece of business from enquiry to payment. Riverstone's order 5001 took ten steps: 103 days from enquiry to cash, 28 from order to cash.
 - **Business systems** record daily work: **ERP** (orders, stock, invoices, accounts), **CRM** (leads, quotes, deals), **HRMS** (employees, payroll), **POS** (shop sales), **e-commerce** (online orders), and **support desks** (tickets). Spreadsheets and email fill the gaps between them.
 - Each fact should have one **system of record**. Systems that aren't **integrated** drift apart.
 - A **transaction** records one event; a **report** summarizes many, using rules, at a point in time.
 - "Sales" can mean **bookings** (orders placed), **billings** (invoices raised), or **collections** (cash received). For Riverstone's January: ₹116,210, ₹104,210, and ₹0, all correct. Reconcile them with cancellations, pending orders, and payment terms.
 - A **KPI** is a chosen metric with a **definition**, an owner, a target, and a review. Pair KPIs so none can be gamed alone; watch **leading** as well as **lagging** indicators.
 - **Dashboards and meetings** turn data into decisions on a rhythm. **Decision rights** say who decides; the decider rarely holds the data.
 - **Manual work hides** in re-keying, copy-paste, emailed files, manual matching, and shadow systems. Find it with five questions, count it in hours, fix the process first, and keep judgment with people.
-

Practice exercises

Warm-up

1. Match each record to the department that creates it: (a) a delivery challan, (b) a purchase order to Western Polymers for plastic granules, (c) a payslip, (d) a complaint about a cracked crate, (e) a quote, (f) a record of how much plastic was scrapped on a machine, (g) a list of people who visited Riverstone's stall at a trade fair.
2. Transaction or report? (a) Payment 3: ₹33,260 against invoice 9002 on 2026-03-02. (b) "Overdue receivables by customer, as of 31 March 2026." (c) Order 5012, placed 2026-03-15, status Pending. (d) "Average days to collect in Q1: 31.6." (e) Ticket 208: "lid doesn't fit", opened 2026-03-18.
3. Which Riverstone system would you look in first to answer each question? (a) How many enquiries came through the website last week? (b) Has invoice 9007 been paid? (c) How many days of leave does Neha have left? (d) How many storage boxes are in stock at Bhiwandi Main? (e) Which quotes are still waiting for a customer's reply?
4. For each event, say whether it changes bookings, billings, collections, or none of them: (a) Metro Mart places an order. (b) Coastal Foods pays ₹20,000 against invoice 9006. (c) A customer asks for a quote. (d) Order 5011 ships and the ERP raises an invoice. (e) Order 5004 is cancelled before shipping. (f) A lead is marked *Qualified* in the CRM.

Core

5. February 2026 at Riverstone: orders 5005 to 5009 were placed (₹14,550, ₹14,640, ₹32,625, ₹23,325, ₹76,560), all shipped and invoiced in February, and none was cancelled. Payments received in February: ₹14,700 (invoice 9001), ₹40,000 (invoice 9002), ₹10,000 (invoice 9003). Calculate February's booked, billed, and collected amounts, and explain in two sentences why collections are so much lower than billings.
6. A department head writes this KPI definition: "*Sales growth: sales this month compared with last month.*" List at least four things that are missing or ambiguous, then rewrite it as a complete definition using the six questions in section 3.5.
7. In March 2026, three orders were placed: 5010 (₹20,100, Delivered), 5011 (₹11,700, Shipped), and 5012 (₹26,220, Pending). Only 5010 and 5011 were invoiced. Calculate (a) the cancellation rate for March, (b) average order value on billings, and (c) average order value on bookings. Explain why (b) and (c) differ.
8. List every manual handover in this process and name its type (re-keying, copy-paste, emailed file, manual matching, or shadow system): "*An employee fills in a paper expense form with receipts, and their manager signs it. The employee scans and emails it to finance. A finance assistant types each line into the ERP and checks the total against the receipts. Every Friday, the assistant copies the week's claims into the Finance Manager's budget spreadsheet.*"
9. Riverstone's finance team types 25 supplier invoices a day into the ERP, taking about 3 minutes each, on 22 working days a month. How many hours a month is that? Show your working, and name two costs of this work that the hours don't capture.

Stretch

- l0. It's 31 March 2026. Northgate Distributors placed order 5009 on 25 February: 60 industrial crates at ₹1,450 with a 12% discount. Invoice 9008 was raised on 26 February with 30-day terms, due 28 March. Northgate paid ₹30,000 on 20 March. The order's status is *Shipped*. Answer: (a) Check the order value by hand. (b) How much is booked, billed, and collected for this order? (c) How much does Northgate still owe, and is it overdue? By how many days? (d) What would you check before calling Northgate about the balance?
- l1. Build a simple **KPI tree** for collections: write "Collections" at the top and break it into the two or three things that drive it, then break each of those down one more level. For each box at the bottom, name the department that can influence it.
- l2. Farah wants to offer Northgate Distributors 15% off another order of industrial crates, which cost ₹1,100 to make and list at ₹1,450. (a) Who approves it? (b) Calculate the gross margin per crate at list price, 12% off, and 15% off. (c) What other data should the approver look at, and what should they ask before agreeing?

Think about it (no calculation needed)

- l3. Riverstone is considering paying sales executives a commission of 1% of the bookings they bring in. Describe two ways this could reward the wrong behavior, and suggest a better basis for the commission, or a second KPI to pair with it.
- l4. At the monthly business review, the Sales Head wants "sales" to mean bookings and the Finance Manager wants it to mean billings. Who should decide, and how would you present the numbers so that the meeting can move on?
- l5. The Sales Manager makes ten CRM fields mandatory before a lead can be saved, including the customer's annual turnover. What will happen to the quality of those fields, and what would you suggest instead?

Key terms

department · lead · quote / quotation · order · delivery challan / delivery note · picking list · proof of delivery (POD) · invoice · due date · payment terms · payment · made to stock · lead to cash · order to cash · business system · ERP · CRM · HRMS · POS · e-commerce platform · support desk / ticketing system · ticket · system of record / source of truth · integration · transaction · report · data warehouse · bookings · billings · collections · receivables / accounts receivable · revenue · cut-off · metric · KPI · KPI definition · leading indicator · lagging indicator · gross margin · average order value (AOV) · cancellation rate · overdue · dashboard · decision rights · re-keying · copy-paste integration · reconciliation / manual matching · shadow system · KPI tree

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 4, Numbers Without Fear**, teaches the percentages, averages, and growth rates behind every KPI in section 3.5.
 - **Chapter 5, Thinking Like an Analyst**, turns vague questions like "why is January low?" into precise ones, the way Meera did.
 - **Chapters 10 and 12** put the order-to-cash records into tools: a spreadsheet sales tracker, then the ERP's `orders`, `invoices`, and `payments` tables in SQL, where you'll calculate booked, billed, and collected yourself.
 - **Chapters 19 and 20** automate the monthly report: macros and Apps Script first, then scheduled email reports and alerts.
 - **Chapter 23, Business Acumen, KPIs & Metrics**, builds a full KPI tree for Riverstone and adds finance and operations metrics such as days sales outstanding.
 - **Chapter 25, The Business Analyst Track**, maps Riverstone's order-to-cash process formally and writes requirements for an improvement.
 - **Chapters 45 and 51** connect the systems: moving data from the ERP and CRM into a warehouse, and sending results back into them.
 - **Chapter 58** automates the re-typing of emailed purchase orders with AI, with a person checking uncertain cases.
 - **Interview preparation:** metric definitions, KPI trees, and business-process questions appear in Chapter 75 (product sense, metrics, and case studies) and Chapter 76 (the Business Analyst question bank), with model answers.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Warehouse and dispatch. (b) Purchasing. (c) HR (payroll). (d) Customer support. (e) Sales. (f) Production. (g) Marketing.
2. (a) Transaction: one payment event. (b) Report: a summary of many invoices and payments at a cut-off date. (c) Transaction: one order. (d) Report: an average over several invoices. (e) Transaction: one support ticket.
3. (a) The CRM, where the website form creates leads. (b) The ERP, where payments are recorded against invoices. (c) The HRMS. (d) The ERP, but its stock is updated once a day (section 3.2), so for an urgent answer also check with Bhiwandi Main. (e) The CRM.
4. (a) Bookings. (b) Collections. (c) None: a quote isn't a sale. (d) Billings. (e) Bookings go down if you count bookings net of cancellations (the KPI definition in section 3.5); nothing is billed or collected. (f) None: it's a leading indicator, not a sale.

5. Booked: ₹14,550 + ₹14,640 + ₹32,625 + ₹23,325 + ₹76,560 = **₹161,700**. Billed: all five were invoiced in February, so also **₹161,700**. Collected: ₹14,700 + ₹40,000 + ₹10,000 = **₹64,700**. Collections are lower because all three payments were for January's invoices; February's invoices have 30-day terms and weren't due until March.

6. Missing: which "sales" (booked, billed, or collected); whether cancellations, returns, and tax count; whether "compared" means rupees or percent; which date decides the month; the source system; the owner and target. One good rewrite:

Question	Answer
Name	Monthly billings growth
Formula	(billings this month – billings last month) ÷ billings last month, as a percentage to one decimal
Inclusions and exclusions	invoices raised; net of discount; tax excluded; credit notes subtracted in the month they're raised
Time	invoice date decides the month; calendar months
Source	ERP, invoices table
Owner, target, review	Finance Manager; target set in the annual plan; reviewed at the monthly business review

7. (a) 0 cancelled out of 3 placed: **0%**. (b) ₹20,100 + ₹11,700 = ₹31,800, divided by 2 invoiced orders = **₹15,900**. (c) ₹58,020 ÷ 3 orders placed = **₹19,340**. They differ because order 5012 (₹26,220), the largest of the three, is booked but not yet invoiced. Neither is wrong; a report must say which it uses.

8. (1) Paper form and receipts: data created on paper instead of in a system. (2) Scanning and emailing the form to finance: **emailed file**. (3) Typing each line into the ERP: **re-keying**. (4) Checking totals against receipts: **manual matching**. (5) Copying the week's claims into the Finance Manager's spreadsheet: **copy-paste** into a **shadow system** (the budget tracker lives outside the ERP).

9. $25 \times 3 = 75$ minutes a day. $75 \times 22 = 1,650$ minutes a month. $1,650 \div 60 = 27.5$ **hours a month**, more than three working days. Costs the hours don't show: typing errors (paying a supplier the wrong amount), delay (untyped invoices make reports of what Riverstone owes out of date), and dependence on the people who do it.

10. (a) $60 \times ₹1,450 = ₹87,000$; 12% off is ₹10,440; $₹87,000 - ₹10,440 = ₹76,560$. ✓ (b) Booked ₹76,560 (in February); billed ₹76,560 (in February); collected ₹30,000 (in March). (c) $₹76,560 - ₹30,000 = ₹46,560$ owed. It was due on 28 March, so on 31 March it's **3 days overdue**. (d) Whether the goods were delivered (the status is still *Shipped*, so Northgate may be waiting for them), whether a payment arrived but isn't recorded, and whether there's an open complaint. Three days overdue after a part-payment calls for a polite reminder, not an escalation.

11. One good tree (other sensible splits are fine):

- **Collections**

- **Billings:** orders shipped (sales, warehouse) · average order value (sales) · invoicing accuracy (finance)
- **Share paid on time:** payment terms and credit checks (sales, finance) · disputes from wrong deliveries or missing proof of delivery (warehouse, support)

- **Overdue recovered:** reminders and escalation (finance, sales)

When collections drop, you can ask which branch moved, and each branch has an owner.

12. (a) Over 10%, so the Sales Head, Anita Rao, after checking with the Finance Manager, Suresh Menon. (b) At list price: $(₹1,450 - ₹1,100) \div ₹1,450 = 24.1\%$. At 12% off: price ₹1,276; $(₹1,276 - ₹1,100) \div ₹1,276 = 13.8\%$. At 15% off: price ₹1,232.50; $(₹1,232.50 - ₹1,100) \div ₹1,232.50 = 10.8\%$. 15% off leaves less than half the list-price margin. (c) Northgate's payment record (₹46,560 overdue, exercise 10), the volume the discount would win, and competitors' prices. A sensible question before agreeing: "Can we settle the overdue balance first, or tie the discount to paying on time?"

13. (1) Reps can book large orders that are later cancelled: bookings rise, no cash arrives. (2) Reps are pushed toward customers who pay late, or heavy discounts at poor margins. Better: pay on **collections**, or on billings less cancellations and long-unpaid invoices; or pair bookings with **cancellation rate** and **gross margin**.

14. The managing director, because the definition affects everyone who reads the pack (often finance proposes and management approves). Meanwhile, don't pick a winner: show **booked, billed, and collected** as labeled lines with definitions and sources, reconciled, as Meera did.

15. People fill mandatory fields with anything that gets past the screen: "0", "NA", or a guess at turnover. The fields become full but untrustworthy, which is worse than blank (Chapter 1 lists this problem for data typed by people). Better: require only what's known at that stage (name, contact, interest), collect the rest later, use pick-lists, and offer an "unknown" option.

Chapter 4. Numbers Without Fear

Chapter at a glance

You will learn to: calculate percentages forward and backward, and see why a rise and an equal fall don't cancel · tell percentage points from percent change · choose the right denominator for a ratio or rate · measure growth month by month and year by year, and calculate compound growth and CAGR · choose between mean, median, and mode, and use weighted averages · round without changing the story · read tables and charts without being fooled · think about probability as "how often, out of how many" · estimate quickly and sanity-check any number · spot the number tricks in business news.

Before you start: Chapter 1 (especially section 1.6, levels of measurement) and Chapter 3 (bookings, billings, KPIs).

Time needed: 4–5 hours, including the exercises and the project.

Tools: a calculator (your phone's is fine), a pen, and a notebook. A spreadsheet is optional; the *Spreadsheet link* notes show each calculation in Excel and Google Sheets.

Practice data: Riverstone's 2025 sales from the one-year database: monthly revenue, targets, and margins, and 173 orders. The companion workbook `numbers_practice.xlsx` (Appendix E) holds the same numbers with every formula from this chapter. Every number was checked by script.

Why this matters

Data work is mostly arithmetic. Not advanced math: percentages, averages, growth rates, and a little probability. The difficulty isn't the calculation. It's knowing *which* calculation answers the question, and noticing when a number that looks right is wrong.

Here are three sentences a manager at Riverstone could say in a meeting, each built on a real 2025 number:

- "Revenue grew 13.1% a month on average." True arithmetic, badly misleading. The steady monthly rate that gets from January to December is 7.3%.
- "Margin fell 3.9% in November." It fell 3.9 percentage points, which is a 14.0% fall.
- "Our average order is ₹24,243." That's an average of monthly averages. The real average order is ₹25,061, and half of all orders are below ₹21,375.

None of these is a lie. Each is a small slip that changes what people decide. By the end of this chapter you'll catch all three on sight, and you'll never be the person who makes them.

This chapter has no code. Every idea is worked by hand with real numbers, because being able to check a number on the back of an envelope is what makes you trustworthy when the tools do the work later.

In plain English

A shop has a sign: "50% off, and an extra 20% off at the till!" How much do you save?

It's tempting to add them: 70%. But the second discount is taken from the *already reduced* price. On a ₹1,000 item, 50% off leaves ₹500. Then 20% off ₹500 is ₹100 off, leaving ₹400. You pay 40% of the original price, so you save **60%**, not 70%.

That one example holds the most important idea in this chapter: **every percentage is a percentage of something**. When the "something" changes, the same percentage means a different amount. Most number mistakes in business, and most number tricks in the news, come from losing track of what the percentage is of.

Keep asking two questions and you'll be right far more often than wrong:

1. **Of what?** What's the base, the denominator, the starting point?
 2. **Compared with what?** Last month, last year, the target, another group?
-

4.1 Percentages: of, back, and change

A **percentage** is a fraction out of 100. 12% means 12 out of every 100, or 0.12. There are three everyday calculations, and it helps to name them.

1. A percentage of a number. Multiply by the percentage as a decimal.

"What is a 12% discount on an Industrial Crate listed at ₹1,450?" $0.12 \times ₹1,450 = ₹174$. The crate sells for $₹1,450 - ₹174 = ₹1,276$, which is the same as $₹1,450 \times 0.88$. (That's order 5009's price per crate, from Chapter 3.)

2. What percentage one number is of another. Divide the part by the whole.

"What share of 2025 revenue came from Sharma Hardware?" $₹502,775 \div ₹4,335,471 = 0.116$, or **11.6%**.

3. Percent change. The change, divided by the starting value:

$$\text{percent change} = (\text{new} - \text{old}) \div \text{old}$$

"How did November's revenue compare with October's?" October was ₹681,071, November ₹633,408. $(₹633,408 - ₹681,071) \div ₹681,071 = -0.070$, or **-7.0%**.

Working backward

"Northgate paid ₹1,276 per crate after a 12% discount. What was the list price?"

The tempting wrong answer is to add 12% back: $₹1,276 \times 1.12 = ₹1,429.12$. That's wrong because the 12% was taken off ₹1,450, not off ₹1,276. The right way: ₹1,276 is 88% of the list price, so the list price is $₹1,276 \div 0.88 = ₹1,450$. ✓

Watch out: undoing a percentage. To reverse "x % off", divide by $(1 - x \%)$. To reverse "x % added", divide by $(1 + x \%)$. Never add or subtract the same percentage again.

Rises and falls don't cancel

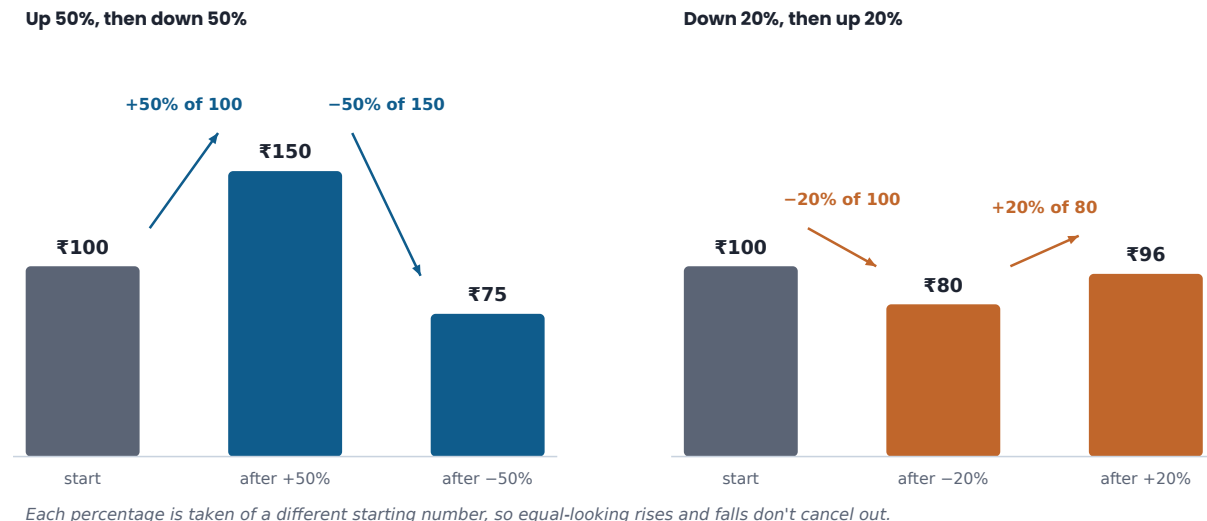


Figure 4.1 – A 50% rise followed by a 50% fall leaves you at 75, not 100.

Figure 4.1 shows why. Up 50% takes ₹100 to ₹150. Down 50% is half of ₹150, which is ₹75. Down 20% then up 20% leaves ₹96. The fall and the rise are each calculated on a different base.

Two consequences come up all the time:

- **Recovering from a fall needs a bigger rise.** After a 50% fall, you need a 100% rise to get back. After a 20% fall, you need 25%.
- **Discounts stack by multiplying.** A 10% trade discount followed by an extra 5% for early payment is $0.90 \times 0.95 = 0.855$ of the price, a total of **14.5% off**, not 15%.

Spreadsheet link. In Excel and Google Sheets, with the old value in B2 and the new value in C2: percent change is $=(C2-B2)/B2$, formatted as a percentage. A list price from a discounted price in B2 is $=B2/(1-12\%)$. The monthly sheet of `numbers_practice.xlsx` calculates every month's percent change this way.

4.2 Percentage points and percent change

When the thing that changes is *itself* a percentage, there are two different ways to describe the change, and mixing them up is one of the most common errors in business writing.

Riverstone's gross margin (Chapter 3, section 3.5) was **27.9%** in October 2025 and **24.0%** in November.

- The difference is $27.9 - 24.0 = 3.9$ **percentage points**. A **percentage point** is the unit for the plain difference between two percentages.

- The **percent change** is $(24.0 - 27.9) \div 27.9 = -14.0\%$. The margin is 14.0% smaller than it was.

Both are correct. They answer different questions, and they sound very different. "Margin fell 3.9%" is wrong: it mixes the size of one with the unit of the other, and it makes the fall sound less than a third as big as it was.

Here are three more from Riverstone's data:

Measure	Before	After	Change in points	Percent change
Gross margin, January → February 2025	29.2%	26.0%	−3.1 points	−10.8%
Revenue as % of target, August → September 2025	102.9%	146.9%	+44.0 points	+42.8%
Win rate, clean leads → duplicated leads (Chapter 13)	20.0%	14.0%	−6.0 points	−30.2%

(The first row shows −3.1 points, not $29.2 - 26.0 = 3.2$, because it's calculated from the unrounded margins, 29.18% and 26.04%. Section 4.6 is about exactly this kind of rounding.)

Which should you report? Use **points** when you talk about the level of a rate: "conversion fell 6 points, from 20% to 14%". Use **percent change** when you want the relative size: "a 30% drop in conversion". Whenever you can, give the before and after values too, and the reader can see both.

Watch out: "percent" for a change in a percentage. When a rate, share, margin, or interest rate moves, say "percentage points" (or "points") for the difference. If a loan's interest rate goes from 8% to 9%, it rose 1 point, which is a 12.5% increase in what you pay in interest.

4.3 Ratios and rates: always ask about the denominator

A **ratio** compares two quantities by dividing one by the other. A **rate** is a ratio where the bottom is a unit of something else: per day, per customer, per 1,000 people. The bottom number is the **denominator**, and choosing it is the whole skill.

Here are Riverstone's three customer segments in 2025:

Segment	Revenue	Orders	Customers	Revenue per customer	Average order value
Wholesale	₹1,702,658	55	6	₹283,776	₹30,957
Retail	₹1,488,774	59	8	₹186,097	₹25,233
Hospitality	₹1,144,039	59	9	₹127,115	₹19,390
Total	₹4,335,471	173	23	₹188,499	₹25,061

Hospitality has the most customers and ties for the most orders, but the least revenue. Which segment is "biggest" depends entirely on the denominator you pick:

- **By customers:** Hospitality (9).
- **By revenue:** Wholesale (₹1,702,658).

- **Per customer:** Wholesale again, and by a lot. A wholesale customer brings in ₹283,776 ÷ ₹127,115 = **2.23 times** as much as a hospitality customer.

Rates also make different-sized periods comparable. ₹4,335,471 in a year is about **₹11,878 per day** (÷ 365), and 173 orders is about **3.3 orders per week** (÷ 52). Neither is a number anyone at Riverstone reports, but both are useful sanity checks: if someone claims a single ordinary day brought in ₹2 lakh, you know to ask what happened that day.

A **share** (or proportion) is a ratio where the part is inside the whole, like Sharma Hardware's 11.6%. Shares of a whole add up to 100%. Ratios between separate groups, like 2.23 times, don't add up to anything.

Try it. Riverstone collected ₹197,250 of ₹297,710 billed in the first quarter of 2026 (Chapter 3). What's the collection rate? If next quarter it's 75.0%, how many points is that up, and what percent change?

4.4 Growth, compounding, and CAGR

Month-over-month growth

Here is Riverstone's revenue for every month of 2025, with the percent change from the month before. (Chapter 13 calculates this same table in SQL.)

Month	Revenue	Change from previous month
January	₹202,640	
February	₹253,664	+25.2%
March	₹278,008	+9.6%
April	₹210,282	−24.4%
May	₹329,359	+56.6%
June	₹186,928	−43.2%
July	₹232,692	+24.5%
August	₹329,282	+41.5%
September	₹558,315	+69.6%
October	₹681,071	+22.0%
November	₹633,408	−7.0%
December	₹439,824	−30.6%
Year	₹4,335,471	

The monthly changes swing wildly, from −43.2% to +69.6%. Most of that is the calendar: monsoon months are slow and the festive season is busy. Month-over-month percentages exaggerate seasonal patterns, which is why businesses with more than a year of data also compare each month with the

same month last year.

The average of growth rates is a trap

"What was the typical monthly growth in 2025?"

The tempting method is to average the eleven monthly changes. They add up to 143.8, and $143.8 \div 11 = 13.1\%$. It sounds reasonable. It's wrong, and you can prove it: start with January's ₹202,640 and grow it by 13.1% eleven times. You get **₹782,621** for December. The real December was ₹439,824.

The problem is Figure 4.1 again. A +56.6% month and a -43.2% month don't cancel, because each is a percentage of a different base. Averaging percentages that compound on each other always overstates growth when the numbers bounce around.

The right question is: *what single, steady monthly rate would take ₹202,640 to ₹439,824 in eleven steps?* That's the **compound growth rate**:

$$\text{compound growth rate} = (\text{end} \div \text{start})^{(1 \div \text{number of periods})} - 1$$

$₹439,824 \div ₹202,640 = 2.17$. The eleventh root of 2.17 is 1.073. So the compound monthly growth rate is **7.3%**. Grow ₹202,640 by 7.3% eleven times and you land exactly on December.

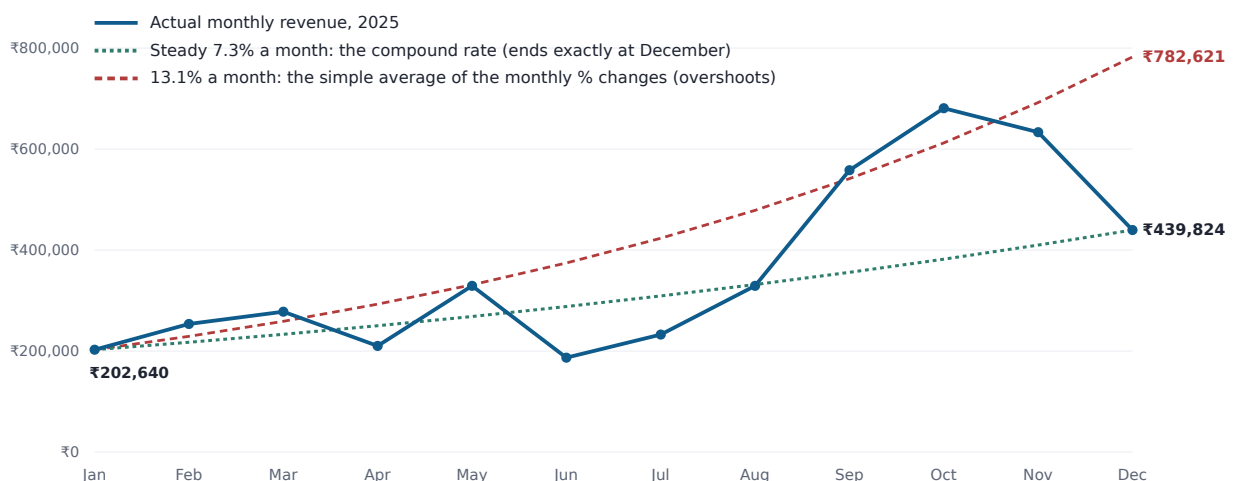


Figure 4.2 — The average of the monthly changes (red) overshoots December by ₹342,797. The compound rate (green) connects the real start and end.

"The eleventh root" sounds hard. You won't do it by hand; any calculator with a power key does it as $2.17^{(1/11)}$, and the spreadsheet functions are below.

Watch out: start and end points drive compound growth. The compound rate only uses the first and last values. From June (₹186,928, the lowest month) to October (₹681,071, the highest), revenue grew **264.3%**, a number that's true and tells you almost nothing about the year. Always ask why a growth figure starts and ends where it does.

Compounding

Compounding means growth that builds on previous growth. ₹100 growing 10% a year becomes ₹110, then ₹121, then ₹133.10 after three years. Simple growth, adding ₹10 a year, would give ₹130. The extra ₹3.10 is growth on growth. Over short periods the gap is small. Over long periods it dominates: that's why a loan's interest and a company's growth rate both matter so much over ten years.

A handy shortcut is the **rule of 72**: a quantity growing at $r\%$ a year doubles in about $72 \div r$ years. At 12%, about 6 years (the exact answer is 6.12). At 8%, about 9 years (exactly 9.01). It's for quick thinking in meetings, not for reports.

CAGR

CAGR (compound annual growth rate) is the compound growth rate when the periods are years. It's the standard way to describe growth over several years in company reports, investor decks, and interviews.

"Anita wants the business in the one-year database to reach ₹60 lakh (₹6,000,000) of revenue by 2028, three years after 2025's ₹4,335,471. What growth rate does she need each year?"

1. Ratio of end to start: $₹6,000,000 \div ₹4,335,471 = 1.384$.
2. Three years, so take the cube root: $1.384^{(1/3)} = 1.1144$.
3. Subtract 1: **CAGR = 11.4% a year.**

Check it by growing forward: $₹4,335,471 \times 1.1144 = ₹4,831,418$ in 2026, ₹5,384,098 in 2027, and ₹6,000,000 in 2028. ✓

Two tempting shortcuts both get the plan wrong:

- **Splitting the gap evenly.** $₹6,000,000 - ₹4,335,471 = ₹1,664,529$, or ₹554,843 a year. That's 12.8% of 2025's revenue, but a fixed rupee amount is a smaller percentage each year, so it isn't a growth rate at all.
- **Rounding down to a nice number.** 10% a year for three years reaches ₹5,770,512, which is ₹229,488 short. Over several years, one percentage point matters.

Spreadsheet link. Compound growth in both Excel and Google Sheets: `= (end/start)^(1/periods)-1`, or the built-in `=RRI(periods, start, end)`. `=RRI(3, 4335471, 6000000)` returns 0.1144, and `=RRI(11, 202640, 439824)` returns 0.0730. The plain average of monthly changes is `=AVERAGE(F3:F13)` on the monthly sheet, which returns the misleading 0.1307.

4.5 Averages: mean, median, mode, and weighted

An **average** is one number that stands for many. There are three common kinds, and Chapter 1 (section 1.6) showed that the level of measurement decides which ones make sense. This section shows how to choose between them for amounts, like order values, where all three are allowed.

- The **mean** adds up the values and divides by how many there are. It's what most people mean by "average".
- The **median** is the middle value when the values are sorted. Half are below it, half above.
- The **mode** is the most common value.

Riverstone received 173 orders in 2025 (not counting the two that were cancelled), worth ₹4,335,471 in total.

- **Mean:** ₹4,335,471 ÷ 173 = **₹25,061**.
- **Median:** sort the 173 order values; the 87th is **₹21,375**.
- **Mode:** order values are almost never exactly equal, so the mode of the values themselves is useless here. The mode is useful for categories and repeated counts instead: the most commonly ordered product is the Storage Box 10L (on 76 order lines), the most common quantity on a line is 15, and the most common discount is 0% (140 of 326 lines).

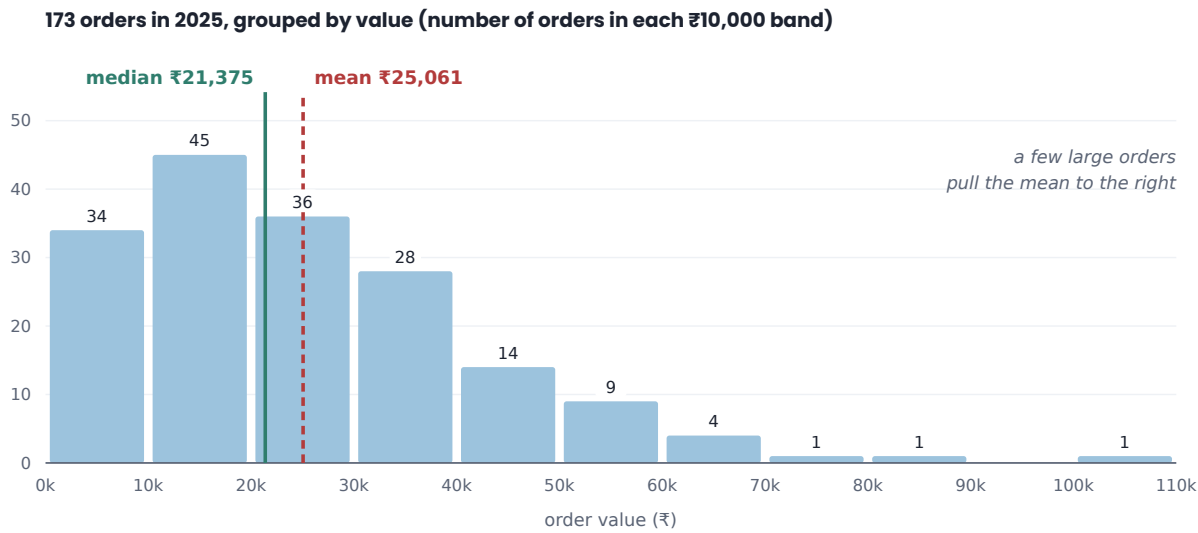


Figure 4.3 – Most orders are small; a few large ones stretch the tail to the right and pull the mean above the median.

Why is the mean higher than the median? Look at Figure 4.3. Most orders are under ₹40,000, but a handful are much larger: the biggest five are ₹65,818, ₹68,875, ₹74,218, ₹82,250, and ₹100,278. Large values pull the mean toward them; they don't move the median at all, because the median only cares about which value is in the middle. Data with a long tail on the high side is called **right-skewed**, and it's everywhere in business: order values, salaries, house prices, time to resolve a ticket.

The result: only **70 of the 173 orders (40.5%)** are above the "average" order. If a sales executive is told the average order is ₹25,061, most of their orders will feel below average.

Which to use?

Situation	Use	Why
Amounts that add up to a total you care about (revenue, cost, hours)	mean	mean × count = total, so it connects to budgets and plans
Amounts with a long tail (order values, salaries, delivery times)	median, alongside the mean	tells you what's <i>typical</i> ; the gap between them shows the skew

Situation	Use	Why
Categories (products, payment methods, segments)	mode	the only average that works for nominal data (Chapter 1)
Ratings and ranks (ordinal data)	median, plus the count in each category	the gaps between ratings aren't equal (Chapter 1)

Chapter 21 adds measures of spread, like percentiles and the standard deviation, which tell you how far values stray from the average.

Weighted averages

"What's Riverstone's average discount?"

Every order line has a discount of 0%, 5%, 8%, 10%, or 12%. The simple average of the discount on all 326 lines is **4.06%**. But a 12% discount on a ₹50,000 line costs far more than 12% on a ₹1,000 line. To know how much discounting costs the business, each line's discount must count in proportion to its value. That's a **weighted average**:

weighted average = sum of (value × weight) ÷ sum of weights

Weighted by each line's value before discount, the average discount is **4.69%**. You can check it without a formula: the lines were worth ₹4,548,725 at list price and ₹4,335,471 after discounts, so discounts took ₹213,254, and $₹213,254 \div ₹4,548,725 = 4.69\%$. ✓ Larger lines tend to get larger discounts (5.3% on average for lines worth ₹20,000 or more, 3.8% for smaller ones), which is why the weighted figure is higher.

The same trap appears with prices. Riverstone's four product categories sold at average net prices of ₹435 (Storage), ₹341 (Kitchen), ₹1,273 (Industrial), and ₹1,121 (Furniture) per unit. The simple average of those four prices is ₹793. But the company sold 9,475 units for ₹4,335,471, an average of **₹458 per unit**, because Storage and Kitchen sold thousands of units and Furniture sold 30. **An average of averages ignores how many items stand behind each one.**

The average of averages

The monthly business review pack often shows a row of monthly averages. Averaging that row gives the wrong annual figure:

- Average of the twelve monthly average order values: **₹24,243**.
- Actual average order value for the year: $₹4,335,471 \div 173 \text{ orders} = \text{₹25,061}$.

The busy months (September to November, with 18 to 21 orders each and high order values) count the same as quiet January with 8 orders. To combine averages, go back to the totals: add up all the revenue, add up all the orders, then divide.

Spreadsheet link. `=AVERAGE(B2:B174)` and `=MEDIAN(B2:B174)` on the `orders` sheet give ₹25,061 and ₹21,375; `=COUNTIF(B2:B174, ">"&E1)` counts the 70 orders above the mean. A weighted average is `=SUMPRODUCT(values, weights)/SUM(weights)`, which is how the `discounts` sheet gets 4.69%. The functions have the same names in Excel and Google Sheets.

4.6 Rounding and significant figures

Rounding makes numbers readable. It also creates small puzzles that make careful readers distrust a report.

Shares that don't add up to 100%

Riverstone's 2025 revenue by segment, rounded to whole percentages:

Segment	Share (whole %)	Share (one decimal)
Wholesale	39%	39.3%
Retail	34%	34.3%
Hospitality	26%	26.4%
Total	99%	100.0%

Nothing is missing. The unrounded shares are 39.27%, 34.34%, and 26.39%, and each one rounded down a little. Rounding to one decimal happens to fix it here, but not always: the same thing can happen at any precision (exercise 8). You have three honest options: show one more decimal place, add a note ("shares may not add to 100% because of rounding"), or leave it and expect the question. Never quietly change one number to force the total, because then the table no longer matches the data.

Round at the end, not in the middle

Rounding in the middle of a calculation carries the error forward. Section 4.2's margin example showed it: from rounded margins, the January to February fall looks like 3.2 points; from the real ones, it's 3.1. Keep full precision in the spreadsheet or calculator, and round only what you show.

Significant figures and false precision

The **significant figures** of a number are the digits that carry meaning. ₹4,335,471 has seven. Does the reader need all seven? In a finance reconciliation, yes. In a meeting, "about ₹43 lakh" or "₹4.3 million" says the same thing and is easier to remember.

The opposite mistake is **false precision**: more digits than the data can support. "*Customers take an average of 31.6 days to pay*" (Chapter 3) is correct arithmetic from five invoices. With five invoices, "about a month" is more honest. The number of digits you write is a claim about how sure you are.

A rough guide for reports:

- **Money in tables:** whole rupees in detail tables; thousands or lakhs in summaries, with the unit stated in the header.
- **Percentages:** one decimal place, unless the numbers are small samples (then whole numbers or "about").
- **Averages of small counts:** round more, and give the count: "31.6 days (5 invoices)".

4.7 Reading tables and charts correctly

Before you read any number in a table or chart, read the frame around it. Five checks catch most misreadings:

1. **Units.** Rupees, thousands, lakhs, crores, or millions? Percent or percentage points? A column headed "Revenue (₹ lakh)" with a value of 43.4 means ₹4,340,000.
2. **Period and cut-off.** A month, a quarter, year to date? Chapter 3 showed that the same report run on different dates gives different numbers.
3. **Per-period or cumulative.** A **cumulative** (running total) line always goes up as long as the values are positive, even in a terrible month. December 2025's revenue fell 30.6%, but the year-to-date line still rose.
4. **Definition.** Booked, billed, or collected? With or without cancellations? (Chapter 3.)
5. **The axis.** Where does it start?

The last one deserves a picture.

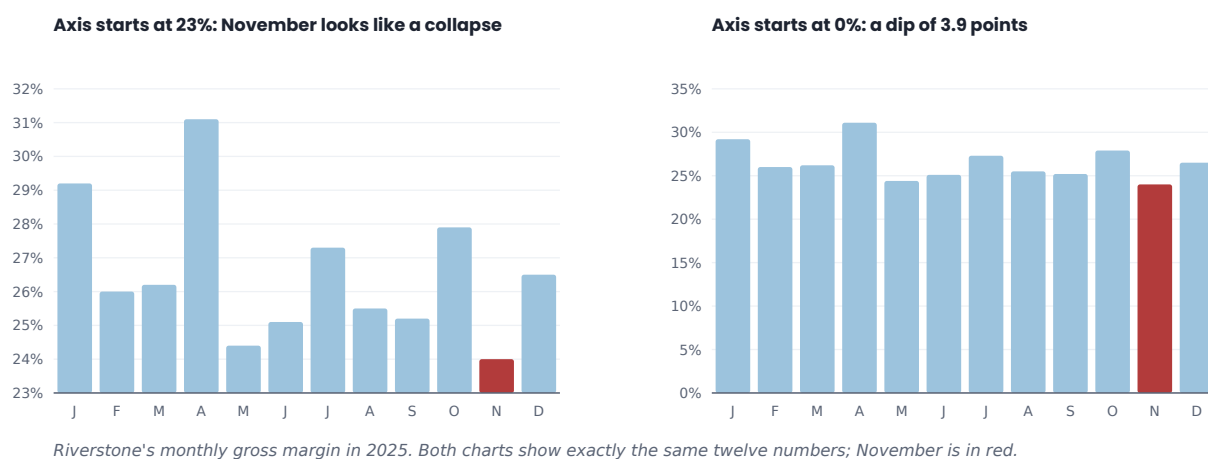


Figure 4.4 — The same twelve numbers. A bar chart whose axis doesn't start at zero turns a 3.9-point dip into what looks like a collapse.

In the left chart of Figure 4.4, the axis starts at 23%, so November's 24.0% bar is about a fifth the height of October's 27.9%. The eye reads "November's margin was a fifth of October's". In the right chart, starting at zero, the bars are 24 and 28 units tall, and the dip looks like what it is.

The rule: **bar charts must start at zero**, because readers compare bar *lengths*. Line charts can start elsewhere, because readers compare *positions* and *slopes*, but the axis should be labeled clearly. Chapter 15 covers chart choice and design in depth.

Watch out: one chart, two axes. A chart with revenue on a left axis and margin on a right axis lets the designer stretch either line to make them appear to move together. When you see two vertical axes, read each line against its own axis before believing the pattern.

4.8 Probability: how often, out of how many

A **probability** is how often something happens, out of how many chances. Written as a fraction, a decimal, or a percentage, it's always between 0 (never) and 1 (always), or 0% and 100%. You don't need formulas to use it well; you need to keep asking "out of how many?"

Counting what happened

In 2025, **2 of 175 orders** were cancelled. As a rate: $2 \div 175 = 1.1\%$, or about 1 order in 88. In the first quarter of 2026 (the mini database), 1 of 12 orders was cancelled: **8.3%**.

Did Riverstone's customers get seven times more likely to cancel? Almost certainly not. With 12 orders, a single cancellation moves the rate by 8.3 points. **Small counts make rates jumpy.** Before comparing two rates, look at how many cases each is based on; Chapter 22 shows how to judge whether a difference is bigger than chance.

"At least one"

"If each order has a 1.1% chance of being cancelled, what's the chance that at least one of the next 20 orders is cancelled?"

It's easier to work out the chance that *none* is cancelled and subtract from 1. The chance one order goes through is $1 - 0.0114 = 0.9886$. The chance all 20 go through is 0.9886 multiplied by itself 20 times, which is 0.795. So the chance of at least one cancellation is $1 - 0.795 = 20.5\%$. Using the first quarter's 8.3% rate instead gives **82.5%**. The rate you assume changes the answer completely.

(This assumes one order's cancellation doesn't affect another's, which is called **independence**. If one big customer cancels several orders at once, the assumption breaks.)

Chaining stages

Chapter 13's sales funnel took 30 unique leads to 22 contacted, 14 quoted, and 6 won. Each step has its own rate: 22 of 30 (73.3%), 14 of 22 (63.6%), and 6 of 14 (42.9%). The chance that a new lead is eventually won is the rates multiplied: $0.733 \times 0.636 \times 0.429 = 0.200$, or 20%, which matches 6 of 30. ✓

That makes planning concrete. To win 10 new customers at a 20% win rate, the sales team needs about $10 \div 0.20 = 50$ **leads**. To win more with the same number of leads, improve the weakest step.

The order of "given" matters

Of the 173 orders, 16 were worth more than ₹50,000, and 10 of those came from wholesale customers, who placed 55 orders in all.

- Chance an order is over ₹50,000: $16 \div 173 = 9.2\%$.
- Chance a **wholesale** order is over ₹50,000: $10 \div 55 = 18.2\%$. This is a **conditional probability**: the chance of one thing *given* another. The denominator shrinks to wholesale orders only.
- Chance a large order is **from wholesale**: $10 \div 16 = 62.5\%$.

The last two sound alike and differ by a factor of more than three, because the denominators differ. Mixing them up is one of the most common errors in medicine, law, and business alike: "most large orders are wholesale" is not the same as "most wholesale orders are large". Chapter 21 treats conditional probability and Bayes' rule properly.

4.9 Orders of magnitude and quick estimation

The **order of magnitude** of a number is its rough size in powers of ten: thousands, lakhs, millions, crores. Getting the order of magnitude right catches more errors than any formula, because the most damaging mistakes are the ones that are off by a factor of 10 or 100: a misplaced decimal, a figure in thousands read as rupees, an extra zero.

Indian and international units

Indian business writing uses lakhs and crores; international writing uses thousands, millions, and billions. You'll switch between them constantly.

Indian	Digits	International
1 lakh	100,000	100 thousand
10 lakh	1,000,000	1 million
1 crore (100 lakh)	10,000,000	10 million
100 crore	1,000,000,000	1 billion

Riverstone's 2025 revenue in the one-year database, ₹4,335,471, is about ₹43.4 lakh, ₹0.43 crore, or ₹4.3 million. This book writes rupees with international grouping (₹4,335,471), and mentions lakhs now and then for readers who think in them.

Sanity checks: does the number fit?

When a number arrives, check it against another number you already trust. Two examples from Riverstone:

- **Revenue per day.** ₹43 lakh a year is about ₹11,878 a day. Separately, 3.3 orders a week at ₹25,061 each is about ₹11,911 a day. Two different routes land within ₹50 of each other, so both numbers are probably sound.
- **Units from revenue.** October's revenue was ₹681,071, and the year's average price was ₹458 a unit. So October probably shipped about $681,071 \div 457.57 \approx 1,488$ units. The database says **1,625**. The estimate is 8.4% low, because October sold relatively more low-priced kitchen items and fewer

₹1,450 industrial crates than the year as a whole, but it's the right order of magnitude. If the database had said 16,250 or 162, you'd know something was wrong before looking at a single row.

Estimating from nothing

Sometimes there's no data yet: "Is it worth building a report for this?" or, in interviews, "How many plastic storage boxes are sold in Mumbai each year?" These are called **Fermi estimates** or **guesstimates**. The method is always the same:

1. Break the unknown into smaller pieces you can guess: households, share that buy boxes, boxes per purchase, purchases per year.
2. Write each assumption down with a round number.
3. Multiply, and round the answer to one or two significant figures.
4. Sanity-check the answer against anything you know, and say which assumption matters most.

Being close matters less than being clear. A reader can replace a bad assumption; they can't fix reasoning they can't see. Chapter 75 has worked guesstimates for interviews.

4.10 Number tricks in business news

Most misleading numbers in headlines, press releases, and sales decks are true. They mislead through what they leave out. Here are the common tricks, with invented examples, and the question that exposes each one.

Trick	Example	Question to ask
Tiny base	"Profits up 300%!" (from ₹2 lakh to ₹8 lakh)	<i>300% of what?</i>
Relative without absolute	"New process doubles the risk of a defect" (from 1 in 10,000 to 2 in 10,000)	<i>What are the actual numbers before and after?</i>
Cherry-picked period	"Revenue up 264% since June" (the slowest month to the busiest)	<i>Why start and end there? What about the same period last year?</i>
"Up to"	"Up to 70% off" (on one item)	<i>How many items, and what's the typical discount?</i>
Percent for points	"Interest rates rise 1%" (from 8% to 9%)	<i>Points or percent?</i>
Average hiding the spread	"Average order ₹25,061" (most orders are smaller)	<i>What's the median?</i>
Truncated axis	a bar chart starting at 23% (Figure 4.4)	<i>Where does the axis start?</i>
Mixed units	"₹43 lakh" next to "₹4.3 million" in the same slide	<i>Are these in the same unit?</i>
Record without context	"Our best month ever!" (October, in a seasonal business)	<i>Best compared with the same month last year?</i>
Survivors only	"Our customers grew 40% on average" (counting only customers who stayed)	<i>Who was left out?</i>

A good habit is to rewrite a headline number into a plain sentence with both numbers and the base: "Profit rose from ₹2 lakh to ₹8 lakh, a fourfold increase from a small base." If the rewritten sentence sounds much less exciting, the original was relying on a trick.

Interview extra point. When a case interview gives you a growth figure or a percentage, restate it with its base and period before you use it: "So revenue went from ₹20 lakh to ₹32 lakh over four years, which is about 12.5% a year compounded." It shows you check numbers before trusting them, which is what interviewers for analyst roles are testing. Chapter 73 (statistics and probability) and Chapter 75 (metrics, case studies, and guesstimates) have practice questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Adding percentages that apply one after another	"50% + 20% off = 70% off"	Multiply what remains: $0.5 \times 0.8 = 0.4$, so 60% off
Reversing a percentage by adding it back	list price from a discounted price comes out too low	Divide by $(1 - \text{discount})$
Saying "percent" for a change in a percentage	"margin fell 3.9%" when it fell 3.9 points	Use points for the difference; give before and after
Averaging growth rates that compound	typical monthly growth of 13.1% when the steady rate is 7.3%	Use $(\text{end} \div \text{start})^{(1/\text{periods})} - 1$, or RRI
Quoting growth between hand-picked months	"up 264%" from the slowest to the busiest month	Compare like with like: full years, or same month last year
Reporting only the mean for skewed data	most orders feel "below average"	Report the median too
Averaging averages	₹24,243 instead of ₹25,061	Go back to the totals and divide once
Using a simple average when items differ in size	average discount 4.06% instead of 4.69%	Weight by value, units, or count
Rounding in the middle of a calculation	small errors that grow, points that don't match	Keep full precision; round only what you show
Too many digits	"31.6 days" from five invoices	Match the digits to how sure you are; give the count
Bar charts with a truncated axis	a small dip looks like a collapse	Start bars at zero
Comparing rates from very different sample sizes	1.1% vs 8.3% cancellation read as a real change	Give the counts; be cautious with small samples
Confusing "A given B" with "B given A"	"most large orders are wholesale" read as "most wholesale orders are large"	Say the denominator out loud

In the real world: the board slide

It's the first week of January 2026. The managing director is presenting Riverstone's year to the board, and Vikram Singh's team has drafted the sales slide. Before it goes to the MD, Anita asks Meera Iyer to check the numbers. "Nothing fancy. Make sure nothing on it will embarrass us."

The draft slide has six claims. Meera takes them one at a time, and for each one asks: *of what, and compared with what?*

- 1. "2025 revenue up 117%."** Meera finds where it comes from: December's ₹439,824 against January's ₹202,640. That's true, but it compares two single months of a seasonal business, and it isn't "2025 revenue" at all. The slide has no 2024 figures to compare with, so it can't support any year-on-year growth claim. What the board can use: *full-year revenue ₹43.4 lakh (₹4,335,471), 102.3% of the annual target.*
- 2. "Average monthly growth: 13.1%."** The average of the eleven monthly changes. Meera grows January by 13.1% eleven times and gets ₹782,621 for December, almost double the real figure. The compound rate is 7.3%, but she recommends dropping the line entirely: in a year that peaks in October and dips in the monsoon, a "monthly growth rate" describes nothing real.
- 3. "Gross margin down 3.9% in November."** It was 27.9% in October and 24.0% in November: down 3.9 points, a 14.0% fall. More useful for a board: *full-year gross margin 26.2%; November was the lowest month.*
- 4. "Average order value: ₹24,243."** Meera recognizes the number from the monthly pack: it's the average of the twelve monthly averages. The real figure is $₹4,335,471 \div 173 = ₹25,061$. She adds the median, ₹21,375, because the MD is likely to be asked what a typical order looks like.
- 5. The segment pie chart: Wholesale 39%, Retail 34%, Hospitality 26%.** They add to 99%. A board member will notice. One decimal fixes it: 39.3%, 34.3%, 26.4%.
- 6. The margin bar chart.** Its axis starts at 23%, so November's bar is a stub. Meera redraws it from zero (Figure 4.4).

There's also a note under the slide's goal for 2028, ₹60 lakh: "That's about ₹5.5 lakh more each year." Meera adds the growth rate that the plan really needs, **11.4% a year**, because the board will think in percentages and "₹5.5 lakh a year" gets harder every year as a percentage.

She sends the corrected slide back with a four-line note explaining each change and the check behind it. Vikram's reply: "The 117% was the best number on the slide." Anita's: "It was also the one the board would have questioned first."

Nothing Meera did needed more than a calculator and two questions. What it needed was the habit of never passing a number on without knowing what it's a percentage of.

Tools

- **A calculator.** Your phone's calculator in landscape (scientific) mode has a power key ($\times^y$ or $^$) for compound growth.
- **A spreadsheet** (optional). Excel and Google Sheets both have every function used here: `AVERAGE` , `MEDIAN` , `MODE` , `SUMPRODUCT` , `ROUND` , `COUNTIF` , and `RRI` . Chapter 10 teaches them from the beginning.
- **The companion workbook** `numbers_practice.xlsx` (Appendix E). Its `monthly` , `orders` , `discounts` , and `segments` sheets hold the 2025 numbers from this chapter with the formulas already in place, so you can check every figure and try your own. It was generated from the one-year database, and its formulas were recalculated and compared with this chapter's numbers.

The project: check five statistics

Goal: take five numbers from the real world and check each one the way Meera checked the board slide.

Step 1. Collect five statistics. Find them in news articles, company annual reports or investor presentations, advertisements, or presentations at your workplace. Aim for variety: at least one growth figure, one percentage or share, one average, one chart, and one comparison between two groups. Copy the exact wording and note the source and date.

Step 2. Record each one in a table.

Column	What to write
<code>claim</code>	the exact words, e.g. "Sales up 40% in three years"
<code>source_and_date</code>	where and when it was published
<code>base</code>	what the number is a percentage <i>of</i> , or an average <i>of</i>
<code>comparison</code>	what it's compared with: which period, group, or target
<code>type</code>	percent change, points, share, ratio, average, probability, chart
<code>check</code>	your recalculation, or the missing information you'd need
<code>trick_found</code>	none, or which trick from section 4.10
<code>rewrite</code>	an honest one-sentence version with both numbers and the base

Step 3. Recalculate whatever you can from the numbers given. For growth over several years, work out the CAGR. For a percentage, find both the part and the whole. For a chart, check the axis and the units.

Step 4. Name what's missing. Many claims can't be checked because the base, the period, or the sample size isn't given. Saying exactly what's missing is a result.

Step 5. Rewrite each claim as one honest sentence.

Step 6. Write a three-line summary: how many of the five held up, the most common problem, and the one question you'll ask next time you see a number like it.

Deliverable: your table and summary. Keep it; Chapter 5 asks you to turn one of these claims into an analyst's question.

You've got it when...

- ☐ I can calculate a percentage of a number, a share, a percent change, and a list price from a discounted price.
 - ☐ I know that a rise and an equal fall don't cancel, and that discounts stack by multiplying.
 - ☐ I say "percentage points" for the difference between two percentages.
 - ☐ I ask what the denominator is before comparing ratios or rates.
 - ☐ I calculate compound growth and CAGR, and I never average growth rates that compound.
 - ☐ I choose between mean, median, and mode for a purpose, and use weighted averages when items differ in size.
 - ☐ I don't average averages.
 - ☐ I round only at the end, and I don't write more digits than the data supports.
 - ☐ I check a chart's units, period, and axis before reading its bars.
 - ☐ I can work out "at least one" and conditional probabilities by counting.
 - ☐ I sanity-check numbers by estimating them another way.
 - ☐ I've checked five real statistics and rewritten each one fairly.
-

Recap

- **Every percentage is a percentage of something.** Ask of *what?* and *compared with what?*
- Percent change is $(\text{new} - \text{old}) \div \text{old}$. Reverse a discount by dividing by $(1 - \text{discount})$. Rises and falls don't cancel, and discounts stack by multiplying.
- The difference between two percentages is measured in **percentage points**. Riverstone's margin fell from 27.9% to 24.0%: 3.9 points, a 14.0% fall.
- **Ratios and rates** depend on the denominator. Wholesale earned 2.23 times as much per customer as Hospitality, though Hospitality had more customers.
- **Compound growth** = $(\text{end} \div \text{start})^{(1/\text{periods})} - 1$. Riverstone's 2025 compound monthly rate was 7.3%; averaging the monthly changes gives a misleading 13.1%. **CAGR** is the yearly version: ₹43.4 lakh to ₹60 lakh in three years needs 11.4% a year. The **rule of 72** estimates doubling time.
- The **mean** (₹25,061 per order) is pulled up by large values; the **median** (₹21,375) shows what's typical; the **mode** suits categories. Use **weighted averages** when items differ in size (average discount 4.69%, not 4.06%), and never average averages.

- **Round at the end.** Shares may not add to 100%; say so or add a decimal. Don't claim more precision than the data has.
- **Read the frame first:** units, period, cumulative or not, definition, and where the axis starts. Bar charts start at zero.
- **Probability** is how often, out of how many. Small counts make rates jumpy. "At least one" = $1 - \text{chance of none}$. Chained stages multiply. "A given B" isn't "B given A".
- **Estimate** to catch errors of $10\times$ or $100\times$, and convert confidently between lakhs, crores, and millions.
- **Number tricks** leave out the base, the absolute numbers, the period, or the spread. Rewrite the claim with both numbers and the base.

Practice exercises

Warm-up

1. Calculate: (a) a 5% discount on an order worth ₹14,550; (b) what percentage ₹26,220 is of ₹58,020 (order 5012's share of March 2026 bookings, Chapter 3); (c) the price of a ₹780 Storage Box 25L after 8% off.
2. From Chapter 13's quarterly table: Kitchen revenue was ₹133,888 in the first quarter of 2025 and ₹461,146 in the fourth; Industrial was ₹275,450 in the third quarter and ₹292,040 in the fourth. Calculate each percent change.
3. Wholesale's share of revenue was 39.3% in 2025. If it's 42.0% next year, what's the change in percentage points, and what's the percent change? Write one correct sentence for each.
4. The 11 non-cancelled orders in the mini database (first quarter of 2026) were worth: ₹14,700, ₹73,260, ₹16,250, ₹14,550, ₹14,640, ₹32,625, ₹23,325, ₹76,560, ₹20,100, ₹11,700, ₹26,220. Find the mean, the median, and the mode. Which better describes a typical order, and why?

Core

5. (a) Northgate paid ₹76,560 for an order after a 12% discount. What was the order worth at list price? (b) Riverstone raises a price by 8%, then gives a customer 8% off the new price. Is the customer paying more or less than before, and by what percent?
6. A distributor's revenue grew from ₹25 lakh to ₹40 lakh over five years. (a) What was its CAGR? (b) Using the rule of 72, roughly how long would it take to double at that rate? (c) Why is "it grew 12% a year" ($60\% \div 5$) wrong?
7. Riverstone's 2025 gross margin by category was: Storage 26.9% on ₹2,297,974 of revenue; Kitchen 33.2% on ₹1,208,030; Industrial 13.6% on ₹795,830; Furniture 24.2% on ₹33,638. (a) What's the simple average of the four margins? (b) What's the margin weighted by revenue? (c) Which one is the company's gross margin, and why are they different?
8. The categories' shares of 2025 revenue, to one decimal place, are Storage 53.0%, Kitchen 27.9%, Industrial 18.4%, and Furniture 0.8%. A manager says the table is wrong. Explain what's happening and write the note you'd put under the table.

9. Riverstone wins 20% of its unique leads. (a) How many leads does it need to win 15 new customers? (b) If three new leads arrive this week and each has a 20% chance of being won, what's the chance at least one is won? (c) What assumption does (b) make, and when might it be false?

Stretch

10. A local newspaper profile of Riverstone runs the headline "Riverstone revenue soars 264%", based on June 2025 (₹186,928) and October 2025 (₹681,071). Explain in two sentences why the headline misleads, and write an honest replacement using numbers from this chapter.
11. November 2025's revenue was ₹633,408. Using the year's average price of ₹458 per unit, estimate how many units Riverstone shipped in November. The database says 1,355. How far off is your estimate, in percent? Is that good enough for a sanity check, and why?
12. In 2025, Riverstone received 29 orders worth ₹734,312 in the first quarter and 59 orders worth ₹1,754,302 in the fourth. (a) Calculate each quarter's average order value. (b) Calculate the average order value for the two quarters combined. (c) Explain why the average of your two answers in (a) isn't the answer to (b).

Think about it (no calculation needed)

13. A company's press release says, "Our employees earn an average of ₹18 lakh a year." What else would you want to know before believing that this describes a typical employee?
14. A sales dashboard shows only a year-to-date revenue line, and it has gone up every month. Explain how this chart could hide a bad month, and suggest a better chart to put next to it.
15. A sales manager announces, "Our win rate went up 50% this quarter!" Write the two questions you'd ask before sharing the news.

Key terms

percentage · percent of · share / proportion · percent change · reverse percentage · percentage point · ratio · rate · denominator · month-over-month growth · compound growth rate · compounding · rule of 72 · CAGR (compound annual growth rate) · average · mean · median · mode · right-skewed · weighted average · average of averages · rounding · significant figures · false precision · cumulative / running total · truncated axis · probability · independence · conditional probability · order of magnitude · lakh · crore · sanity check · Fermi estimate / guesstimate · base effect · relative vs absolute change

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 5, Thinking Like an Analyst**, turns the questions of *what?* and *compared with what?* into a method for breaking down any business question.

- **Chapters 10 and 11** build these calculations into spreadsheets: percentages, `SUMPRODUCT`, pivot tables of shares and averages.
- **Chapter 13** calculates month-over-month growth, running totals, and moving averages in SQL on the same 2025 data you used here.
- **Chapter 15, Data Visualization Principles**, goes deeper into honest charts: axes, chart choice, and the visual tricks from section 4.7.
- **Chapter 21, Descriptive Statistics & Probability**, adds spread, percentiles, distributions, and Bayes' rule.
- **Chapter 22, Statistics Without Fooling Yourself**, shows how to tell whether a difference between two rates is real or chance.
- **Chapter 23, Business Acumen, KPIs & Metrics**, applies growth rates, margins, and ratios to reading a company's financial statements.
- **Interview preparation:** percentage, growth, averages, and probability questions appear in Chapter 73 (statistics, probability, and experimentation), and guesstimates and metric questions in Chapter 75, with model answers.

Answers to practice exercises

(In the finished book these move to Appendix G.)

- (a) $0.05 \times ₹14,550 = ₹727.50$. (b) $₹26,220 \div ₹58,020 = 0.452$, so **45.2%**. (c) $₹780 \times 0.92 = ₹717.60$.
- Kitchen: $(₹461,146 - ₹133,888) \div ₹133,888 = +244.4\%$; revenue more than tripled. Industrial: $(₹292,040 - ₹275,450) \div ₹275,450 = +6.0\%$.
- $42.0 - 39.3 = 2.7$ **percentage points**: "Wholesale's share rose 2.7 points, from 39.3% to 42.0%." $(42.0 - 39.3) \div 39.3 = 6.9\%$: "Wholesale's share of revenue grew by 6.9%." The first is clearer for most readers.
- Total $₹323,930 \div 11 = \text{mean } ₹29,448$. Sorted: ₹11,700, ₹14,550, ₹14,640, ₹14,700, ₹16,250, **₹20,100**, ₹23,325, ₹26,220, ₹32,625, ₹73,260, ₹76,560; the sixth value is the **median, ₹20,100**. No value repeats, so there's **no mode**. The median describes a typical order better: two large wholesale orders (₹73,260 and ₹76,560) pull the mean up, and 8 of the 11 orders are below it.
- (a) $₹76,560 \div 0.88 = ₹87,000$. (Check: 12% of ₹87,000 is ₹10,440, and $₹87,000 - ₹10,440 = ₹76,560$. ✓) (b) $1.08 \times 0.92 = 0.9936$. The customer pays **0.64% less** than the original price, because the 8% discount is taken from a bigger number than the 8% rise was.
- (a) $₹40 \text{ lakh} \div ₹25 \text{ lakh} = 1.6$. $1.6^{(1/5)} = 1.0986$, so the CAGR is **9.9% a year**. (b) $72 \div 9.9 \approx 7.3$ **years** (the exact answer is 7.4). (c) $60\% \div 5 = 12\%$ ignores compounding: each year's growth is calculated on a bigger base, so a steady 9.9% a year is enough to add 60% in five years. Growing ₹25 lakh by 12% a year for five years would reach about ₹44 lakh, not ₹40 lakh.
- (a) $(26.9 + 33.2 + 13.6 + 24.2) \div 4 = 24.5\%$. (b) Weighted by revenue, the margin is **26.2%**: total revenue ₹4,335,471 minus total product cost ₹3,198,250, divided by revenue. (c) The weighted figure is the company's gross margin. The simple average gives tiny Furniture (under 1% of revenue) the same say

as Storage (more than half), and it gives low-margin Industrial (18.4% of revenue) a quarter of the weight, which drags the simple average down.

8. Nothing is wrong. The unrounded shares (53.00%, 27.86%, 18.36%, 0.78%) add up to 100%, but three of them rounded up, so the rounded shares add to 100.1%. Note: *"Shares are rounded to one decimal place and may not add to exactly 100%."* Don't adjust one share to force the total.

9. (a) $15 \div 0.20 = 75$ **leads**. (b) Chance none is won: $0.8 \times 0.8 \times 0.8 = 0.512$. Chance at least one is won: $1 - 0.512 = 48.8\%$. (c) It assumes the leads are **independent** and each has the same 20% chance. It's false if, for example, all three come from the same company or from a trade fair where one bad impression affects everyone, or if one lead is far warmer than the others.

10. It compares the slowest month with the busiest month of a seasonal business, so most of the "growth" is the calendar, and it says nothing about the year. A single-month comparison can't be called revenue growth for the company. Honest replacement: *"Riverstone's 2025 revenue was ₹43.4 lakh, 2.3% above its annual target, with a festive-season peak of ₹6.8 lakh in October."*

11. $₹633,408 \div ₹458 \approx 1,383$ **units** (using the unrounded ₹457.57, about 1,384). Against the actual 1,355, that's about **2.1% too high**. That's good enough: a sanity check is looking for errors of 10 times or 100 times, and an estimate within a few percent confirms the order of magnitude. Differences in the product mix explain the rest.

12. (a) First quarter: $₹734,312 \div 29 = ₹25,321$. Fourth quarter: $₹1,754,302 \div 59 = ₹29,734$. (b) $(₹734,312 + ₹1,754,302) \div (29 + 59) = ₹2,488,614 \div 88 = ₹28,280$. (c) The average of the two quarterly figures, ₹27,528, gives each quarter equal weight, but the fourth quarter had twice as many orders. The combined figure must come from the totals.

13. Whether "average" is the mean or the median (a few very high salaries, such as senior leaders', pull the mean up); the median and the range; who is included (full-time only? contractors? leaders?); whether it's salary alone or includes bonuses and benefits; and the date and location. The median, with the number of employees, describes a typical employee far better.

14. A cumulative line rises whenever a month's revenue is positive, so a month that fell sharply (like December 2025, down 30.6%) still shows as the line going up, only less steeply. Put a monthly bar chart next to it (starting at zero), ideally with each month's target or the same month last year, so a bad month is visible as a short bar.

15. (1) *From what to what?* A rise from 10% to 15% is "up 50%" but only 5 points; a rise from 2% to 3% is also "up 50%". (2) *Out of how many leads, and were they counted the same way?* With a small number of leads, one or two extra wins can move the rate a lot, and a change such as removing duplicate leads (Chapter 13) raises the win rate without any change in selling.

Chapter 5. Thinking Like an Analyst

Chapter at a glance

You will learn to: ask the questions that turn a request into useful analysis · rewrite a vague request as a precise question tied to a decision · write hypotheses that data can prove wrong · break a problem into an issue tree whose branches are MECE (no overlaps, no gaps) · tell facts from opinions and assumptions · check a claim or chart before believing it, including correlation that isn't causation · recognize the biases that bend how people read data, including your own · structure a decision so that data can inform it.

Before you start: Chapter 1 (the data-to-insight ladder), Chapter 3 (booked, billed, and collected), and Chapter 4 (percentages, averages, and small numbers).

Time needed: 3–4 hours, including the exercises and the project.

Tools: a pen and paper, or a whiteboard. Issue trees are easiest to draw by hand first.

Practice data: Riverstone's first quarter of 2026 (the mini database) for the worked example, and its 2025 CRM leads (the one-year database) for the story. Every number was checked by script.

Why this matters

Two analysts get the same message from the Sales Head: *"March was terrible. Can you look into it?"*

The first opens the database, pulls every number about March, builds twelve charts, and sends a 15-slide deck two days later. It's accurate. Nobody knows what to do with it.

The second replies with one question: *"Do you mean billed revenue, and is this for the plan you're making with the team next week?"* Then she lists five possible reasons revenue could have fallen, checks each against the data in an afternoon, rules out three, and sends a half-page note: *"Most of the fall is three customers who ordered in February and not in March. All three owe us money. I'd call them before offering anything."*

Both know the same tools. The second knows how to **think about the problem before touching the data**, and that's what separates analysts people ask for by name from analysts who produce reports.

It's a method, not a talent: ask what the question is for, list the possible answers before looking, break the problem into pieces that don't overlap, test each piece, and notice when your own expectations are steering you. Every later chapter gives you a new tool. This chapter decides whether you point it at the right thing.

In plain English

Think about a good doctor.

You walk in and say, "I feel terrible." A bad doctor prescribes something for "feeling terrible". A good doctor asks questions. *Since when? Where does it hurt? Any fever? Did you eat anything unusual?* In their head, they're listing possible causes and ruling them out one by one. Only then do they decide on treatment, and they tell you what to watch for in case they're wrong.

An analyst does the same thing with a business problem:

- **"I feel terrible"** is the vague request: *"Sales are down."*
- **The doctor's questions** turn it into a precise question: *which sales, compared with when, and what are you trying to decide?*
- **The list of possible causes** is a set of hypotheses, organized into an issue tree.
- **The tests** are queries, counts, and comparisons.
- **The diagnosis and treatment** are the conclusion and the recommendation.

A doctor who orders every test on every patient isn't thorough; they're lost. The same is true of an analyst who pulls every number.

5.1 Curiosity and asking good questions

Four kinds of question

Chapter 1 (section 1.2) introduced the ladder from data to insight. Business questions climb the same ladder, and knowing which rung a question is on tells you what kind of work it needs.

Kind	The question	Riverstone example	What it needs
Descriptive	What happened?	<i>What was billed revenue in March 2026?</i>	Counting and summarizing (Chapters 10–13)
Diagnostic	Why did it happen?	<i>Why did March fall 80.3% from February?</i>	Breaking down, comparing, testing hypotheses (this chapter)
Predictive	What will happen?	<i>What will April's revenue be?</i>	Patterns over time and models (Chapters 36 and 40)
Prescriptive	What should we do?	<i>Should sales offer a discount to customers who didn't reorder?</i>	Options, criteria, and judgment (section 5.8)

Most requests arrive as descriptive questions but are really diagnostic or prescriptive underneath. "What was March revenue?" usually means "Is March a problem, and what should we do about it?" Answering only the surface question is the most common way to do correct work that doesn't help.

The habit questions

Keep these in your head, or on a card next to your screen:

1. **What is this for?** What decision will the answer inform, and who makes it?
2. **Of what?** What exactly is being counted or measured? (Chapter 4)
3. **Compared with what?** Last month, last year, the target, another group? (Chapter 4)
4. **How do we know?** Where does the number come from, and can we trust it? (Chapter 1)
5. **So what?** If the answer is X, what changes? If nothing changes whatever the answer, why are we asking?
6. **What else could explain it?** Before accepting the first explanation, name at least one other.

The last one is the one people skip, and it catches the most mistakes.

5.2 From a vague request to a precise question

Real requests are almost always vague, because the people asking are busy and know what they mean. Your first job is to turn the request into a question precise enough that two analysts would answer it the same way.

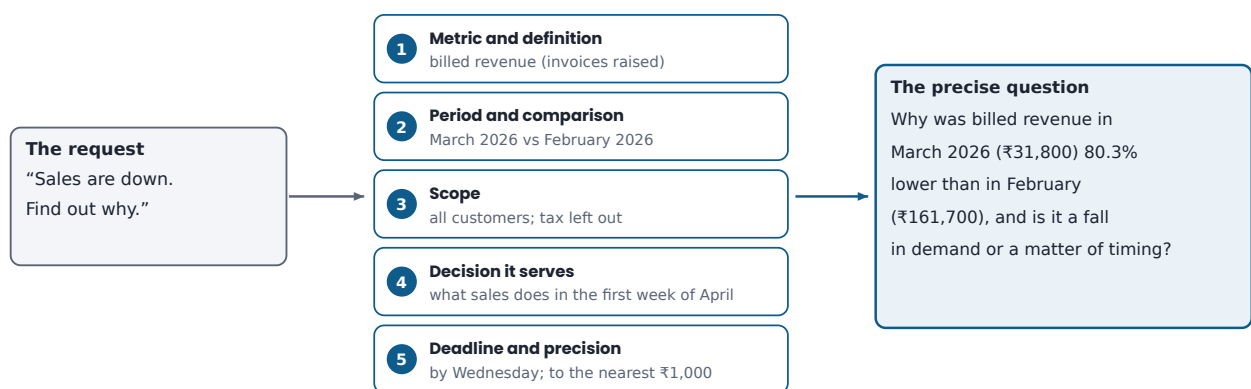


Figure 5.1 – Five checks turn a vague request into a question with one answer.

A precise question pins down five things:

1. **The metric and its definition.** "Sales" could be booked, billed, or collected (Chapter 3). Name one.
2. **The period and the comparison.** March 2026 compared with February? With January? With the quarter's average? Each gives a different story: March was 80.3% below February, 69.5% below January, and 68.0% below the quarter's monthly average of ₹99,237.
3. **The scope.** All customers? One segment? Including the pending order or not?
4. **The decision it serves.** "What should the sales team do in the first week of April?" needs a different depth from "What do we tell the board?"
5. **The deadline and precision.** An answer by Wednesday to the nearest ₹1,000 is a different job from a perfect answer next month.

You don't always need to ask all five. Often you can choose sensibly, state it, and let the requester correct you: "I'm reading 'sales' as billed revenue and comparing March with February; tell me if you meant something else." That one sentence prevents most wasted work.

Chapter 3's question, made precise

In Chapter 3's story, the managing director asked, "Which one is right? And why did we collect nothing?" about January. Meera's first move was to turn "January sales" into three precise questions: What was booked in January? What was billed? What was collected? Once the questions were precise, the "disagreement" disappeared, because the two reports were answering different ones. **A surprising number of business arguments are two people answering different questions with correct numbers.**

Try it. Take this request from a store manager: "Our customers aren't happy. Can you check?" Write down three different precise questions it could mean, each with a metric, a period, and a comparison.

5.3 Hypotheses: possible answers you can test

A **hypothesis** is a possible answer to a question, stated clearly enough that data could show it's wrong. It's a guess with a test attached.

For "Why did March 2026 billed revenue fall 80.3% from February?" here are some hypotheses:

- **H1.** Fewer customers placed orders in March.
- **H2.** Customers placed orders of the same number but smaller size.
- **H3.** Orders were placed in March but haven't been invoiced yet.
- **H4.** Wholesale customers, who place the largest orders, didn't order in March.
- **H5.** March is always a slow month for this business.
- **H6.** A competitor cut prices and took our customers.

What makes a hypothesis useful

Weak	Strong	Why the strong one is better
"Something happened with customers."	"Customers who ordered in February didn't order in March."	It says which customers and what they did, so you can check it.
"Pricing is a problem."	"Customers who didn't reorder were offered smaller discounts than those who did."	It points to specific data (discounts by customer) that could prove it wrong.
"The market is tough."	"March is always slow: March revenue is lower than February in previous years too."	It names the evidence: March in earlier years.

A strong hypothesis is **specific** (who, what, when), **testable** with data you have or could get, and **falsifiable**: you can say in advance what result would prove it wrong. "The market is tough" can't be proved wrong by any number, which is exactly why people like saying it.

Write them down before you look

List your hypotheses before you open the data. If you look first, you'll find a pattern, build a story around it, and stop looking. Writing them first makes you check the ones you didn't expect, and records what you ruled out, which is often as valuable as what you found.

Some hypotheses can't be tested with the data you have. H5 needs March in earlier years, and the mini database only holds the first quarter of 2026. H6 needs information about competitors that no Riverstone table contains. Say so plainly, and name what would test them (last year's data from the ERP; a conversation with customers). **"This data can't answer that" is a legitimate finding.**

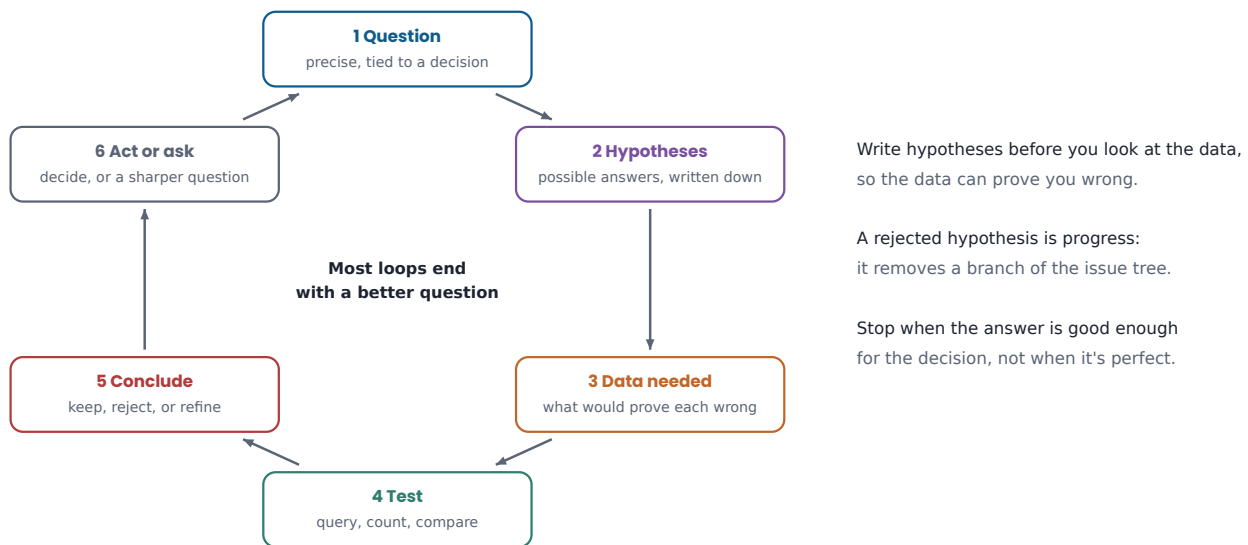


Figure 5.2 — The hypothesis loop. Most loops end with a better question, which starts the next loop.

Every analysis in this book follows the loop in Figure 5.2, whether the test is a pivot table, a SQL query, or a machine learning model.

5.4 Breaking problems down: issue trees and MECE

A list of six hypotheses is a start. But lists get long, overlap, and miss things. An **issue tree** organizes a question into branches, each branch into smaller branches, until every leaf is small enough to check with one piece of data.

MECE: no overlaps, no gaps

The branches of a good tree are **MECE** (pronounced "mee-see"): **mutually exclusive** (no two branches cover the same thing) and **collectively exhaustive** (together they cover everything). The term comes from management consulting, but the idea is plain logic.

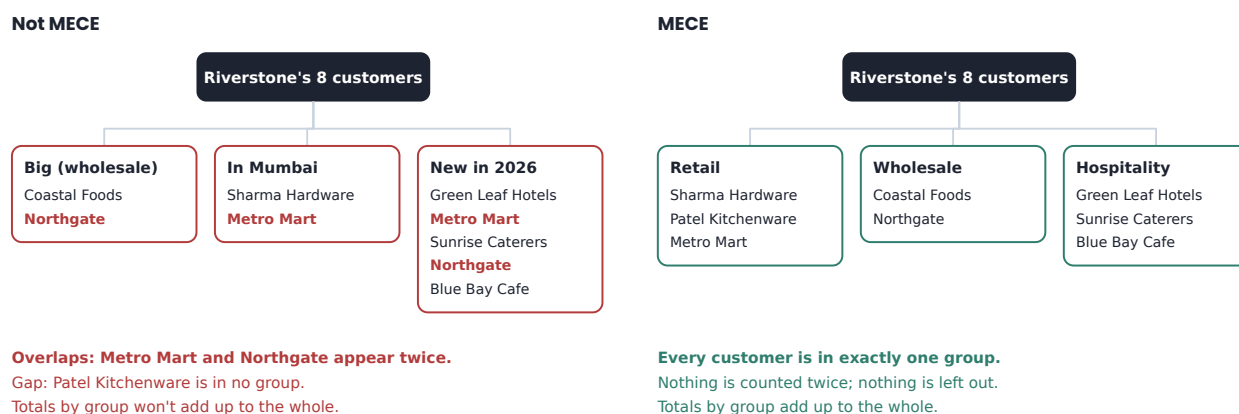


Figure 5.3 – The left split double-counts two customers and misses one. The right split is MECE.

In Figure 5.3, splitting customers into "big", "in Mumbai", and "new in 2026" puts Metro Mart and Northgate in two groups each and Patel Kitchenware in none, so group totals won't match the company's revenue, and a conclusion like "the problem is new customers" might really be about Mumbai. Split by segment instead, and every customer sits in exactly one group.

Four reliable ways to get MECE branches:

Split by	Example	Why it's MECE
A formula	billed revenue = number of invoiced orders × average order value	math can't overlap or leave gaps
A category with one value per record	segment, product category, region	each customer or product has exactly one
A process	lead → quote → order → invoice → payment (Chapter 3)	each step follows the last
Opposites	inside our control / outside it; new customers / existing	"not A" covers everything that isn't A

Splits like "people, process, technology" feel MECE but usually overlap. When in doubt, use a formula or a single-valued category.

Working through March

Here is the question from section 5.2, answered with an issue tree. All the numbers come from the mini database.

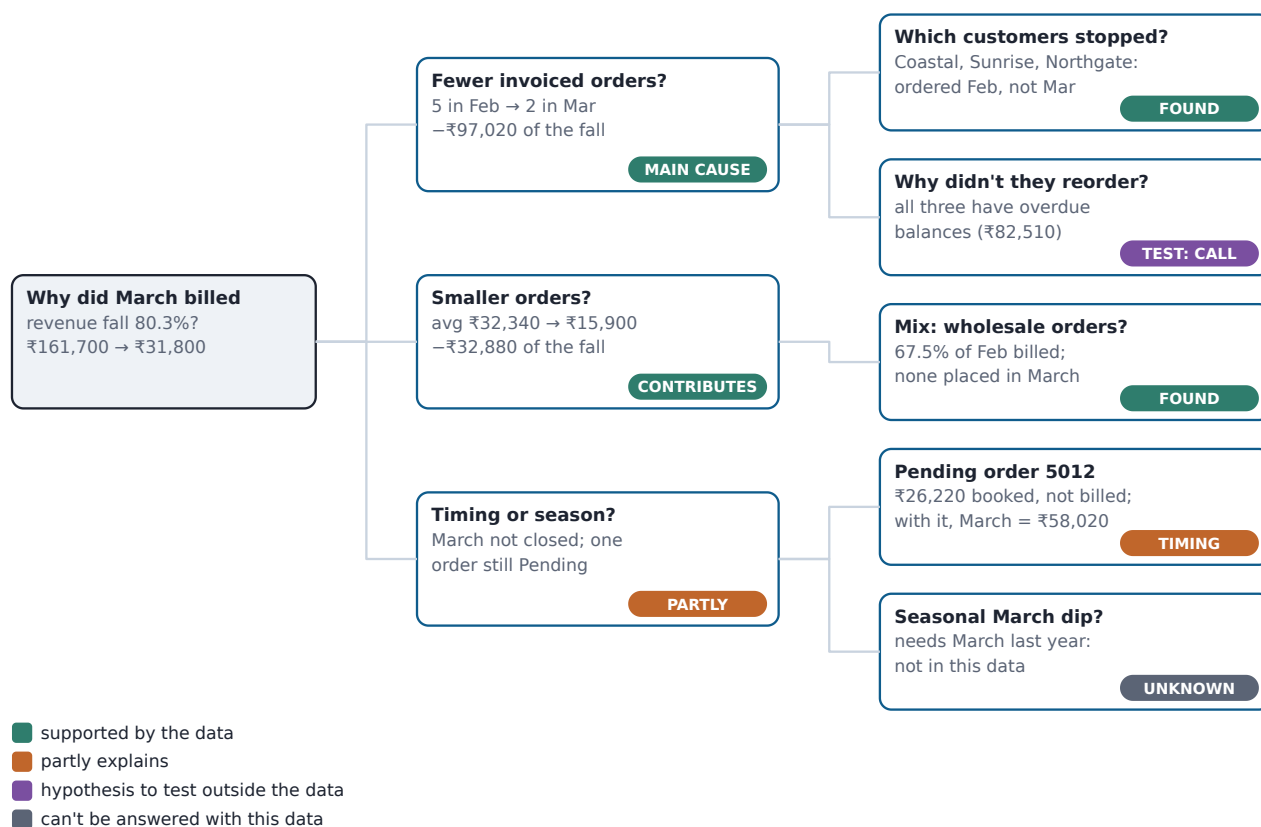


Figure 5.4 – The March issue tree, with what the data says about each branch.

Step 1. Split with a formula. Billed revenue = number of invoiced orders × average invoiced order value.

	Invoiced orders	Average order value	Billed revenue
February 2026	5	₹32,340	₹161,700
March 2026	2	₹15,900	₹31,800

Both parts fell. How much of the ₹129,900 fall does each explain? If March had kept February's average order value, 2 orders would have brought ₹64,680. So the drop in the **number** of orders accounts for ₹161,700 – ₹64,680 = **₹97,020** (74.7% of the fall), and the smaller **size** of March's orders accounts for the remaining ₹64,680 – ₹31,800 = **₹32,880** (25.3%). ✓ ₹97,020 + ₹32,880 = ₹129,900.

Simplification note. The shares depend slightly on whether you change count or size first; the ranking doesn't. Chapter 23 shows more careful ways to break down a change in a KPI.

Step 2. Fewer orders: which customers? Five customers had orders invoiced in February: Sharma Hardware, Metro Mart, Coastal Foods, Sunrise Caterers, and Northgate Distributors. In March, only Sharma Hardware and Green Leaf Hotels were invoiced. Metro Mart ordered in March, but its order is still pending. So three customers ordered in February and not at all in March: **Coastal Foods, Sunrise Caterers, and Northgate Distributors.**

Is that unusual? Coastal Foods ordered on 9 January and 11 February, 33 days apart; by 31 March it's been 48 days. Worth noticing, though two orders are a thin pattern (Chapter 4).

Step 3. Timing: is something stuck? Order 5012 from Metro Mart, worth ₹26,220, was placed on 15 March and is still *Pending*. It's booked, not billed. With it, March would be ₹58,020, still 64.1% below February. **H3 is true but explains only a part.**

Step 4. Smaller orders: the mix. February's two wholesale orders (Coastal Foods ₹32,625 and Northgate ₹76,560) were ₹109,185, or 67.5% of February's billed revenue. No wholesale customer ordered in March. Wholesale orders are the largest, so losing them shrinks both the count and the average. **H4 is supported.**

Step 5. Why didn't they reorder? The ERP's orders can't answer "why", but the invoices and payments can add a clue. On 31 March:

Customer	Ordered in March?	Owed	Overdue
Northgate Distributors	no	₹46,560	₹46,560
Sunrise Caterers	no	₹23,325	₹23,325
Coastal Foods	no	₹12,625	₹12,625
Patel Kitchenware	no (last order in January)	₹6,250	₹6,250
Sharma Hardware	yes	₹11,700	₹0 (not yet due)
Green Leaf Hotels	yes	₹0	₹0
Metro Mart	yes (pending)	₹0	₹0

Every customer with an overdue balance placed no order in March, and no customer who ordered in March had anything overdue. The three customers from step 2 owe ₹82,510 between them, all of it overdue.

That's a striking pattern, and it's exactly the moment to slow down. It doesn't say *which way* the connection runs, or whether there is one. A customer short of cash might stop ordering until it pays. A customer unhappy with a delivery might both withhold payment and stop ordering, so a single cause would explain both (section 5.6). Or, with seven customers, it could be coincidence. One detail points to a specific question: Northgate's February order still shows *Shipped*, not *Delivered*. If the crates never arrived, Northgate isn't a late payer; it's a customer waiting for its goods.

This branch ends as a **hypothesis to test outside the data**: a phone call to each of the three customers, and a check of the delivery records for Northgate.

Step 6. The branches the data can't reach. H5 (March is always slow) needs last year's March, which isn't in the mini database. H6 (a competitor) needs information from customers. Both go on the list of open questions, not in the conclusion.

What to tell Anita. A good answer is short, ranked, and honest about confidence:

"March billed revenue was ₹31,800, down 80.3% from February. About three-quarters of the fall is fewer orders: Coastal Foods, Sunrise Caterers, and Northgate didn't reorder in March, and February's two wholesale orders alone were 67.5% of that month. Metro Mart's ₹26,220 order is still pending; shipping it brings March to ₹58,020. All three customers who didn't reorder have overdue balances (₹82,510 in total), and Northgate's order still shows as not delivered. I'd call all three this week, starting with Northgate, before offering any discounts. I can't tell yet whether March is seasonally slow; that needs last year's data."

SQL link. Every number in this walk-through is a short query on the mini database (Chapter 12); Chapter 13's month-over-month and customer patterns do the same at scale.

5.5 Fact, opinion, and assumption

Meetings mix three kinds of statement, often in the same sentence. Separating them is one of the quickest ways to make a discussion productive.

- A **fact** is a statement that can be checked against a record, and has been. "March billed revenue was ₹31,800." (It still depends on a definition, so a good fact states it.)
- An **opinion** is a judgment. It may be wise, but it isn't checkable as stated. "Northgate is a difficult customer."
- An **assumption** is something treated as true without checking, usually because checking is hard or slow. "Northgate will pay next week."

Here are statements from Riverstone's Monday sales review, sorted:

Statement	Type	What to do with it
"March billed revenue was ₹31,800."	fact	Use it, with its definition.
"Coastal Foods orders about once a month."	a claim from very little data	Treat as a hypothesis: two orders, one gap of 33 days.
"Our prices are too high for wholesalers."	opinion	Turn it into a testable hypothesis: <i>wholesale customers who didn't reorder were quoted higher prices than those who did.</i>
"Northgate will pay next week."	assumption	Write it down, give it an owner, and check it by a date.
"Northgate is a difficult customer."	opinion	Ask what experience it's based on; check the delivery status first.

Businesses run on opinions and assumptions, because there's never time to check everything. The danger is when they're **presented as facts**, or an old assumption quietly becomes "what we know". List your assumptions in one place, so anyone can challenge them.

5.6 Checking claims and charts

Chapter 4 (section 4.10) covered number tricks. This section is about the reasoning behind a claim. Before accepting one, from anyone including yourself, ask five questions:

1. **Who says so, and how do they know?** A measured count, a survey, a guess, or an anecdote?
2. **Compared with what?** A number with no comparison can't be "high" or "low".
3. **How many cases, and which ones?** Five customers or five thousand? Chosen how?
4. **What's the mechanism?** Is there a believable reason why A would cause B?
5. **What else could explain it?** Chance, reverse causation, a common cause, or selection.

Correlation is not causation. Two things that move together are **correlated**. That doesn't mean one causes the other. There are three common alternatives:

- **Reverse causation: B causes A.** "Order lines with bigger discounts are bigger lines, so discounts make customers buy more." In the mini database, the four order lines worth ₹20,000 or more had an average discount of 10.5%; the other fifteen averaged 1.67%. But Riverstone's own rules (Chapter 3, section 3.6) give larger discounts to larger orders. The size comes first; the discount follows. The data can't show that discounts grow orders.
- **A common cause: C causes both.** In section 5.4, overdue balances and missing reorders went together. A delivery problem could cause both: the customer won't pay for goods it hasn't received, and won't reorder either. Chasing payment harder would then make things worse.
- **Selection: the cases were chosen in a way that creates the pattern.** "Customers who attend our trade fair order more." Maybe the ones who attend were already the most engaged customers.

And sometimes it's **chance**: with seven customers, patterns appear by accident. Chapter 22 shows how to judge whether a pattern is bigger than chance, and Chapters 30 and 31 cover how to test cause and effect properly, with experiments and without them.

Watch out: your own analysis is a claim too. The five questions apply to what you're about to send. The note to Anita in section 5.4 states a measured fact ("about three-quarters of the fall is fewer orders") and a recommendation ("I'd call them"), but it's careful not to say "customers stopped ordering *because* they owe us money".

5.7 Bias in how we see data

A **cognitive bias** is a predictable way in which people's judgment drifts from the evidence. Everyone has them, including experienced analysts. You can't switch them off, but you can recognize the common ones and build habits that catch them.

Bias	What it looks like at Riverstone	Antidote
Confirmation bias: noticing evidence that fits what you already believe	Vikram is sure a competitor is undercutting prices, so he asks for lost deals that mention price, and not for the ones that don't	Write hypotheses first (section 5.3); look for evidence that would prove your favorite wrong
Anchoring: judging a number against the first number you saw	March looks like a disaster against February's ₹161,700. But February was unusual: Northgate's first order alone was ₹76,560, 47.3% of the month. Against the quarter's monthly average of ₹99,237, March is still weak, but the comparison is fairer	Compare against several baselines: the previous month, the average, the same month last year, the target
Regression to the mean: an unusually high or low value tends to be followed by a more ordinary one	A month boosted by one big first order is likely to be followed by a lower month, even if nothing went wrong	Before explaining a change, ask whether the starting point was unusual
Survivorship bias: studying only the cases that made it through	Studying won deals to learn "what works", without looking at the lost and never-contacted leads that went through the same steps	Always include the cases that dropped out
Availability and recency: overweighting what's vivid or recent	One angry phone call from a customer on Friday becomes "customers are unhappy" on Monday	Count: how many complaints, out of how many customers, over what period?
Small numbers: trusting patterns from very few cases	"All the customers who owe us stopped ordering" is based on four customers	Give the counts; call it a hypothesis until more cases agree (Chapter 4, section 4.8)
The analyst's own bias: wanting an interesting finding	A clean story about overdue balances is more exciting than "one big order made February unusual", so it's tempting to lead with it	State the ordinary explanation first if it's the bigger one

The last row matters most. Analysts are rewarded for insights, so there's a pull toward the surprising story; a good reputation rests on being right, which often means reporting the ordinary explanation clearly.

5.8 Deciding with data

Analysis exists to help someone decide. Data rarely decides by itself; it narrows the options, estimates consequences, and shows where the uncertainty is.

Part	Question	For the three customers who didn't reorder
Decision	What exactly is being decided, and by whom?	What sales does about Coastal Foods, Sunrise Caterers, and Northgate this week (Anita decides)
Options	What are the choices, including doing nothing?	A. Wait. B. Call each customer to ask why, and check Northgate's delivery. C. Offer 5% off their next order. D. Put overdue accounts on hold until they pay.
Criteria	What matters in choosing?	Revenue, margin, cash owed, the customer relationship

Part	Question	For the three customers who didn't reorder
Evidence	What does the data say about each option?	All three owe ₹82,510, all overdue. Northgate's order isn't marked delivered. A 5% discount on an Industrial Crate cuts its margin from 24.1% to 20.1%.
Reversibility	How costly is it to be wrong?	A call is cheap and can't do much harm. A discount sets a price expectation that's hard to take back. A hold could lose a customer whose goods never arrived.
Recommendation and confidence	What should we do, and how sure are we?	B this week; decide on C or D after the calls. Medium confidence: the pattern is clear, but it's four customers and the cause is unknown.
What would change my mind	Which new fact would change the recommendation?	If customers say price is the reason and their balances are paid, reconsider C. If goods weren't delivered, fix the delivery before anything else.

Three principles sit behind the table:

- **Match the effort to the decision.** A reversible, cheap decision (a phone call) needs less evidence than an irreversible, expensive one (a price change or a hire).
- **Include "do nothing".** It's often the real alternative, and it has costs too.
- **Say what would change your mind.** It shows where the uncertainty is and what to watch.

Interview extra point. In a case interview or a take-home question ("Revenue fell 20%. Why?"), don't start calculating. Spend the first minute restating the question precisely, then sketch a MECE split out loud (for example, number of orders × average order value, then by segment), and say which branch you'd check first and why. Interviewers are grading the structure of your thinking more than the final number. Chapter 75 (product sense, metrics, and case studies) and Chapter 76 (the Business Analyst question bank) have practice cases with model answers.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Answering the surface question	correct numbers nobody can act on	Ask what the answer is for, and who decides
Starting with the data instead of the question	a pile of charts and no conclusion	Write the precise question and hypotheses first
Hypotheses that can't be wrong	"the market is tough" survives any evidence	Make each one specific, testable, and falsifiable
Branches that overlap or leave gaps	group totals don't add up to the whole	Split by a formula, a single-valued category, or a process
Stopping at the first explanation	a confident story that's partly true	Name at least one other explanation and check it
Reading correlation as causation	"discounts make customers buy more"	Check reverse causation, common causes, selection, chance

Mistake	Symptom	Fix
Comparing against one unusual baseline	a normal month looks like a disaster	Use several baselines: average, target, same month last year
Over-analyzing a small, reversible decision	a week spent on something a phone call would settle	Match the effort to the cost of being wrong

In the real world: do we need another sales executive?

It's the first week of January 2026. Vikram Singh, the Sales Manager, has asked the managing director for a fourth sales executive. His reason: *"Leads are piling up. The CRM shows 43 enquiries last year, and my team can't keep up."* The MD forwards it to Anita with one line: *"Is this right?"* Anita asks Meera Iyer to look before Friday.

1. What's the decision, and what's the precise question? The decision is whether to hire. The question underneath: *"Did Riverstone lose new business in 2025 because the sales team didn't have time to handle its leads?"* A hire fixes capacity, not a slow process, poor routing, or poor leads, so the question must separate those.

2. The issue tree, following the lead's journey:

- **Too many leads?** How many real enquiries arrived, per executive?
- **Leads missed or handled slowly?** How many were never contacted, and how long did first contact take?
- **Leads contacted but lost?** What share of contacted leads were won?
- **Capacity or something else?** If the team is too busy, the busiest people should be the ones missing leads.

3. The tests. All from the one-year database.

- **Volume.** The 43 rows include duplicates: the same enquiry submitted two or three times (Chapter 13). There were **30 real enquiries** in the year, about 2.5 a month, or fewer than one a month per sales executive. "Piling up" isn't about volume.
- **Missed and slow. 8 of the 30 (26.7%) were never contacted at all.** The other 22 waited an average of **9.7 days** for a first contact. One referral, Tulip Mart, has been waiting since 23 September, 99 days.
- **Lost after contact.** 6 of the 22 contacted leads were won (27.3%). Of the leads contacted within a week, 3 of 8 were won; of those contacted later, 3 of 14. That fits "faster is better", but with numbers this small it's a hypothesis, not a finding.
- **Capacity.** If the team were overloaded, the executives handling the most customer orders should miss the most leads. The data says the opposite:

Sales executive	Orders handled in 2025	Leads assigned	Never contacted	Average days to first contact
Farah Khan	64	5	0	7.7

Sales executive	Orders handled in 2025	Leads assigned	Never contacted	Average days to first contact
Rahul Mehta	53	10	3	10.9
Neha Kulkarni	46	15	5	9.8

Farah, with the most orders, missed none of her leads. Neha, with the fewest orders, was assigned half of all leads, including 9 of the 14 website enquiries, and missed 5. Website leads were the most often missed: 5 of 14 were never contacted.

4. Checking herself. The CRM doesn't record time spent on visits, calls, or complaints, so orders aren't the whole workload. And reading the table as "Neha is the problem" would be unfair: she got as many leads as Rahul and Farah combined, including most website enquiries. The pattern points at **how leads are routed and followed up**, not at a person.

5. The recommendation. She sends Anita a half-page note:

"The data doesn't support hiring for lead volume: 30 real enquiries came in last year (the CRM's 43 includes duplicates), fewer than one a month per executive. The problem is follow-up: 8 enquiries were never contacted and the rest waited almost 10 days on average. Missed leads are concentrated among website enquiries and in the largest lead list, not with the busiest executive. I'd (1) call the 8 uncontacted leads this week, starting with the Tulip Mart referral, (2) spread website leads evenly, (3) set a two-working-day rule for first contact with a daily reminder, and (4) fix the duplicate website submissions. What would change my mind: if leads grow sharply, or if response times are still slow after a quarter of the new routing, a hire is worth revisiting. The CRM doesn't record time spent, so I can't rule out that the team is busy with work outside orders and leads."

Anita forwards it to the MD and Vikram. Vikram's first reaction is irritation. His second, after reading the table, is to ask Meera how to set up the daily reminder (Chapter 20 does exactly that).

The request was a solution ("hire"). Meera turned it into a question about a cause, tested each branch, avoided blaming one person, stated what she couldn't see, and recommended cheap, reversible steps first, with a clear condition for revisiting the expensive one.

Tools

- **Pen and paper, or a whiteboard.** Issue trees are fastest by hand. Draw the first version in five minutes; tidy it later.
- **A spreadsheet or document** for the hypothesis log: one row per hypothesis, with the data needed, the result, and the status (supported, rejected, open).
- **A diagram tool** (optional): diagrams.net, PowerPoint, or Google Slides for sharing a tree.
- **SQL and spreadsheets** for the tests, from Chapter 10 onward. This chapter's numbers came from short queries on the Riverstone databases.

The project: an issue tree for a real question

Goal: take one real business question, make it precise, and build an issue tree that shows exactly which data would answer each branch.

Step 1. Choose a question from your work, a local business, or your Chapter 4 project: turn one checked claim into an analyst's question. "Sales up 40% in three years" becomes "Did the company's revenue grow faster than its market over those three years, and where did the growth come from?"

Step 2. Make it precise. Write the metric and its definition, the period and the comparison, the scope, the decision it serves, and the deadline (section 5.2).

Step 3. Write at least five hypotheses before looking at any data. Make each specific and testable (section 5.3).

Step 4. Build the tree two or three levels deep, starting with a MECE split (section 5.4). Check each split for overlaps and gaps.

Step 5. Add the data to every leaf. For each leaf, write:

Column	What to write
branch	the leaf's question
hypothesis	what you expect, stated so it could be wrong
data_needed	the table, file, report, or conversation that would test it
available	yes, no, or "needs to be collected"
test	the count, comparison, or query you'd run
result_that_rejects_it	what you'd have to see to drop this branch

Step 6. Label facts, opinions, and assumptions in anything the question's owner has said about it.

Step 7. Name one bias that could affect this analysis (yours or the requester's), and how you'll guard against it.

Deliverable: the precise question, the tree, the data table, and a three-sentence plan for which branch you'd test first and why. If you have access to the data, test one branch and write a half-page note like Meera's.

You've got it when...

- ☐ I ask what an analysis is for before I start it.
- ☐ I can turn a vague request into a precise question with a metric, period, comparison, scope, and decision.
- ☐ I write hypotheses before looking at the data, and each one could be proved wrong.
- ☐ I can build an issue tree with MECE branches, and I know four reliable ways to split.
- ☐ I label statements as fact, opinion, or assumption.

- ☐ I check claims for reverse causation, common causes, selection, and chance.
 - ☐ I can name the common biases and the habit that counters each one.
 - ☐ I can structure a decision with options, criteria, evidence, reversibility, confidence, and what would change my mind.
 - ☐ I've built an issue tree for a real question, with the data for every branch.
-

Recap

- Questions are **descriptive, diagnostic, predictive, or prescriptive**. Most requests are diagnostic or prescriptive underneath.
 - Keep asking: *what is this for? of what? compared with what? how do we know? so what? what else could explain it?*
 - A **precise question** pins down the metric, the period and comparison, the scope, the decision it serves, and the deadline.
 - A **hypothesis** is a possible answer that data could prove wrong. **Write hypotheses before you look**. "This data can't answer that" is a legitimate finding.
 - An **issue tree** breaks a question into branches until each leaf can be tested. Branches should be **MECE**: no overlaps, no gaps. Split by a formula, a single-valued category, a process, or opposites.
 - March 2026's 80.3% fall split into **fewer orders (₹97,020)** and **smaller orders (₹32,880)**, then into three customers who didn't reorder, the missing wholesale orders, and a pending order, ending in a hypothesis to test by phone.
 - Separate **facts, opinions, and assumptions**, and list assumptions where everyone can see them.
 - **Correlation isn't causation**. Check for reverse causation, a common cause, selection, and chance.
 - **Biases** (confirmation, anchoring, regression to the mean, survivorship, availability, small numbers, and the analyst's own) bend everyone's judgment; counter them with habits.
 - **Decide with data** by laying out options (including doing nothing), criteria, evidence, reversibility, a recommendation with confidence, and what would change your mind.
-

Practice exercises

Warm-up

1. Label each question as descriptive, diagnostic, predictive, or prescriptive: (a) *How many orders did Sunrise Caterers place in February?* (b) *Should we stop offering 12% discounts?* (c) *Why did Kitchen revenue grow every quarter of 2025?* (d) *How much will we collect in April?* (e) *Which customers owe us money?*
2. Label each statement as fact, opinion, or assumption, and say what you'd do with it: (a) "Invoice 9007 hasn't been paid." (b) "Sunrise Caterers is a bad customer." (c) "The new price list will be ready by Monday." (d) "Hotels don't care about price." (e) "Blue Bay Cafe signed up on 1 March 2026."

3. Is each split MECE? If not, say whether it overlaps, leaves gaps, or both. (a) Orders: Pending, Shipped, Delivered, Cancelled. (b) Customers: large, loyal, new. (c) Revenue: Storage, Kitchen, Industrial, Furniture. (d) Reasons for a late delivery: warehouse delay, truck problem, customer not available, bad weather. (e) Leads: from the website, from referrals, from Mumbai.
4. Rewrite each vague request as a precise question with a metric, period, comparison, and the decision it might serve: (a) "How are we doing with hotels?" (b) "Is the website worth it?" (c) "Check if customers are paying on time."

Core

5. For the question "Why did hospitality customers bring in less revenue in March 2026 than in February?", write three hypotheses that data could prove wrong, and for each, name the data you'd use. (Hospitality billed ₹23,325 in February and ₹20,100 in March in the mini database.)
6. Billed revenue rose from ₹104,210 in January 2026 (3 invoiced orders) to ₹161,700 in February (5 invoiced orders). (a) Calculate each month's average invoiced order value. (b) Split the ₹57,490 increase into the part from more orders (holding January's average order value) and the part from the change in order size. (c) Which explains more?
7. Chapter 3 showed that Riverstone collected ₹197,250 of ₹297,710 billed in the first quarter of 2026 (66.3%). Build a two-level MECE issue tree for "Why did we collect only two-thirds of what we billed?" For each leaf, name the data that would test it.
8. Name the bias in each situation and suggest one habit that would counter it: (a) After one customer complains about a cracked crate, a manager says, "Our quality has slipped." (b) An analyst studies the five best customers to learn why customers stay. (c) Revenue drops after a record month, and the team spends a week looking for what went wrong. (d) A manager who wanted a new CRM highlights only the figures that make the old one look bad.
9. A manager says: "Customers with overdue balances order less. So chasing payments hurts sales, and finance should send fewer reminders." Give three other explanations for the pattern, and describe what data would help decide between them.

Stretch

10. Using the table from the story: Farah handled 64 orders and missed none of her 5 leads; Neha handled 46 orders and missed 5 of her 15. (a) What does this data support? (b) What doesn't it support? (c) What additional data would you want before saying anything about individual performance?
11. In the story, leads contacted within a week were won 3 times out of 8 (37.5%); leads contacted later were won 3 times out of 14 (21.4%). (a) Why isn't this proof that faster contact wins more deals? (b) Name one alternative explanation. (c) How could Riverstone test the idea properly?
12. Blue Bay Cafe signed up on 1 March 2026 and hadn't placed an order by 31 March. Structure the decision "What should sales do about Blue Bay Cafe?" using the table in section 5.8: at least three options (including doing nothing), criteria, the evidence you'd look for, reversibility, and what would change your mind.

Think about it (no calculation needed)

- l3. When should you stop analyzing and give an answer? Describe two signals that you've done enough, and one signal that you haven't.
- l4. A senior manager says, "I've already decided to cut the Furniture range. Can you find me some numbers that support it?" How would you respond, and what would you offer to do instead?
- l5. You ask an AI assistant why March revenue fell, and it gives a fluent, confident explanation. Using this chapter, list three things you'd check before repeating it.

Key terms

descriptive question · diagnostic question · predictive question · prescriptive question · habit questions · precise question · hypothesis · testable · falsifiable · issue tree · MECE · mutually exclusive · collectively exhaustive · decomposition (count × size) · fact · opinion · assumption · claim · correlation · causation · reverse causation · common cause · selection · chance · cognitive bias · confirmation bias · anchoring · regression to the mean · survivorship bias · availability bias · recency · small-numbers bias · decision rights · reversibility · confidence · "what would change my mind"

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 6, Setting Up to Learn**, installs the tools you'll use to test hypotheses, and helps you build a study plan.
- **Chapters 10–13** give you the tests: spreadsheets and SQL to count, compare, and break down numbers the way section 5.4 did.
- **Chapter 14, Data Cleaning & Preparation**, handles the "is the data even right?" branch that every issue tree should include.
- **Chapter 22, Statistics Without Fooling Yourself**, shows whether a pattern like "3 of 8 versus 3 of 14" is bigger than chance.
- **Chapter 23, Business Acumen, KPIs & Metrics**, builds full KPI trees for Riverstone and diagnoses a revenue dip with more careful breakdowns.
- **Chapter 24, Requirements, Storytelling & Stakeholders**, turns notes like Meera's into memos and presentations, and covers handling "can you find numbers that support this?"
- **Chapters 30 and 31** test cause and effect properly: experiments, and methods for when experiments aren't possible.
- **Interview preparation**: case questions ("revenue fell; why?"), structuring, and hypothesis-driven thinking appear in Chapter 75 (product sense, metrics, and case studies) and Chapter 76 (the Business Analyst question bank), with model answers.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Descriptive. (b) Prescriptive. (c) Diagnostic. (d) Predictive. (e) Descriptive.
2. (a) Fact (checkable in the payments table); use it. (b) Opinion; ask what it's based on and turn it into something checkable, such as "Sunrise pays later than other customers". (c) Assumption (a plan); give it an owner and confirm it. (d) Opinion, and a sweeping one; turn it into a hypothesis, such as "hotel customers accept price increases as often as retailers". (e) Fact (the `signup_date` in the customers table); use it.
3. (a) MECE, as long as every order has exactly one of these four statuses. (b) Not MECE: it overlaps (a new customer can be large; a large customer can be loyal) and has gaps (a small, occasional, long-standing customer is none of them). (c) MECE: each product belongs to one category, and the four cover every product. (d) Not MECE: it can overlap (a truck problem caused by bad weather) and has gaps (a wrong address, missing stock, a paperwork delay). Add "other" and define each reason so that one is chosen. (e) Not MECE: "Mumbai" is a location, not a source, so a website lead from Mumbai is in two groups, and leads from trade fairs or cold calls are in none.
4. One good version of each: (a) "What was billed revenue from hospitality customers in the first quarter of 2026, compared with the previous quarter, and which customers drove the change? (For deciding on an April hotel promotion.)" (b) "How many 2025 website enquiries became customers, and how much revenue did they bring, compared with referrals and trade fairs? (For next year's website budget.)" (c) "What share of first-quarter 2026 invoices were paid by their due date, and which customers are most often late? (For deciding whom finance calls first.)"
5. Examples: (1) Fewer hospitality customers ordered in March than in February. Data: orders by customer and month (Sunrise Caterers ordered in February, Green Leaf Hotels in March). (2) Hospitality customers who ordered placed smaller orders. Data: order values by customer and month. (3) The hospitality order that would have made the difference is still pending or was cancelled. Data: order status for hospitality customers in March. Note that the gap is small (₹3,225) and each month has one order, so almost any difference is normal variation.
6. (a) January: $₹104,210 \div 3 = \text{₹}34,737$. February: $₹161,700 \div 5 = \text{₹}32,340$. (b) Holding January's average order value, 5 orders would bring $5 \times \text{₹}34,736.67 = \text{₹}173,683$, which is ₹69,473 more than January: that's the **count effect**, +₹69,473. The change in size is $5 \times (\text{₹}32,340 - \text{₹}34,736.67) = -\text{₹}11,983$. Check: ₹69,473 – ₹11,983 = ₹57,490. ✓ (c) The increase came entirely from **more orders**; the average order actually got a little smaller.
7. One good tree:
 - **Collected only 66.3% of billings**
 - **Not yet due** (the amount billed recently, within 30-day terms). Data: invoices by due date as of 31 March (invoice 9010's ₹11,700 isn't due until 10 April).
 - **Due but unpaid (overdue)**
 - Customers can't or won't pay. Data: overdue by customer; payment history; credit checks.
 - Customers dispute the invoice. Data: support tickets, delivery status, proof of delivery.

- *Riverstone hasn't chased.* Data: reminder logs from finance.
- **Paid but not recorded yet.** Data: bank statement lines not yet matched to invoices.

The first split (not yet due / overdue / paid but unrecorded) is MECE for the money not collected. From Chapter 3: ₹100,460 is unpaid, of which ₹88,760 is overdue and ₹11,700 is not yet due.

8. (a) Availability (one vivid complaint). Habit: count complaints over a period, out of how many deliveries. **(b) Survivorship** (studying only customers who stayed). Habit: compare with customers who left. **(c) Regression to the mean** (after a record month, a lower one is normal). Habit: compare with the average and the same month last year before searching for a cause. **(d) Confirmation bias.** Habit: write down in advance what evidence would show the old CRM is fine, and look for it.

9. (1) Common cause: a delivery or quality problem makes customers both withhold payment and stop ordering. **(2) Reverse causation:** customers who order less have less reason to keep their account current. **(3) Cash-strapped customers** pay late and order less, whatever the reminders. Data that helps: complaints and delivery records for overdue customers; whether ordering fell before or after the balance became overdue; and a few customer conversations.

10. (a) Missed leads weren't concentrated with the executive handling the most orders, so "too busy with orders" isn't supported, and lead assignment was very uneven. **(b)** Nothing about individual effort or ability: the numbers are tiny, Neha got most website leads, and orders aren't all of anyone's work. **(c)** Time spent on other work, each person's lead sources, how leads were assigned, and absences over the year.

11. (a) The counts are very small (6 wins in total), so the difference could well be chance (Chapter 22), and the leads weren't assigned a response time at random. **(b)** Executives may contact the most promising leads first, such as referrals, so the leads contacted quickly were already more likely to be won (selection, or a common cause: lead quality affects both speed and winning). **(c)** Run a simple experiment (Chapter 30): for a quarter, randomly assign new leads to "contact within two working days" or "normal process", then compare win rates, or at least compare fast and slow contact within the same lead source.

12. Decision: what sales does about Blue Bay Cafe in April. **Options:** A. wait; B. call or visit to understand its needs; C. a first-order offer; D. send the catalog and a sample. **Criteria:** chance of a first order, cost, margin, fit. **Evidence:** 30 days since signup without an order; what it enquired about; how long other new customers took (Green Leaf Hotels signed up on 8 January and placed its first order on 20 January). **Reversibility:** a call is cheap; a discount sets an expectation. **Recommendation:** B first, then D if interest is confirmed. **What would change my mind:** a specific, large first order that depends on price would make C worth considering.

13. Enough: more precision wouldn't change the decision; the remaining branches can't be answered with available data, and you've said so. Not enough: you can't yet explain most of the change, or the recommendation would flip on a branch you haven't checked.

14. Don't cherry-pick; it risks your credibility and the manager's. Offer a fair picture instead: "I'll look at Furniture's revenue, margin, and trend. If it supports cutting the range, that's a stronger case; if not, better you hear it from me than from the board." Chapter 24 covers handling this kind of pressure.

15. (1) Are its numbers right? Check the metric, definition, and figures against the database. **(2) Is it presenting hypotheses as facts?** Order tables can't tell you *why* customers behaved as they did. **(3) What has it left out?** Compare with your own issue tree: timing, mix, and the limits of the data. Chapter 26 covers working with AI assistants.

Chapter 6. Setting Up to Learn

Chapter at a glance

You will learn to: check whether your computer is ready, and what to do if it isn't · install the book's core tools in a sensible order, and confirm each one works · organize the companion files so you can always find them · use the keyboard, files, and terminal basics that save hours · read official documentation to settle a question for yourself · use AI assistants to learn faster without letting them do your thinking · build a study plan and a weekly habit you can keep · get the most from the exercises and answers.

Before you start: Chapters 1–5. Chapter 2 (files, folders, extensions, and paths) is especially useful here.

Time needed: 3–4 hours, most of it installing. Spread the installs over two or three sittings if you like.

Tools: the computer you'll study on, an internet connection, and permission to install software on it.

Practice data: the Riverstone mini database (`riverstone_setup.sql`) and a small script, `check_setup.py` , both in the companion files (Appendix E). Every version, install method, and output in this chapter was checked against official sources or run, in September 2026.

Why this matters

From Chapter 10 onward, every chapter asks you to type, run, break, and fix things: formulas, queries, scripts, dashboards. Readers who set up properly before they need to get through those chapters in a fraction of the time, because a missing driver or a wrong Python version never interrupts them halfway through an exercise. Readers who skip setup often lose an evening to an installation problem and quietly stop.

This chapter also covers the other half of setting up: **you**. A plan you can keep, a weekly rhythm, a way to use AI assistants that helps you learn instead of replacing the learning, and the habit of checking the official documentation when two sources disagree. It all decides whether you finish.

In plain English

Think of a kitchen before cooking a big meal. A good cook doesn't start chopping and then discover there's no gas in the cylinder. They check the stove, lay out the knives and pans, read the recipe once through, and put the spices within reach. Cooks call it *mise en place*: everything in its place.

Setting up to learn is the same:

- **The stove and utensils** are your computer and the tools: a spreadsheet, a database, Python, Power BI.
- **The pantry** is the companion files: the Riverstone data you'll practice on.
- **The recipe book** is this book, plus the official documentation for each tool, which is where you look when the recipe is unclear.
- **The meal plan** is your study plan: what you'll cook this week, and when.

You don't need the most expensive equipment, only the right equipment, working, and in reach.

6.1 The computer you need (and don't)

You don't need a new or high-end computer to become a data analyst. Almost everything in Parts 0 to II runs comfortably on an ordinary laptop from the last five or six years.

	Enough to start	Comfortable
Operating system	Windows 10 or 11, or a recent macOS	Windows 11 or the latest macOS your Mac supports
Memory (RAM)	8 GB	16 GB
Free disk space	20 GB	50 GB or more
Screen	a laptop screen	a second monitor, so the book or documentation sits next to your work
Internet	enough to download installers	enough for video calls and cloud tools later

Three situations need a little planning:

- **A Mac.** Everything in the book runs on a Mac except **Power BI Desktop**, which is Windows only. The usual answer in Microsoft's Q&A forums is to run Windows 11 in a virtual machine (for example, Parallels Desktop). The browser version of Power BI can view and interact with reports but can't build the data models Chapter 16 teaches, and it needs a work or school account. You can do everything else on your Mac and handle Chapter 16 separately.
- **A Chromebook, tablet, or phone.** These can run Google Sheets, Excel for the web, and online SQL playgrounds, which is enough for Chapters 10, 11, and much of 12 and 13. They can't run Python properly, the databases, or Power BI Desktop. Plan to borrow or buy a laptop before Chapter 17.
- **A work laptop you can't install software on.** Many companies lock their laptops, for good security reasons. Ask your IT team; learning tools like PostgreSQL, Python, and DBeaver are commonly approved. Some tools install without administrator rights: Microsoft's documentation notes that Power BI Desktop from the Microsoft Store doesn't need admin rights, and Python's install manager installs for your own user. The story later in this chapter shows one way through.

Watch out: Windows 10. Microsoft stopped regular security updates for Windows 10 in October 2025. It still runs every tool in this book, but a computer that no longer gets security updates is a risk when you start connecting to company data. If you can upgrade to Windows 11, do.

6.2 Your toolkit at a glance

The core toolkit, in the order the book first uses it



Later parts install their own tools when you reach them: dbt, the command line, cloud accounts, containers, and ML libraries.

Figure 6.1 — Five groups of tools cover the whole analyst path. Later parts add their own tools when you reach them.

Tool	What it's for	First used	Cost	Runs on	Version at the time of writing
Excel or Google Sheets	spreadsheets	Chapter 10	Excel for the web and Google Sheets are free with a Microsoft or Google account; desktop Excel needs a Microsoft 365 subscription	Windows, Mac, browser	always updated
PostgreSQL	the book's main database	Chapter 12	free	Windows, Mac, Linux	18 (the current major version, supported until November 2030)
MySQL Community Server	the second database	Chapter 12	free	Windows, Mac, Linux	9.7 LTS (8.4 LTS is also fine)
DBeaver Community	the app you type SQL into	Chapter 12	free	Windows, Mac, Linux	26.2
Power BI Desktop	dashboards and data models	Chapter 16	free (Microsoft describes the desktop app as free)	Windows only	updated monthly
Python	programming for data	Chapter 17	free	Windows, Mac, Linux	3.13 or 3.14
VS Code	the editor for Python, SQL files, and notes	Chapter 17	free	Windows, Mac, Linux	updated monthly
Git	saving versions of your work	Chapter 26	free	Windows, Mac, Linux	2.55

A few words on these choices. **Free tools only**, so nobody is locked out. **PostgreSQL and MySQL both**, because between them they cover most databases an analyst meets (Chapter 12 explains). **DBeaver**, because one app works with both databases. **Power BI**, because it's widely used and free to start with; Chapter 16 also translates the ideas to Tableau and Looker. **Python with VS Code**, because it's free, runs everywhere, and the same editor serves you from your first script to production code in Part V.

Simplification note: versions move. Tools release new versions every few months. The book's examples were tested on PostgreSQL 16, MySQL 8.0, and Python 3.11 and 3.13, and use features that work the same way in the newer versions listed above. If a newer version than the table shows is available when you read this, install it. Appendix B keeps current instructions.

6.3 Installing, in order

Install in this order. Each step ends with a check, so a problem shows up immediately, not three chapters later. Use the official website for every tool; download sites that repackage installers sometimes bundle unwanted software.

Step 1. A spreadsheet

If you already have Excel through work or a Microsoft 365 subscription, you're done. If not, either works:

- **Google Sheets:** sign in to a Google account and open Google Sheets in your browser.
- **Excel for the web:** sign in with a free Microsoft account at Microsoft 365 on the web. Files are saved in OneDrive, which comes with 5 GB of free storage.

Check: create a sheet, type `2.5` in A1 and `=ROUND(A1,0)` in B1. You should see `3`. (Section 6.6 explains why that's worth checking.)

Tool note. Microsoft ran a free *desktop* Excel editing preview in some markets until July 2026. It has ended: the free desktop apps now open and view files, and editing is free only in the web versions. Chapters 10 and 11 point out the few features that need desktop Excel.

Step 2. The databases and DBeaver

Chapter 12, section 12.3, has full, tested steps for both databases and DBeaver on Windows, macOS, and Linux. You don't need to repeat them here; do them now, and come back. A summary:

1. **Install PostgreSQL** from the official PostgreSQL website. On Windows, the page recommends the interactive installer by EDB, which also includes pgAdmin and StackBuilder (you won't need either). On a Mac, the same installer works, or Postgres.app, a small app that runs PostgreSQL from the menu bar. Choose the current version, and **write down the password** you set for the `postgres` user.
2. **Install DBeaver Community.** It includes its own Java, so there's nothing else to install.
3. **Install MySQL** (optional until Chapter 12's MySQL sections). Choose the current LTS release, and write down the `root` password.
4. **Create the database and load the data**, as in section 12.3: `CREATE DATABASE riverstone;` in PostgreSQL, then run `riverstone_setup.sql`; in MySQL, run `riverstone_setup_mysql.sql`, which creates the database for you.

Check: in DBeaver, connected to `riverstone`, run the same query in each database.

```
SELECT COUNT(*) FROM order_items;
```

```
count
-----
    19
(1 row)
```

```
SELECT COUNT(*) FROM order_items;
```

```
+-----+
| COUNT(*) |
+-----+
|        19 |
+-----+
```

Nineteen order lines: the same twelve orders you followed through Chapters 1 to 5. If you see an error instead, the troubleshooting notes in section 12.3 cover the common ones, including MySQL's "Public Key Retrieval is not allowed".

Step 3. Python and the analyst's packages

Install Python from python.org:

- **Windows:** python.org now recommends the **Python install manager**, available from the python.org downloads page or the Microsoft Store (they're the same tool). The older full installer is being retired. After installing, open a terminal (section 6.5) and run `py install 3.14` if it didn't install a version for you. The commands `python` and `py` then start Python.
- **Mac:** download the macOS installer package from python.org and run it. When it finishes, open the new Python 3.14 folder in *Applications* and double-click **Install Certificates.command**, which lets Python download packages securely. On a Mac, the command is `python3`.
- **Linux:** use your distribution's packages or python.org's source; most distributions already include a recent Python 3.

Create a folder and a virtual environment for the book. A **virtual environment** is a private copy of Python's package list for one project, so installing something for this book can't break anything else on your computer. Chapter 17 explains it properly; for now, follow the steps. In a terminal, go to your book folder (section 6.4) and run:

```
# Windows (PowerShell or Command Prompt)
python -m venv .venv
.venv\Scripts\activate
python -m pip install pandas openpyxl matplotlib jupyterlab

# macOS or Linux (Terminal)
python3 -m venv .venv
source .venv/bin/activate
python -m pip install pandas openpyxl matplotlib jupyterlab
```

When the environment is active, your terminal prompt starts with `(.venv)`. Each time you open a new terminal to work on the book, run the `activate` line again. If PowerShell refuses to run the activation script, the Python documentation's fix is to run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` once, or to use Command Prompt instead.

Check: run the companion script `check_setup.py` from the `ch06` folder:

```
python check_setup.py
```

On a correctly set-up computer, it prints:

```
python      3.13.13  OK
pandas      3.0.5    OK
openpyxl    3.1.5    OK
matplotlib  3.11.2   OK
jupyterlab  4.6.3    OK
All set.
```

Your version numbers may be newer; that's fine. If something is missing, the script says exactly what to run. Here's a real run on a computer where only one package had been installed:

```
python      3.11.15  OK
pandas      -         MISSING: run  python -m pip install pandas
openpyxl    3.1.5    OK
matplotlib  -         MISSING: run  python -m pip install matplotlib
jupyterlab  -         MISSING: run  python -m pip install jupyterlab
Fix the lines above, then run this script again.
```

Watch out: "pip installed it, but Python can't find it." This almost always means the package went into a different Python from the one running your script, often because the virtual environment wasn't active. Activate `.venv`, then install with `python -m pip install ...` rather than plain `pip install ...`, so the package goes into the Python you're actually using.

Step 4. VS Code

Install VS Code from its official website. Open it, go to the *Extensions* view, and install two extensions published by Microsoft: **Python** and **Jupyter**. Then use *File* → *Open Folder* to open your book folder.

Check: press `Ctrl+Shift+P` (`Cmd+Shift+P` on a Mac), type *Python: Select Interpreter*, and choose the one inside `.venv`. Create a file called `hello.py` containing `print("ready")`, and run it with the play button. The terminal at the bottom should print `ready`.

Step 5. Git

- **Windows:** install **Git for Windows** from the official Git website, accepting the default options.
- **Mac:** the Git website lists three ways; the simplest is to run `xcode-select --install` in Terminal, which installs Apple's command line tools, including Git. (Homebrew users can run `brew install git`.)
- **Linux:** install the `git` package from your distribution.

Check: in a terminal, run `git --version`. You'll see a version number such as `git version 2.55.0`. Chapter 26 teaches how to use it.

Step 6. Power BI Desktop (Windows)

On Windows, install **Power BI Desktop** from the **Microsoft Store**. Microsoft recommends the Store version because it updates itself and doesn't need administrator rights. Microsoft lists at least 2 GB of free memory (4 GB or more recommended) and a screen of at least 1440×900. Signing in matters later, when you publish and share reports through the Power BI service (Chapter 16).

On a Mac, skip this step for now, and decide before Chapter 16 whether to use a Windows virtual machine, a Windows computer at work or college, or to follow Chapter 16 by reading and doing its exercises on a borrowed machine.

Check: open Power BI Desktop, close the welcome screen, and you should see an empty report canvas.

6.4 The companion files and a folder that stays tidy

The **companion files** (Appendix E) hold every dataset, script, and workbook the book uses. So far that includes:

Folder or file	What it is	Used in
ch02/	four orders as CSV, Excel, JSON, XML, and Parquet; <code>api_demo.py</code>	Chapter 2
riverstone_setup.sql and mysql/riverstone_setup_mysql.sql	the mini database (first quarter of 2026)	Chapters 12, 13, and many examples
riverstone_2025_setup.sql and its MySQL twin	the one-year database (2025)	Chapter 13 onward
postgresql/ch12_lab_postgresql.sql , mysql/ch12_lab_mysql.sql , new_suppliers.csv	the build-a-database lab	Chapter 12
mysql/ch12_queries_mysql.sql , mysql/ch13_queries_mysql.sql	every query in Chapters 12 and 13, tested in MySQL	Chapters 12, 13
ch04/numbers_practice.xlsx	the numbers workbook	Chapter 4
ch06/check_setup.py	the setup checker	this chapter

Later chapters add their own folders. Appendix E gives the address to download them from and lists every file.

Set up one home for everything, and keep it that way:

```

analyst-to-architect/
  companion/           the downloaded companion files, unchanged
  work/
    ch10/              your own files for each chapter
    ch12/
    ...
  notes/              your study log, plans, and questions
  .venv/              the Python environment from section 6.3

```

Two rules keep it useful. **Never edit the companion files themselves**: copy what you need into `work/` first, so you can always start again from a clean copy. And **name your files the way Chapter 2 (section 2.4) taught**: dates as year-month-day, lower-case words joined with hyphens or underscores, and no "final".

6.5 Keyboard, files, and the terminal: the basics that save hours

Keyboard shortcuts worth learning this week

Action	Windows	Mac
Copy, cut, paste	Ctrl+C , Ctrl+X , Ctrl+V	Cmd+C , Cmd+X , Cmd+V
Undo, redo	Ctrl+Z , Ctrl+Y	Cmd+Z , Cmd+Shift+Z
Find in a page or file	Ctrl+F	Cmd+F
Save	Ctrl+S	Cmd+S
Switch between apps	Alt+Tab	Cmd+Tab
Open the search for apps and files	Windows key, then type	Cmd+Space , then type
Take a screenshot of part of the screen	Windows+Shift+S	Cmd+Shift+4
Command palette in VS Code	Ctrl+Shift+P	Cmd+Shift+P

Chapters 10, 12, and 17 add the shortcuts for spreadsheets, DBeaver, and VS Code. Learn a few at a time: use one new shortcut deliberately for a week, until your hands do it on their own.

Files

From Chapter 2, make sure you can: **show file extensions** (so `orders.csv` doesn't appear as plain `orders`); **find a file's full path** (Windows: hold `Shift`, right-click the file, *Copy as path*; Mac: select the file, then `Option+Cmd+C`); and **unzip** a download before opening what's inside it, because opening files directly from inside a zip is a common cause of "my changes disappeared".

The terminal

A **terminal** is a window where you type commands instead of clicking. You'll use it for Python, Git, and later for much more (Chapter 34 teaches it properly). For now, four things are enough:

To do this	Type	Example
Open a terminal	Windows: search for <i>Terminal</i> or <i>PowerShell</i> . Mac: search for <i>Terminal</i> . In VS Code: <i>Terminal</i> → <i>New Terminal</i> (opens in your current folder).	
See where you are	<code>pwd</code> (PowerShell, Mac, Linux); <code>cd</code> on its own in Command Prompt	
Move into a folder	<code>cd</code> followed by the folder name	<code>cd analyst-to-architect</code>
Run a Python script	<code>python</code> (Mac: <code>python3</code>) followed by the file name	<code>python check_setup.py</code>

The most common beginner error is running a command in the wrong folder, which gives "No such file or directory" or "can't open file". When you see that, check where you are first.

6.6 Reading documentation

Every tool in this book has **official documentation**: the manual written by the people who make it. When a tutorial, a colleague, and an AI assistant disagree, the documentation settles it.

A real question: how does 2.5 round?

Chapter 4 said to round at the end, not in the middle. Here's a follow-up question that sounds trivial: *what does rounding 2.5 to a whole number give?* The same question, in three of the book's tools.

In PostgreSQL:

```
SELECT ROUND(2.5)           AS exact_half,
       ROUND(3.5)           AS exact_three_half,
       ROUND(2.5::double precision) AS float_half,
       ROUND(3.5::double precision) AS float_three_half;
```

exact_half	exact_three_half	float_half	float_three_half
3	4	2	4

(1 row)

In MySQL:

```
SELECT ROUND(2.5) AS exact_half,
       ROUND(3.5) AS exact_three_half,
       ROUND(2.5E0) AS float_half,
       ROUND(3.5E0) AS float_three_half;
```

exact_half	exact_three_half	float_half	float_three_half
3	4	2	4

In Python:

```
print(round(2.5), round(3.5), round(2.675, 2))
```

```
2 4 2.67
```

And in a spreadsheet, `=ROUND(2.5,0)` gives 3 and `=ROUND(2.675,2)` gives 2.68 .

So 2.5 rounds to 3 in some places and to 2 in others, and 2.675 rounds to 2.68 in a spreadsheet but 2.67 in Python. Who's right? This is exactly the kind of question to settle in the documentation, not by guessing:

- **The PostgreSQL manual** (*Mathematical Functions and Operators*, `round`): "For *numeric* , ties are broken by rounding away from zero. For *double precision* , the tie-breaking behavior is platform dependent, but 'round to nearest even' is the most common rule." 2.5 typed on its own is *numeric* , so it goes to 3; converted to *double precision* , it goes to the even number, 2.
- **The MySQL manual** (`ROUND()`): exact-value numbers round "away from zero"; for approximate-value numbers, "the result depends on the C library", and on many systems that means "round to nearest even". 2.5E0 is written in scientific notation, which makes it an approximate value.
- **The Python documentation** (built-in `round`): "if two multiples are equally close, rounding is done toward the even choice". And a note: "`round(2.675, 2)` gives 2.67 instead of the expected 2.68 . This is not a bug": 2.675 can't be stored exactly as a floating-point number (Chapter 2, section 2.1), and the stored value is a hair below 2.675.

Rounding halves to the nearest even number is called **banker's rounding**, or round-half-to-even. It's used because, over many values, rounding every half up nudges totals upward, while rounding to even balances out. Neither rule is wrong. What's wrong is not knowing which one your tool uses, and then wondering why a Python report and an Excel report differ by a rupee.

How to read a documentation page

You don't read documentation front to back. You go in with a question and look for five things:

1. **The signature:** the name and what goes in, such as `ROUND(number, num_digits)` .
2. **The description:** what it returns, in one or two sentences. Read every word; the details (like "for *numeric* ") are usually the point.
3. **Examples:** run one yourself, exactly as written, before changing it.
4. **Notes and warnings:** often in a box. This is where `round(2.675, 2)` was explained.
5. **The version:** documentation sites usually cover several versions. Make sure you're reading the one you installed (for example, PostgreSQL's pages have a version switcher at the top).

If the documentation doesn't answer it, try the tool's official forums, then a well-asked search.

6.7 Learning with AI assistants without letting them think for you

AI assistants can explain a concept five different ways, spot the typo in a query, and describe what an error message means. Used well, they make you learn faster. Used badly, they let you finish every exercise without learning anything, and you find out in an interview, or in your first week at work, when the assistant isn't the one being asked.

Rules that keep the learning yours

1. **Try first.** Spend real effort on an exercise before asking for help: at least fifteen to twenty minutes, or until you can say exactly where you're stuck. The struggle is where the learning happens.
2. **Ask for explanations, not answers.** "Why does my query return 21 rows instead of 19?" teaches you something. "Write the query for exercise 14" teaches you nothing.
3. **Check everything against a source you trust.** Assistants can be fluent and wrong at the same time. Section 6.6's rounding question is a good test: ask an assistant how 2.5 rounds in Python, then check the documentation and run it yourself.
4. **Never paste data you don't own.** Company data, customer names, invoices, and passwords don't belong in a chat with an outside service unless your employer has explicitly approved that tool for that data (Chapter 2, section 2.9, and Chapter 64 go deeper). Practice data like Riverstone's is fine.
5. **Explain it back.** After the assistant helps, close the chat and explain the idea in your own words, or redo the exercise from a blank page. If you can't, you haven't learned it yet.
6. **Use it as a tutor, not a crutch.** Good requests: "Quiz me on percentage points versus percent change", "Give me a harder version of this exercise", "Explain this error message; don't fix it".

Chapter 26 covers AI assistants at work, and Chapters 54 and 55 explain how they work and how to build with them.

6.8 Building a study habit

The book is long. The difference between finishing and stopping is rarely intelligence; it's a plan, a rhythm, and a way of keeping going after a missed week.

A sample six-month plan

A sample six-month plan at about 8 hours a week

Month 1 Weeks 1-4	Part 0 and Part I: Chapters 1-9 foundations, the map, how learning works
Month 2 Weeks 5-8	Chapters 10-11: spreadsheets Excel and Google Sheets, pivot tables, Power Query
Month 3 Weeks 9-12	Chapters 12-13: SQL queries on the Riverstone database, in PostgreSQL and MySQL
Month 4 Weeks 13-17	Chapters 14-16: cleaning, charts, Power BI a first dashboard
Month 5 Weeks 18-22	Chapters 17-18 and 21-22: Python, statistics pandas, descriptive statistics, not fooling yourself
Month 6 Weeks 23-26	Chapters 19-20 and 23-27: automation, business, portfolio capstone project; start interview practice (Chapters 68-71)

Every week: one review session, the chapter exercises, and a little SQL or Python practice on something you've already learned.

Figure 6.2 — One way through the analyst path (Parts 0 to II) in six months, at about 8 hours a week.

Figure 6.2 is a sample, not a rule. It's ambitious: about 8 hours a week for 26 weeks. At 5 hours a week, stretch it to nine or ten months, and nothing is lost. A few choices behind it:

- **Spreadsheets before SQL, SQL before Python.** Each builds on the last, and together they cover most day-to-day analyst work.
- **Power BI in month 4**, once you can shape data, so the dashboards have something good to show.
- **Automation (Chapters 19-20) after Python**, because Chapter 20 uses it.
- **Interview practice in month 6, not at the end.** Answering questions from Part VIII (Chapters 68-71) while you build your portfolio shows you what to revise.
- **Business analysts** can swap some Python time for Chapters 24 and 25, which cover requirements, stakeholders, and process mapping.

A weekly rhythm

One week, about 8 hours

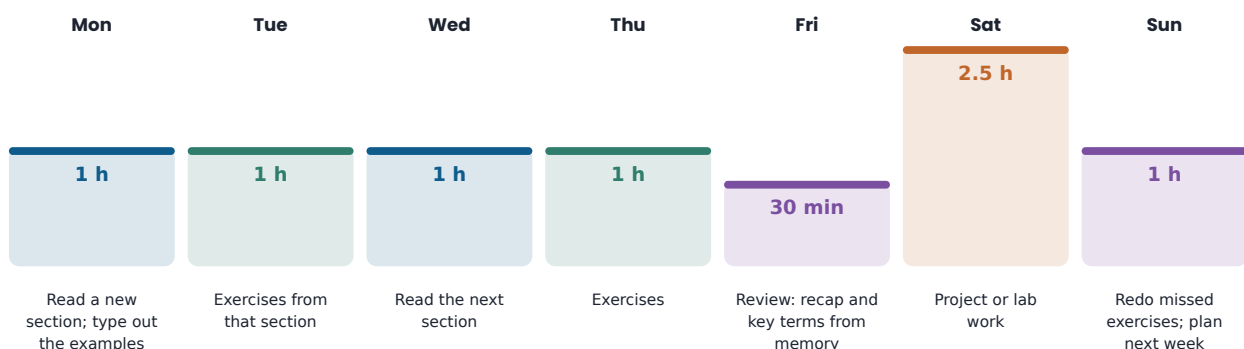


Figure 6.3 — Short, regular sessions, one longer session for projects, and a review built into the week.

Figure 6.3 adds up to 8 hours: $1 + 1 + 1 + 1 + 0.5 + 2.5 + 1$. Three ideas are built into it:

- **Alternate reading and doing.** A section read on Monday gets practiced on Tuesday, while it's fresh but no longer in front of you.
- **Review from memory.** On Friday, close the book and write down the chapter's key terms and main ideas before checking the recap. Pulling ideas out of your memory strengthens them far more than reading them again. Chapter 9 explains why this works and how expertise forms.
- **Plan at the end of the week.** Ten minutes on Sunday deciding exactly which sections you'll do on which days makes Monday simpler to start.

When you fall behind

You will miss weeks. Everyone does. When it happens, don't try to catch up by doubling the hours; that's how a missed week becomes a missed month. Instead: **move the plan back a week, do one short review session to reconnect, and start the next section.** Keep a one-line **study log** in your `notes/` folder (date, what you did, one thing you learned), so that when you return, you can see how far you've come.

6.9 How to use the exercises and answers

Every teaching chapter ends with practice exercises in four groups:

Group	What it's for	How to use it
Warm-up	checks you understood the basics	do them right after reading; they should take a few minutes each
Core	the skills the chapter exists to teach	do all of them; this is where most of the learning happens
Stretch	harder, closer to real work	do at least one; come back to the rest on a second pass
Think about it	judgment, with no single right answer	write a short answer, then compare it with the model answer

The answers are at the end of each chapter in this draft and move to **Appendix G** in the finished book. To get the most from them:

- **Write your answer before you look.** Even a partial answer. Reading an answer you haven't attempted feels like learning and isn't.
- **Compare the reasoning, not only the number.** Many answers show a method or a check (like " $\checkmark$ reconciles to the total"); copy the method.
- **Mark the ones you got wrong or guessed**, and redo them on Sunday from a blank page.
- **For code exercises, run your version.** If it gives the same result by a different route, that's often fine; if it gives a different result, find out why before moving on.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Downloading installers from third-party sites	unwanted extra software; outdated versions	Use each tool's official website or the Microsoft Store
Forgetting the database password	can't connect to PostgreSQL or MySQL	Write it down during installation, somewhere safe
Installing packages without the virtual environment active	"pip installed it, but Python can't find it"	Activate <code>.venv</code> ; use <code>python -m pip install ...</code>
Running commands in the wrong folder	"No such file or directory"	Check where you are (<code>pwd</code>) before running
Trusting a tutorial or AI over the documentation	results that differ between tools, unexplained	Check the official documentation for the version you installed
Letting an AI assistant do the exercises	you can follow solutions but can't write them	Try first; ask for explanations; redo from a blank page
Reading answers before attempting exercises	answers "make sense" but skills don't stick	Write something first, always

In the real world: Meera sets up

Meera Iyer has done analyst work for months without the title: the customer counts in Chapter 1, the January reconciliation in Chapter 3, the board slide in Chapter 4. She wants to become an analyst properly, and she has two computers to do it with. Neither is ideal.

Her work laptop runs Windows 11 and is locked: she can't install software. **Her home laptop** is a MacBook Air from 2020 with 8 GB of memory.

1. Deciding what goes where. She reads section 6.1 and makes a list. The Mac can run everything except Power BI Desktop. The work laptop is where Power BI would be most useful, since the company already uses Microsoft 365. So: the book's main setup goes on the Mac at home; Power BI Desktop goes on the work laptop, if IT agrees.

2. The work laptop. She emails IT with a short, specific request: *"I'd like to install Power BI Desktop from the Microsoft Store for learning and for building sales reports. It doesn't need admin rights. I won't connect it to any company data without approval."* IT approves the same week, and adds that she should use the company's Microsoft 365 account for sign-in when she's ready to share reports.

3. The Mac. She installs Postgres.app, DBeaver, MySQL, Python from python.org (remembering `Install Certificates.command`), VS Code with its Python and Jupyter extensions, and Git with `xcode-select --install`. She creates the `analyst-to-architect` folder, the `.venv`, and runs `check_setup.py`. It reports matplotlib missing: she had typed `matplotlab`. One command later: *All set*. In DBeaver, `SELECT COUNT(*) FROM order_items;` returns 19 in both databases.

4. The 8 GB question. Her Mac slows down with DBeaver, VS Code, twenty browser tabs, and a video call all open. The fix costs nothing: close what she isn't using while studying.

5. The plan. She has about 6 hours a week: an hour on three weekday evenings after work, and three hours on Sunday morning. At that pace, Figure 6.2 becomes a nine-month plan. She writes her first 90 days in `notes/plan.md`: Chapters 7 to 11 by the end of month 2, Chapters 12 and 13 in month 3. At work, she'll rebuild the January reconciliation from Chapter 3 in a spreadsheet as her month 2 practice project, using the practice data rather than company files, so nothing sensitive leaves the company's systems.

6. The AI rule. She sets herself one rule, taped to the edge of her screen: *"Try for 20 minutes. Ask why, not what. Never paste Riverstone's real data."*

She matched tools to computers, asked IT the right way, fixed the one problem the checker found, and wrote a plan for her real week, not an ideal one.

Tools

- **The official websites** for each tool: PostgreSQL, MySQL, DBeaver, python.org, VS Code, Git, and Microsoft for Excel and Power BI. Appendix B lists the current download pages and install steps.
 - `check_setup.py` (companion files, `ch06/`): checks your Python version and the four packages. Tested on Python 3.11.15 and 3.13.13, with pandas 3.0.5, openpyxl 3.1.5, matplotlib 3.11.2, and JupyterLab 4.6.3.
 - **A notes file** (`notes/plan.md` and a study log): a plain text or Markdown file is enough.
-

The project: set up, check, and plan your first 90 days

Goal: a working toolkit, verified, and a study plan you can keep.

Step 1. Install and check. Work through section 6.3. For each tool, record the version and the result of its check:

Tool	Version installed	Check	Result
Spreadsheet		<code>=ROUND(2.5,0)</code>	gives 3
PostgreSQL + DBeaver		<code>SELECT COUNT(*) FROM order_items;</code>	gives 19
MySQL (optional for now)		the same query	gives 19
Python + packages		<code>check_setup.py</code>	prints <i>All set.</i>
VS Code		<code>hello.py</code>	prints <code>ready</code>
Git		<code>git --version</code>	prints a version
Power BI Desktop (Windows)			opens to an empty report

Step 2. Organize. Create the folder structure from section 6.4 and put the companion files in `companion/`.

Step 3. Test the documentation habit. Pick one function you used in Chapter 4 (for example, `MEDIAN` or `RRI`) and find its official documentation page for your spreadsheet. Write down its signature, one example, and one note or warning.

Step 4. Write your 90-day plan in `notes/plan.md` :

- how many hours a week you can study, and on which days and times;
- which chapters you'll finish by day 30, day 60, and day 90 (use Figure 6.2 and adjust for your hours);
- one practice project for each month, using practice data;
- your rule for using AI assistants;
- what you'll do when you miss a week.

Step 5. Start your study log with today's entry.

Deliverable: the completed check table, a screenshot of `check_setup.py` printing *All set.*, and your 90-day plan.

You've got it when...

- ☐ I know whether my computer can run each tool, and I have a plan for any it can't.
- ☐ Every core tool is installed from its official source, and each check passed.
- ☐ `SELECT COUNT(*) FROM order_items;` returns 19 in the Riverstone database.
- ☐ `check_setup.py` prints *All set.*
- ☐ My companion files are in one place, unedited, with my own work in a separate folder.
- ☐ I can open a terminal, see where I am, move into a folder, and run a script.
- ☐ I can find and read the official documentation for a function in each tool.
- ☐ I have rules for using AI assistants that keep the learning mine, and I never paste data I don't own.
- ☐ I have a weekly rhythm and a written 90-day plan.
- ☐ I attempt exercises before reading the answers.

Recap

- **You don't need a high-end computer.** 8 GB of memory is enough to start, 16 GB is comfortable. Power BI Desktop is Windows only; Mac users can run Windows in a virtual machine. Chromebooks cover spreadsheets and online SQL, not Python or Power BI.
- **The core toolkit is free:** a spreadsheet, PostgreSQL and MySQL with DBeaver, Power BI Desktop, Python with VS Code and Jupyter, and Git. Later parts add their own tools.
- **Install in order, from official sources, and check each tool** before moving on. Chapter 12, section 12.3, has the full database steps.

- **Use a virtual environment** for the book's Python packages, and install with `python -m pip install . check_setup.py` confirms your setup.
- **Keep the companion files unedited** and your own work in a separate folder, with file names that sort and search well.
- **A few shortcuts and four terminal commands** save hours over the course of the book.
- **Documentation settles disagreements.** Rounding 2.5 gives 3 or 2 depending on the tool and the number type, and each tool's manual says exactly why.
- **AI assistants are tutors, not substitutes:** try first, ask why, check against sources, never paste data you don't own, and explain it back.
- **A plan and a rhythm finish books.** About 8 hours a week covers the analyst path in six months; fewer hours stretch it, and a missed week moves the plan instead of doubling the load.
- **Attempt every exercise before reading its answer**, and redo the ones you missed.

Practice exercises

Warm-up

1. For each computer, say which of the book's core tools it can run, and what the owner should do about the rest: (a) a Windows 11 laptop with 8 GB of memory; (b) a MacBook with 16 GB of memory; (c) a Chromebook; (d) a locked-down work laptop running Windows 11.
2. Match each check to the tool it confirms: (a) `git --version`; (b) `SELECT COUNT(*) FROM order_items`; returning 19; (c) `check_setup.py` printing *All set.*; (d) `=ROUND(2.5,0)` returning 3.
3. A friend's terminal shows `python: can't open file 'check_setup.py': No such file or directory`. List two likely causes and how to check each.
4. Sort these requests to an AI assistant into "helps you learn" or "does the learning for you", and rewrite the ones that don't help: (a) "Solve Chapter 4, exercise 6." (b) "Explain why averaging monthly growth rates overstates growth." (c) "My query returns 21 rows instead of 19. What kinds of mistake cause extra rows?" (d) "Write my 90-day plan."

Core

5. After installing pandas, a script fails with `ModuleNotFoundError: No module named 'pandas'`. Explain the most likely reason, and write the two commands (for your operating system) that would fix it.
6. Using the documentation quotes in section 6.6, predict the result of each, and explain your reasoning: (a) PostgreSQL `SELECT ROUND(4.5)`; (b) PostgreSQL `SELECT ROUND(4.5::double precision)`; (c) Python `round(4.5)` (d) a spreadsheet `=ROUND(4.5,0)` (e) MySQL `SELECT ROUND(45E-1)`;
7. A Python report and an Excel report of the same invoices show totals that differ by ₹1. Using section 6.6, give one possible explanation, and describe how you'd confirm it.

8. You can study 5 hours a week. Adapt Figure 6.3's weekly rhythm to 5 hours, keeping reading, exercises, review, and project work. Then estimate how long the six-month plan in Figure 6.2 would take at that pace.
9. Your company uses a customer list with names, phone numbers, and outstanding balances. You want an AI assistant's help writing a formula that flags overdue customers. Describe how you'd get the help without sharing the real data.

Stretch

- l0. Write a short, specific request to your IT team asking permission to install PostgreSQL, DBeaver, and Python on a work laptop for learning. Include what each tool is for, what data you will and won't connect to, and one question for them.
- l1. Find the official documentation page for Python's `round()` or PostgreSQL's `ROUND` for the version you installed. Write down its signature, one example you ran yourself, and one note or warning. Then find one tutorial or video that explains rounding, and say whether it matches the documentation.
- l2. Draft your 90-day plan using the checklist in the project, with specific chapters for days 30, 60, and 90, and a plan for a missed week.

Think about it (no calculation needed)

- l3. Why does this book recommend free tools, even though many companies use paid ones? What might you still need to learn on the job?
- l4. A friend says, "I'll skip the exercises and read the answers; it's faster." What would you tell them, using section 6.9?
- l5. When is it reasonable to ask an AI assistant for a complete answer, rather than an explanation? Give one example where it's fine and one where it isn't.

Key terms

operating system · RAM (memory) · virtual machine · installer · Microsoft Store · administrator (admin) rights · LTS (long-term support) · PostgreSQL · MySQL · DBeaver · Power BI Desktop · Python install manager · virtual environment · pip · package · VS Code · extension · Jupyter · Git · companion files · terminal · command · path · keyboard shortcut · official documentation · signature · banker's rounding (round half to even) · round half away from zero · floating-point number · AI assistant · study plan · study log · retrieval practice · warm-up, core, stretch exercises

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 7, The Data Landscape**, and **Chapter 8, The Career Tree**, show where the skills you're setting up for lead, and which roles use which tools.
 - **Chapter 9, How Expertise Actually Forms**, explains the learning science behind section 6.8: practice, review from memory, and feedback.
 - **Chapters 10 and 11** use your spreadsheet; **Chapters 12 and 13** use the databases and DBeaver you installed, starting with the Riverstone data you loaded.
 - **Chapter 16** uses Power BI Desktop; **Chapters 17 and 18** use Python, VS Code, and Jupyter, and explain virtual environments properly.
 - **Chapter 26** puts Git to work and covers AI assistants in a professional setting; **Chapter 34** teaches the command line in depth.
 - **Appendix B** keeps current install steps for every tool; **Appendix E** lists all companion files; **Appendix G** holds the answers.
 - **Interview preparation:** Chapter 81 (behavioral, HR, and offer conversations) covers how to talk about how you learned and what you've built, and Chapter 68 explains how data hiring works.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

- (a) Everything, including Power BI Desktop; 8 GB is enough to start, and closing unused apps helps. (b) Everything except Power BI Desktop; for Chapter 16, run Windows 11 in a virtual machine, or use a Windows computer at work or college. (c) Google Sheets or Excel for the web, and online SQL playgrounds for practice; plan to use a laptop before Chapter 17 for Python, the databases, and Power BI. (d) Nothing until IT approves; ask for Power BI Desktop from the Microsoft Store (no admin rights needed) and for PostgreSQL, DBeaver, and Python, and meanwhile use a home computer or the web tools with practice data.
- (a) Git. (b) The database and DBeaver (with the Riverstone data loaded). (c) Python and the four packages. (d) The spreadsheet.
- (1) The terminal is in a different folder from the script: check with `pwd`, then `cd` into the `ch06` folder. (2) The file name is different, such as `check_setup.py.txt` because extensions are hidden, or the file is still inside a zip: show file extensions (Chapter 2) and unzip the download.
- (a) Does the learning for you. Better: "I got a different answer for Chapter 4, exercise 6. Here's my working; where did my reasoning go wrong?" (b) Helps you learn. (c) Helps you learn. (d) Does the planning for you. Better: "Here's my draft 90-day plan and my available hours. What's unrealistic about it?"
5. pandas was installed into a different Python from the one running the script, usually because the virtual environment wasn't active. On Windows: `.venv\Scripts\activate`, then `python -m pip install pandas`. On macOS or Linux: `source .venv/bin/activate`, then `python -m pip install`

pandas . (Also make sure VS Code's selected interpreter is the one in .venv .)

6. (a) 5 : 4.5 is numeric , and PostgreSQL breaks ties away from zero. (b) 4 : as double precision , the common rule is round half to even, and 4 is even. (c) 4 : Python rounds halves to the even choice. (d) 5 : spreadsheets round halves away from zero. (e) 4 : 45E-1 is an approximate value, so on most systems MySQL rounds to the nearest even number. (All five were run for this chapter, with exactly these results.)

7. The two tools may round halves differently: Python's round() rounds halves to even (and floating-point values like 2.675 are stored slightly below the half), while Excel's ROUND rounds halves away from zero. If each invoice is rounded before the totals are added, small differences can add up to a rupee. To confirm: find invoices whose unrounded values end in exactly .5 of the rounding unit, compare each line's rounded value in both reports, and check whether the differences add up to ₹1. The lasting fix is to agree on one rule and round at the end (Chapter 4).

8. One way: Monday 1 hour reading, Wednesday 1 hour exercises, Friday 30 minutes review from memory, Saturday 2 hours project, Sunday 30 minutes redoing missed exercises and planning: $1 + 1 + 0.5 + 2 + 0.5 = 5$ hours. The six-month plan assumes about 8 hours a week for 26 weeks, roughly 208 hours; at 5 hours a week that's about 42 weeks, or around ten months.

9. Describe the structure, not the data: "I have a table with columns customer_name , due_date , and balance . How do I flag rows where the due date is before today and the balance is above zero?" Or build a small made-up sample (three fake customers with invented numbers) and share that. Test the formula on the fake data, then apply it to the real file on your own computer. If your company has approved an AI tool for internal data, follow its rules instead.

10. One good version: "Hello, I'd like permission to install three free tools on my work laptop for learning data analysis: PostgreSQL (a database, used to practice SQL on sample data), DBeaver Community (an app for writing SQL), and Python from python.org (for data analysis scripts). I'll only use practice datasets from a textbook, and I won't connect them to any company system or customer data without your approval. Could you tell me whether there's an approved way to connect to company data later, if my manager asks for reports?"

11. Answers vary. A complete answer includes the signature (for example, round(number, ndigits=None) in Python), an example the reader ran themselves, and the note about round(2.675, 2) or PostgreSQL's tie-breaking rule. Many tutorials say "Python rounds .5 up", which doesn't match the documentation; noticing that is the point of the exercise.

12. Answers vary. A good plan names specific hours ("Tuesday and Thursday, 8–9 p.m.; Saturday, 9–11 a.m."), specific chapters for each checkpoint, one project a month on practice data, an AI rule, and a concrete plan for a missed week ("move the plan back a week; do one review session; start the next section").

13. Nobody is locked out by cost, and PostgreSQL, MySQL, Python, and Git are widely used at work anyway. The ideas transfer: most SQL runs elsewhere, and Power BI's concepts map to Tableau and Looker (Chapter 16). On the job you may still need to learn a company's warehouse, BI tool, data definitions, and security rules.

14. Reading answers you haven't attempted feels like learning because the answers make sense, but it doesn't build the skill of producing them. The exercises are where most of the learning happens, especially the core group. Suggest a compromise: attempt every warm-up and core exercise, even partially, before reading its answer, and redo the missed ones at the end of the week.

15. It's reasonable when the task isn't the skill you're trying to learn, or when you already understand it and are saving time, and you'll check the result. Fine: asking for a list of keyboard shortcuts for DBeaver, or for a first draft of an IT request that you then edit. Not fine: asking for the answer to a SQL exercise while you're learning SQL, because producing that answer is exactly the skill you need.